ERMES 20.0.3 User Manual

Ruben Otin

United Kingdom Atomic Energy Authority
E-mail: ruben.otin@ukaea.uk
Web: ruben-otin.blogspot.com

Oxford (UK) - May 2026

Contents

Summary

ERMES 20.0 is an open-source C++ finite element software which solves the Maxwell's equations in frequency domain. This document is the user manual of ERMES 20.0 and it contains a step-by-step guide on how to install, run, and visualize results. The present document also includes a description of the graphical user interface, the C++ source code, and the finite element formulations implemented inside ERMES 20.0.

This document should be accompanied by two folders: `\ERMES_20.0.3`, which is the GiD problemtype user interface, and `\ERMES_CPP_20.0.3`, which contains the C++ source code. These folders also contain documents (e.g., additional manuals, scripts, templates, and examples) that are referenced throughout this user manual. The names ERMES, ERMES 20.0, and ERMES 20.0.3 are used interchangeably throughout this manual.

Chapter 1

Introduction

ERMES (Electric Regularized Maxwell's Equations with Singularities) is an open-source software which solves the Maxwell's equations in frequency domain with the Finite Element Method (FEM). The new ERMES version 20.0 presented in this document is an upgrade of the ERMES version 7.0 presented in [47]. New features, modules, and FEM formulations have been implemented inside ERMES 20.0 to deal with the challenging problems usually found in the design and analysis of nuclear fusion reactors [63].

ERMES 20.0 is available for Windows and Linux systems and it has improved its capabilities to solve large problems on High Performance Computing (HPC) infrastructures thanks to its new interface with the solver libraries PETSc [55] and Python NumPy [44]. As in previous versions, ERMES 20.0 features a graphical user-friendly interface integrated into the pre- and post- processor GiD [15]. GiD handles geometrical modelling, data input, meshing, and result visualization. ERMES 20.0 is licensed under the open-source software 2-clause BSD license and it is freely available to download on the developer's website [49].

New FEM formulations has been added to the regularized Maxwell's equations method already present in the previous versions [45, 50]. These new additions are: the double-curl edge element formulation [24], and a local L^2 projection method with nodal and bubble elements [17]. For all the formulations, an $\mathbf{A} - \phi$ potentials version has been included and, also stabilization terms with Lagrange multipliers which can be switched on and off (see Appendix B for details). The ample set of methods available in ERMES 20.0 allows the user to select the most suitable and stable formulation for each situation and generate the best possible conditioned matrix for each problem. These matrices can usually be solved efficiently with low-memory consuming iterative methods.

ERMES 20.0 operates in the static, quasi-static and the high-frequency regimes, making it a versatile tool which can be used in a wide variety of situations. For instance, it had been applied to microwave engineering [45], bioelectromagnetics [40, 51, 46], electromagnetic compatibility [53, 61], and electromagnetic forming [62, 48]. Now, thanks to the new electrostatic and cold plasma modules, the range of applications has been extended to relevant nuclear fusion engineering problems as: the computation of induced forces, plasma control, probability estimation of electric arc initiation, current distribution in arbitrary geometries, and the study of electromagnetic wave-plasma-wall interactions inside a fusion reactor [63, 65, 64, 60, 33].

1.1 User interface

ERMES 20.0 has a Graphical User Interface (GUI) based on GiD [15], which is an universal, adaptive and user-friendly tool for geometrical modelling, data input and visualization of results. GiD can be customized for being used with all types of numerical simulation programs. ERMES 20.0 uses GiD to create or import geometries, apply boundary conditions to surfaces, apply materials to volumes, set problem parameters, mesh, and visualize results. ERMES 20.0 interface is included in the problemtype folder `\ERMES_20.0`. ERMES-GiD workflow is as follows:

In GiD pre-processor, the user:

1. Creates a geometry on GiD or imports it from another CAD program,
2. Applies materials and boundary conditions on the CAD geometry,
3. Sets problem parameters (e.g., FEM formulation, frequency, solver type),
4. Generates a mesh clicking on the *Mesh* button,
5. Clicks on the *Calculate* button, and finally,
6. GiD writes and saves ERMES 20.0 input files on the problem folder 3.9.

ERMES 20.0 is called from the GiD pre-processor, and then ERMES 20.0:

1. Reads the input files saved on the problem folder,
2. Calculates, and
3. Writes the results file on the problem folder.

In GiD post-processor, the user:

1. Opens ERMES 20.0 results file, and
2. Visualizes and analyses the results.

GiD simplifies the process of problem preparation and results visualization and it is the recommended pre- and post- processor tool for ERMES 20.0. Nevertheless, it is also possible to use a different pre- and post- processor software.

If a pre-processor different from GiD is preferred, the only essential condition is to write the input files of ERMES 20.0 in the same format as stated in the `.bas` templates 3.9.1. These templates can be found in the GiD problem-type folder `\ERMES_20.0\ERMES.gid`. The `.bas` templates get the information from GiD and write the input files `.dat` in the format required by ERMES 3.9.2. The input files `.dat` can be edited before calling ERMES 20.0 without having to use or opening GiD, which is also very useful for parametric batch calculations. In the GiD problemtype folder `\ERMES_20.0\Documents`, there are several batch and Python scripting examples that can be useful for automating tasks.

If a post-processor different from GiD is preferred, ERMES 20.0 can generate results files with different formats that can be edited to meet the needs of any other post-processor.

1.2 C++ source code

The ERMES 20.0 C++ source code, required external libraries, and license information are inside the folder `\ERMES_CPP_20.0` organized in eight different subfolders as described in the following:

1. **`\ERMES`** : contains the finite element formulations, boundary conditions, and source elements. This folder also contains Gaussian integral tables and functions to write and read external files.
2. **`\external _libraries`**: includes the libraries boost, gidpost, and dirent. It also contains the software required to generate Linux makefiles from a Microsoft Visual Studio 2022 solution in the folder `\cbp2make`, and the programs required to generate the files `input_c_parser.cpp` and `input_c_scanner.cpp` from the template scripts `input_c_parser.y` and `input_c_scanner.l` in the folder`\flex++bison++`.
3. **`\includes`** : contains all the C++ objects template definitions and header files.
4. **`\license`** : contains the details of the ERMES 20.0 open-source license.
5. **`\linear _solvers`**: houses the internal iterative linear solvers used by ERMES 20.0.
6. **`\sources`** : contains the main C++ objects of ERMES 20.0. The file `modeler.cpp` is where the solving strategies are implemented, the system matrix is build, discontinuities and singularities are treated and the geometrical normals are calculated. The file `kernel.cpp` connects the input data with the objects implemented in `modeler.cpp`. The files `input_c_parser.cpp` and `input_c_scanner.cpp` read the input files generated by the GiD pre-processor and give the data to the C++ objects implemented in `kernel.cpp` and `modeler.cpp`. The files `input_c_parser.cpp` and `input_c_scanner.cpp` are automatically generated with the programs flex++ and bison++ using the scripts `input_c_parser.y` and `input_c_scanner.l` as templates. Follow the steps given in `\external_libraries\flex++bison++\README.txt` to generate and include `input_c_parser.cpp` and `input_c_scanner.cpp` in ERMES 20.0. The file `gid_output.cpp` writes the calculated results in GiD post-processor format. The file `gid_output.cpp` uses the gidpost library located in `\external_libraries\gidpost`.
7. **`\unix`** : includes the scripts required to compile ERMES 20.0 for Linux.
8. **`\windows`** : contains the scripts required to compile ERMES 20.0 for Windows.

Microsoft Visual Studio 2022 [1], which is free for open-source code development, is the recommended tool to compile ERMES 20.0 for Windows and to navigate its source code. Open the file `\windows\ERMES.sln` with Microsoft Visual Studio 2022 to read the source code, load project settings, and set compiler configuration parameters. The current ERMES 20.0 project configuration parameters are shown in the Microsoft Visual Studio 2022 window of Figure 1.1. Further instructions to compile for Windows can be found in the file `\windows\README.txt`.

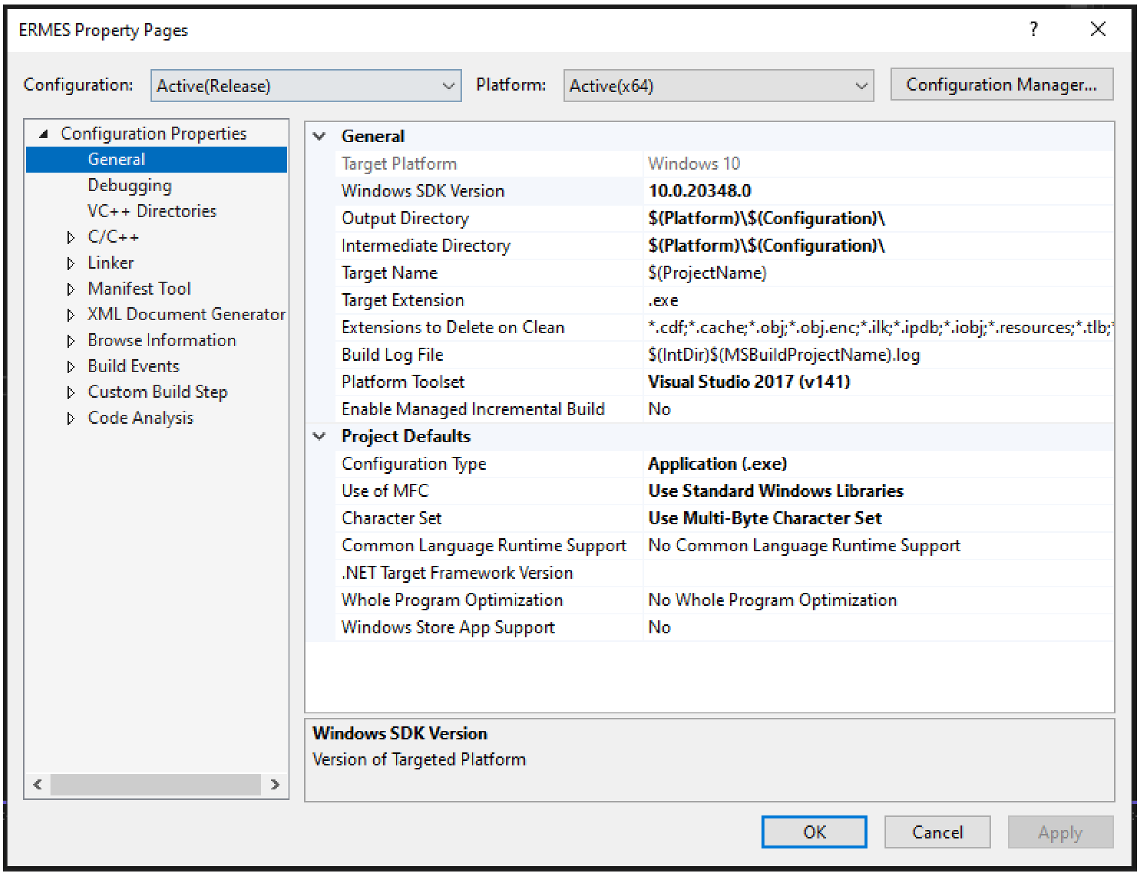

Figure 1.1: Microsoft Visual Studio 2022 project properties window for ERMES 20.0.

To compile ERMES 20.0 for Linux, simply type the command *make* in a Linux console inside the folder `\unix`. Further instructions for Linux compilation and how to generate the `makefile` can be found in `\unix\README.txt`.

1.3 License

ERMES 20.0 source code, binary files and interface are under the following open-source 2-clause BSD license:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

This software is provided by the copyright holders and contributors "as is" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the copyright holder or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage.

Chapter 2

Installation

This chapter is a step-by-step guide on how to install the graphical user interface GiD, the finite element computational tool ERMES 20.0 and the external solver library PETSc.

2.1 Graphical user interface

The recommended pre- and post- processor tool for ERMES 20.0 is GiD [15]. To install GiD, download the last version from `https://www.gidsimulation.com` and follow the instructions provided on that same webpage. After finishing the installation, open GiD, and click on the upper menu: **Help** → **Register GiD**, as shown in Figure 2.1:

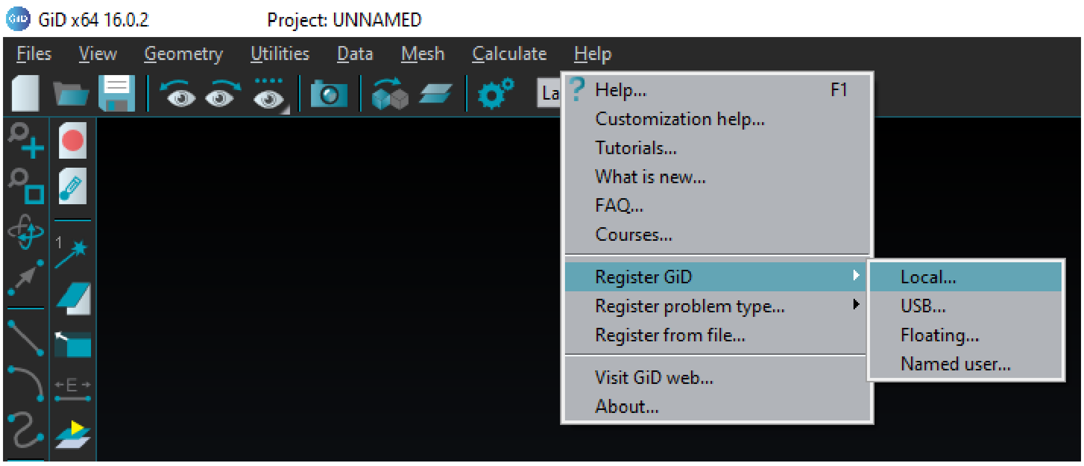

Figure 2.1: Register GiD drop-down menu.

Select **Local...** to register the local computer, **USB...** to register a USB stick attached to the local computer, **Floating...** to register a local network, or **Named user...** to attach the license to an email account. After clicking one of these options, a new window will appear with information related to the method selected and a link to obtain the password for a full professional license. Follow the link to get a one month free password or a permanent licence. The one month free license can be renewed up to three times for the same computer, USB stick, or email account. If the registration has been successful, the icon on the upper right corner of GiD will change colour as shown in Figure 2.2:

Figure 2.2: GiD upper right icon indicating a successful registration.

If a different pre- and post- processor software is preferred, the input files required by ERMES 20.0, and the output files it generates, can be edited and customized to meet the needs of any other pre- and post- processor tool (see Chapters 3 and 4 for a description of these input and output files).

2.2 ERMES 20.0

If GiD is the chosen pre- and post- processor, the installation of ERMES 20.0 is very simple. The folder `\ERMES_20.0` is a problemtype of GiD. Therefore, the installation process consists in *Copy&Paste* the whole folder `\ERMES_20.0` inside the `\problemtypes` folder of GiD (located, for instance, in `C:\MySoftware\GiD 16.0.2\problemtypes`). The same installation process applies to Windows and Linux.

To check that the installation has been done correctly, re-start GiD, and click on the GiD upper menu: **Data** → **Problem type**. ERMES 20.0 must appear in the drop-down menu as shown in Figure 2.3:

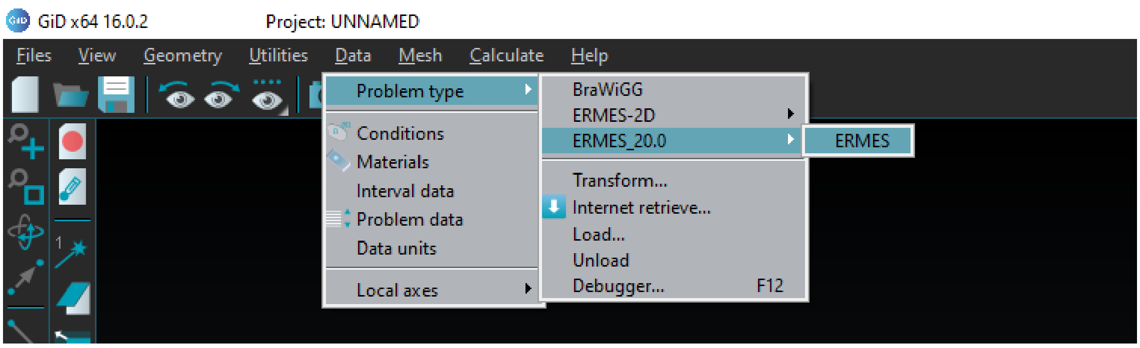

Figure 2.3: GiD **Data** → **Problem type** drop-down menu showing ERMES 20.0 problemtype. Click on the ERMES submenu **Data** → **Problem type** → **ERMES_20.0** → **ERMES** to start ERMES 20.0.

Click on the ERMES submenu **Data** → **Problem type** → **ERMES_20.0** → **ERMES** and the ERMES logo, together with the ERMES menu bar, must appear as shown in Figure 2.4, indicating a successful installation of ERMES 20.0. If you open a GiD problem created previously with ERMES 20.0, you can convert it to the latest version (e.g., ERMES 20.0.3) by selecting ERMES 20.0.3 on the above submenu and choosing **Transform to new problem type**. The option **Reset to new problem type** must be selected only when the differences between versions are significant.

Figure 2.4: ERMES 20.0 starting logo and menu bar.

If the ERMES 20.0 C++ source code needs to be recompiled (e.g. because it has been modified to add a new feature or because the local Linux distribution is not compatible with the distribution used to compile the executable provided in `\ERMES_20.0\ERMES.gid\execs\unix`), follow the instructions in `\ERMES_CPP_20.0\windows\README.txt` to compile ERMES for Windows, and the instructions in `\ERMES_CPP_20.0\unix\README.txt` to compile ERMES for Linux.

If the ERMES 20.0 C++ source code is recompiled, verify that the ERMES version called by the `ERMES.unix.bat` script for Linux or the `ERMES.win.bat` script for Windows matches the version you intend to use. These scripts, located in the GiD problem type folder `\ERMES_20.0\ERMES.gid`, are responsible for launching ERMES when the *Calculate* button is clicked in GiD. They can also be modified to include additional program calls, delete files, or perform other custom tasks. For Linux systems, ensure that execution permissions are granted to the script `ERMES.unix.bat` and any executables it references.

2.3 PETSc solvers library

Inside ERMES 20.0 there are implemented two iterative solvers (the bi-conjugate gradient and the transpose-free quasi-minimal residual) with diagonal preconditioning and OpenMP parallelization. These solvers are low-memory consuming and very efficient for well-conditioned matrices but, they can be insufficient for large, ill-conditioned linear systems. The recommended solver library for large complex linear systems is PETSc [55], which offers access to direct and iterative solvers with several preconditioning options and MPI parallelization. PETSc can be installed in Windows and Linux. We recommend the installation in Linux and, if required in Windows, call PETSc from the Windows Subsystem for Linux 2 (WSL2). In the problemtype folder `\External_Solvers\PETSc` there is a bash file called `win2wsl.sh` that can be used as a template to call PETSc from Windows using WSL2. To install PETSc in Linux, follow these steps:

1. Download PETSc from its git repository to the local folder `/petsc`:

```
>> git clone -b release https://gitlab.com/petsc/petsc.git petsc
```

2. Configure PETSc using the command `./configure` inside the folder `/petsc` (check PETSc manual for configuration parameters definitions). Note that multiple configurations can be installed on the same machine, for instance:

```
>> ./configure PETSC_ARCH=arch-complex-M1 --with-debugging=0
   --download-mumps --download-scalapack --download-parmetis
   --download-metis --download-ptscotch --with-64-bit-indices=1
   --with-scalar-type=complex --download-mpich --download-cmake
   --with-openmp --download-hwloc --with-cc=gcc --with-cxx=g++
   --with-fc=gfortran --download-fblaslapack --download-bison
   --download-make

>> ./configure PETSC_ARCH=arch-complex-S1 --with-debugging=0
   --download-superlu_dist --download-parmetis --download-metis
   --download-ptscotch --with-64-bit-indices=1
   --with-scalar-type=complex --download-mpich --download-cmake
   --download-make --download-bison --with-cc=gcc --with-cxx=g++
   --with-fc=gfortran --download-fblaslapack
```

3. Follow the instructions provided during the configuration process.

4. Add paths and preferred PETSC_ARCH configuration to `.bashrc`:

```
>> export PETSC_DIR=$HOME/petsc
>> export PETSC_ARCH=arch-complex-M1
>> export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:
          $PETSC_DIR/$PETSC_ARCH/lib
```

5. Reload `.bashrc` (or exit and log back in to your account):

```
>> source $HOME/.bashrc
```

6. Compile `PETScSolver.cpp` for the selected configuration by typing inside its folder location (e.g. `\External_Solvers\PETSc\Solver`):

```
>> make PETScSolver
```

7. To run PETSc, use the Python scripts examples provided in the folder `\External_Solvers\PETSc`, or the batch files examples in `\Documents\Batch_Scripts`, or call PETSc directly from the command line using, for instance:

```
>> mpiexec -n 6 ./PETScSolver -ksp_type bcgs -pc_type asm
   -ksp_converged_reason -ksp_monitor_true_residual
   -log_view -memory_view

>> mpiexec -n 8 ./PETScSolver -ksp_type richardson -pc_type lu
   -pc_factor_mat_solver_type mumps -ksp_max_it 1
   -log_view -ksp_error_if_not_converge

>> mpiexec -n 4 ./PETScSolver -ksp_type gmres -pc_type lu
   -pc_factor_mat_solver_type mumps -ksp_max_it 1
   -ksp_monitor_true_residual

>> mpiexec -n 8 ./PETScSolver -ksp_type richardson -pc_type lu
   -pc_factor_mat_solver_type superlu_dist -ksp_max_it 1
   -log_view -malloc_view
```

The above installation process can change depending on the Linux distribution. For further information, check the PETSc manual in `\External_Solvers\PETSc` or visit the webpage: `https://petsc.org`.

If a different solver library is preferred (as, for instance, Python NumPy [44]), check the Python scripts in `\External_Solvers\Python` and `\External_Solvers\PETSc` for examples of how to read the linear system produced by ERMES 20.0 and how to transform the matrix and vectors into different formats.

Chapter 3

Pre-process

This chapter provides a step-by-step guide for preparing a simulation in ERMES 20.0. Before initiating the simulation by clicking the *Calculate* button, the following preparatory steps must be completed:

1. Create (or import) a geometry.
2. Define and apply materials to the volumes of the geometry.
3. Define and apply sources to the volumes and surfaces of the geometry.
4. Define and apply boundary conditions to the surfaces of the geometry.
5. Set problem parameters (e.g., frequency, FEM formulation, solver type).
6. Select the results to calculate, export, and display in post-process.
7. Mesh the geometry.

The geometry can be created inside GiD or imported from a different CAD tool. The materials, sources, boundary conditions, and problem parameters can be defined and assigned from the windows associated to the ERMES menu bar shown in Figure 3.4. All the windows associated to the ERMES menu bar of Figure 3.4 will be described in this chapter. Once the problem is set up properly, it will be only necessary to mesh the geometry (clicking on the *Generate mesh* button of Figure 3.4) and, finally, call to execute ERMES 20.0 by clicking on the *Calculate* button of Figure 3.4.

The last sections of this chapter will be dedicated to describe the format of the input files required to run ERMES 20.0, how to call ERMES 20.0 from bash scripts to perform batch calculations, and how to use the external solvers libraries.

3.1 Geometry

The first step in the preparation of a simulation for ERMES 20.0 is to have a CAD geometry that represents the problem under study. The CAD model can be created inside GiD or imported from a different CAD tool.

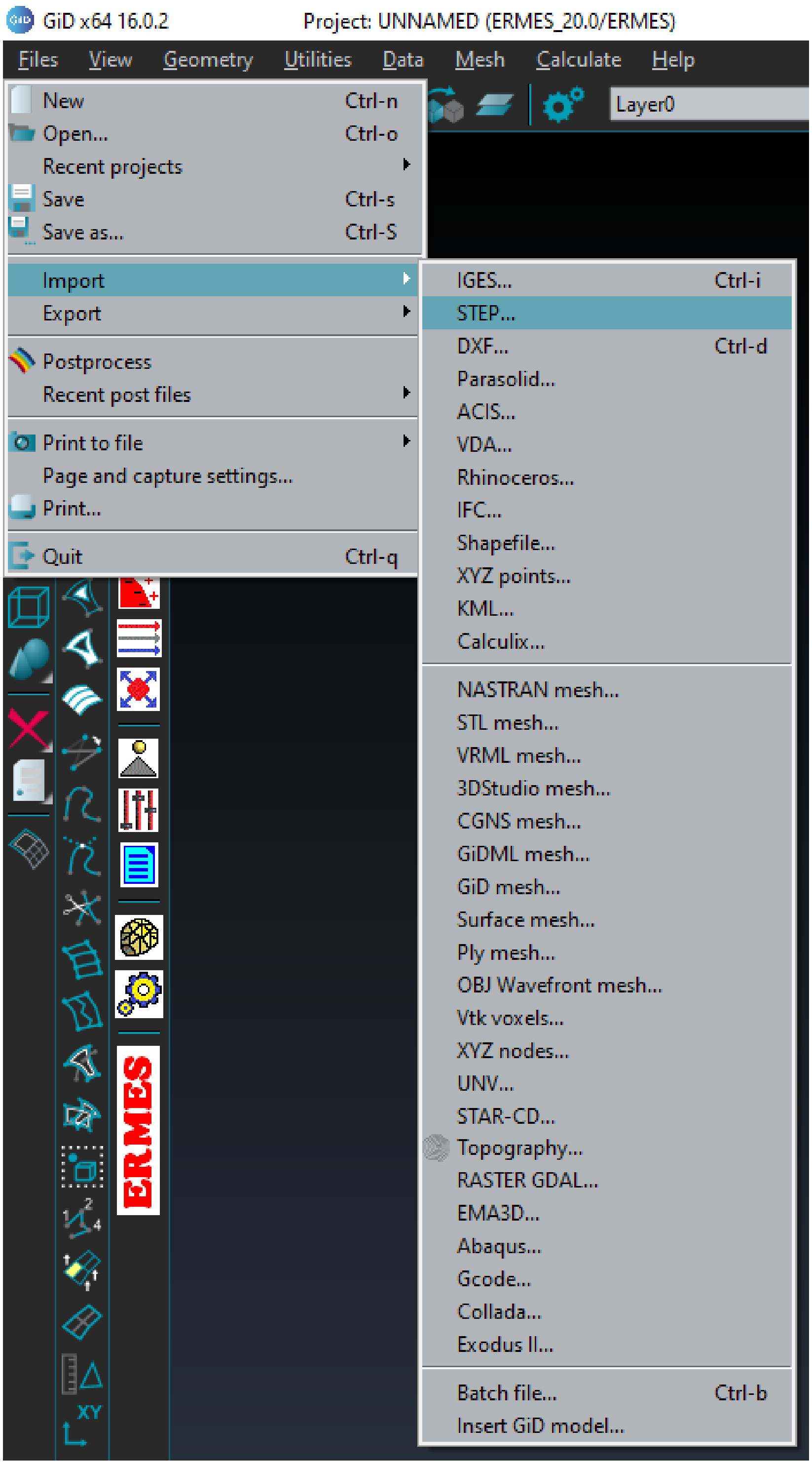

Figure 3.1: Supported CAD import formats in GiD.

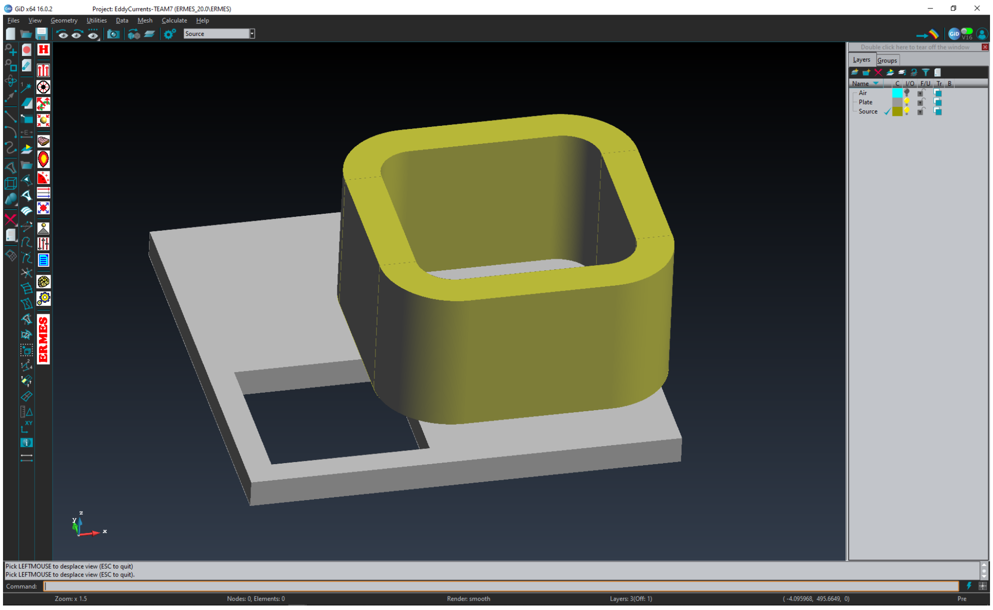

Figure 3.2: CAD geometry of a coil and an asymmetrical conductive plate with a hole from the TEAM benchmark problem 7 (see description and results in [27]). The geometry is shown using **Render** → **Flat mode** from the right mouse button contextual menu.

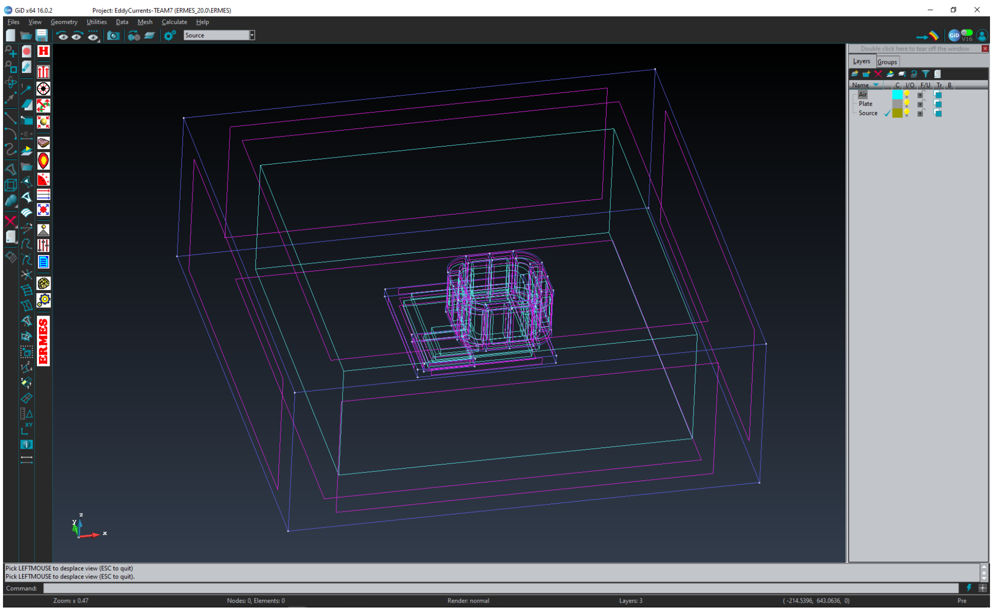

Figure 3.3: Points (white), lines (blue), surfaces (purple) and, volumes (blue) of the geometry shown in Figure 3.2 with an added volume of air surrounding the coil and plate. The geometry is shown using **Render** → **Normal mode** from the right mouse button contextual menu.

To create a CAD geometry inside GiD, it is advisable to follow first the tutorials provided in the GiD upper menu **Help** → **Tutorials...** or the tutorials and manuals available to download in `https://www.gidsimulation.com/gid-for-science/support/manuals/`.

To import a geometry from a different CAD tool, go to the GiD upper menu and select **Files** → **Import**. Next, choose the appropriate format from the available options (see Figure 3.1).

After creating (or importing) the geometry, we must have a set of points, lines, surfaces, and volumes in where the materials, sources, and boundary conditions will be applied (see Figures 3.2 and 3.3). The original geometry may need to be cleaned, repaired, or modified. For instance, we may need to add a volume to include the surrounding air, generate contact elements to allow nodal fields discontinuities (see Section B.3.1), or create separate contact volumes to define periodic boundary conditions. All these actions can be performed using GiD (check GiD tutorials and manuals for further information).

3.2 ERMES menu bar

All the input parameters required to run a simulation with ERMES 20.0 can be accessed from the menu bar shown in Figure 3.4. It is advisable to go from top to bottom before starting a simulation, checking that all the necessary parameters have been defined and applied correctly. By clicking on the icons of Figure 3.4, you will have access to the following:

1. **ERMES user manual**: opens the user manual stored in `\Documents\Manual`. It may be necessary to set a PDF reader in your system to open it automatically.
2. **Rectangular waveguide port parameters**: definition of the ports ID number, input power of the TE10 modes, location, and size of the rectangular waveguide ports.
3. **Coaxial waveguide port parameters**: definition of the ports ID number, input power of the TEM modes, location, and size of the coaxial waveguide ports.
4. **Robin conditions coefficients**: definition of the input parameters for plane waves and Gaussian beams, and definition of the coefficients for generic quasi-static and electrostatic Robin boundary conditions.
5. **Current density properties**: definition of the module and phase of Cartesian, axisymmetric, helicoidal, and plasma current densities.
6. **IHL material properties**: definition of the complex electromagnetic properties of Isotropic Homogeneous Linear (IHL) materials (conductivity, permittivity, and permeability) and application to volumes.
7. **Cold plasma properties**: definition of cold plasma properties (i.e., geometry, paths to external files containing measured electron density and magnetic field, plasma composition, damping modes, and numerical tuning parameters) and application to volumes.
8. **Dirichlet boundary conditions**: application to boundary surfaces, lines, or points of electric voltages, Perfect Electric Conductor (PEC) boundary conditions, Perfect Magnetic Conductor (PMC) boundary conditions, Periodic Boundary Conditions (PBC), Transversal Electric Conditions (TEC), and singularity layers.

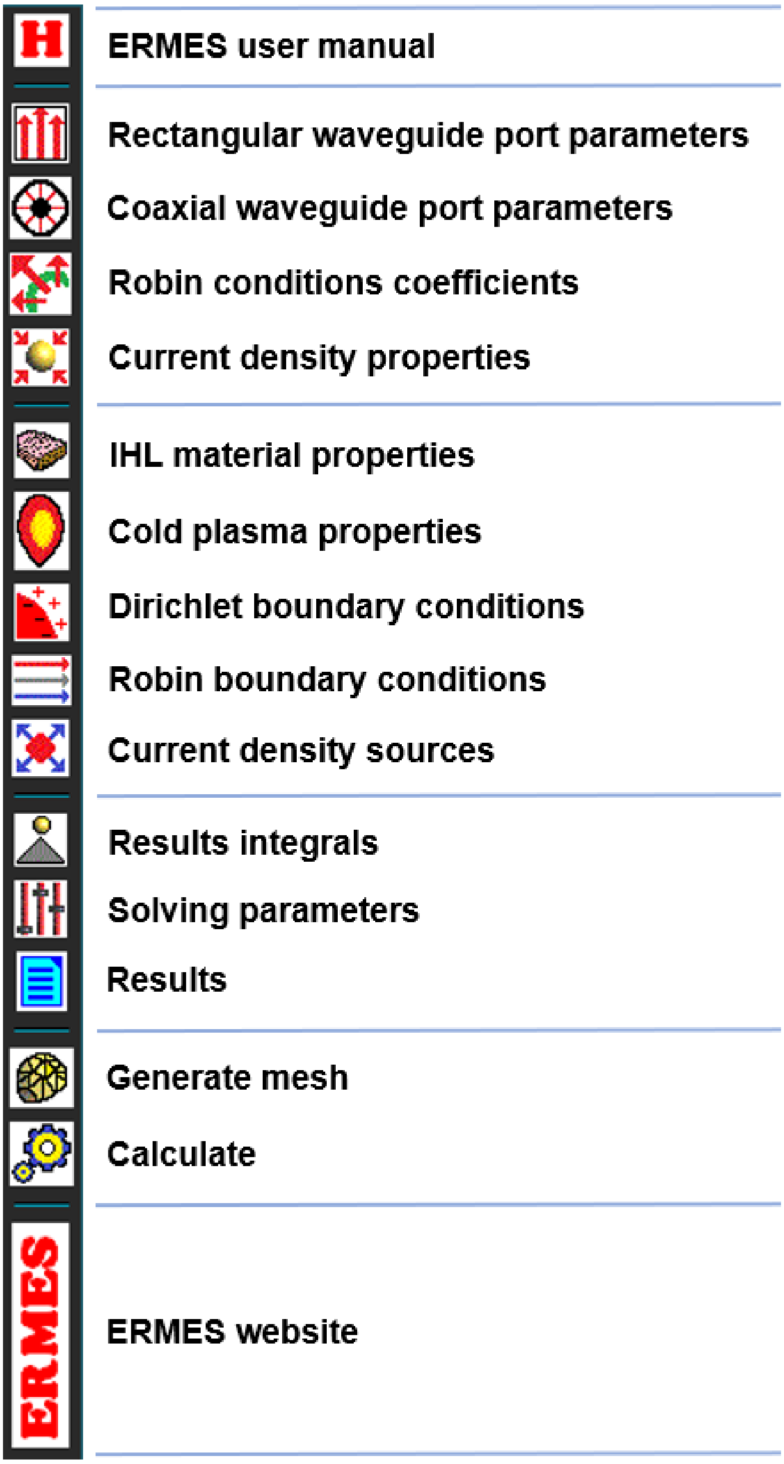

Figure 3.4: ERMES menu bar.

9. **Robin boundary conditions**: application to surfaces of far field boundary conditions, Robin boundary conditions, rectangular waveguide ports, and coaxial waveguide ports.
10. **Current density sources**: application to volumes of electric current densities.
11. **Results integrals**: selection of surfaces and volumes in where the result fields are going to be integrated to calculate average fields, total power, total current, and scattering parameters.
12. **Solving parameters**: configuration of the global problem parameters (e.g., FEM formulation, stabilization mode, frequency, solver type, output file format).
13. **Results**: selection of the results that are going to be stored in files and visualized in post-process.
14. **Generate mesh**: generates a mesh of tetrahedral elements on the geometry.
15. **Calculate**: writes the ERMES 20.0 input files and calls ERMES 20.0 to run the simulation.
16. **ERMES website**: link to the last version of ERMES, manuals, documents, source code, executables, and simulation examples.

3.3 Materials

In ERMES 20.0 there are implemented two types of materials:

1. Isotropic Homogeneous Linear (IHL) materials,
2. Cold plasma (inhomogeneous and gyrotropic).

Material properties are defined in the windows *IHL materials* and *Plasma*, which can be open by clicking on the icons *IHL material properties* and *Cold plasma properties* of the menu bar shown in Figure 3.4. Materials are assigned to volumes using the **Assign** button located in the same windows.

To assign a material, first click on **Assign** → **Volumes**. Then, select a volume as shown in Figure 3.5. To confirm the application, click on the central mouse button, the keyboard key *Escape*, or the *Finish* button. To check if the material has been correctly assigned, click on **Draw** → **All Materials** (located in the same material properties window). If properly assigned, the selected volume will appear in a solid colour as shown in Figure 3.6.

It is important to remember that all the volumes in the simulation domain must have a material assigned. If a volume does not have a material assigned, ERMES 20.0 will rise an error.

To save the material data, create new materials, delete materials, rename, or get help, click on the buttons located at the upper part of the material property window. A short description of the button functionality will appear when hovering the mouse pointer above the button. Clicking on the help button and hovering above a material property will give a short description of that property.

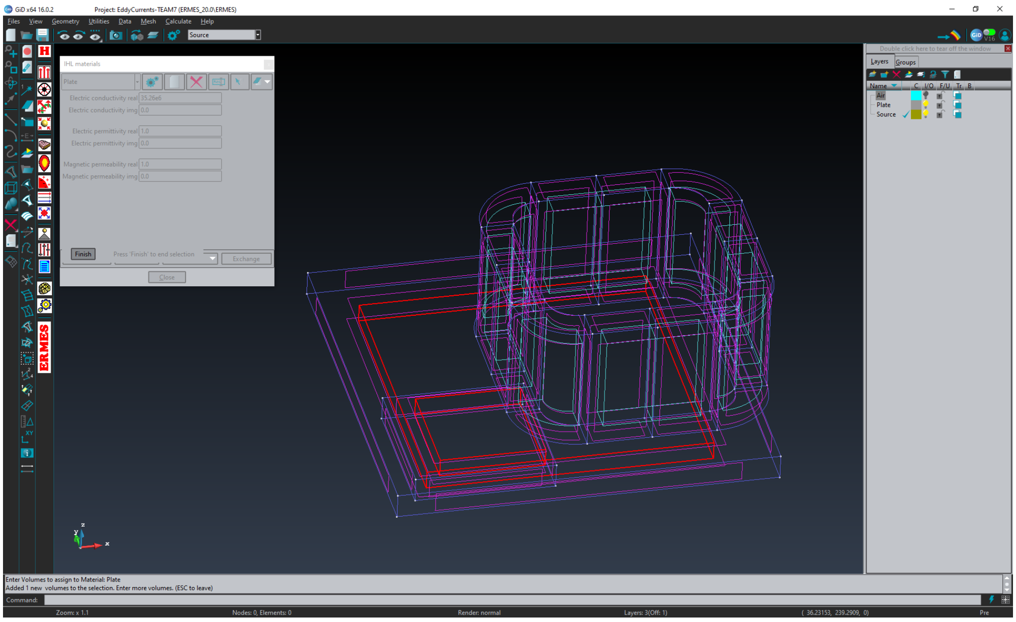

Figure 3.5: To assign a material, first click on **Assign** → **Volumes** (located in the same material properties window). Then, select a volume. The selected volume will appear in red. Then, click on the central mouse button, the keyboard key *Escape*, or the *Finish* button to confirm the application.

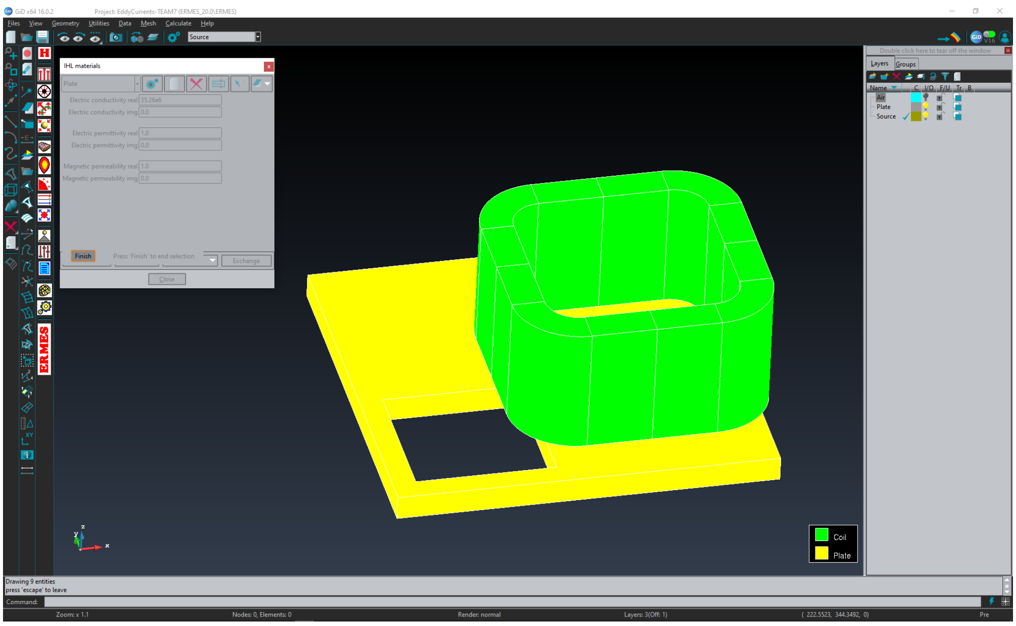

Figure 3.6: To check if the materials has been correctly assigned, click on **Draw** → **All Materials** (located in the same material window). The volumes will appear in solid colours if properly assigned.

3.3.1 IHL materials

Isotropic Homogeneous Linear (IHL) material properties are defined in the window *IHL materials* shown in Figure 3.7, which can be open from the icon *IHL material properties* of the menu bar of Figure 3.4. IHL materials are assigned to volumes using the **Assign** button located in the same *IHL materials* window (see button on the lower left corner of Figure 3.7). In ERMES 20.0, IHL materials are characterized by a complex permittivity ε and a complex permeability μ [57, 23]:

$$\varepsilon = \epsilon^{'} - \frac{\sigma^{''}}{\omega} + i\left(\epsilon^{''} + \frac{\sigma^{'}}{\omega}\right) \tag{3.1}$$

$$\mu = \mu^{'} + i\mu^{''} \tag{3.2}$$

where:

- $\sigma^{'}$: real part of the electric conductivity,
- $\sigma^{''}$: imaginary part of the electric conductivity,

- $\epsilon^{'}$: real part of the electric permittivity,
- $\epsilon^{''}$: imaginary part of the electric permittivity,

- $\mu^{'}$: real part of the magnetic permeability,
- $\mu^{''}$: imaginary part of the magnetic permeability.

These properties are related to the magnitudes in Figure 3.7 as follows:

- **Electric conductivity real**: $\sigma^{'}$ (S/m),
- **Electric conductivity img** : $\sigma^{''}$ (S/m),

- **Electric permittivity real**: $\epsilon_r^{'}$ $(\epsilon^{'}/\epsilon_0)$,
- **Electric permittivity img** : $\epsilon_r^{''}$ $(\epsilon^{''}/\epsilon_0)$,

- **Magnetic permeability real**: $\mu_r^{'}$ $(\mu^{'}/\mu_0)$,
- **Magnetic permeability img** : $\mu_r^{''}$ $(\mu^{''}/\mu_0)$.

where ϵ_r is the electric permittivity of the IHL material relative to the electric permittivity of vacuum ϵ_0 and μ_r is the magnetic permeability of the IHL material relative to magnetic permeability of vacuum μ_0.

3.3.2 Cold plasma

Cold plasma properties are defined in the *Plasma* window shown in Figures 3.8, 3.9, and 3.10, which can be open by clicking on the icon *Cold plasma properties* of the menu bar shown in Figure 3.4. Cold plasma materials are assigned to volumes using the **Assign** button located in the same *Plasma* window. Cold plasma materials will be only considered if the *Cold plasma* problem mode is active (see Section 3.6), otherwise the volume will be considered the same as vacuum.

Only one cold plasma material can be defined per simulation and almost one of the problem domain volumes must be assigned to this cold plasma material. If a volume has an IHL material assigned with the same properties as vacuum then, that volume will be considered part of the cold plasma. On the other hand, if the IHL material assigned has different properties to vacuum (even tiny differences) then, that volume will be considered an IHL material and not part of the cold plasma.

Applying different IHL materials to distinct volumes can be helpful to visualize the results in GiD post-process. Distinct volumes with different IHL materials assigned will be stored in separated layers by the GiD post-process. This separation happens even when the different materials have the same properties, or even when they have the same properties as vacuum and, therefore, are considered part of the cold plasma. The separation in layers makes easier the visualization of results because each layer can be displayed independently, which prevents one volume from blocking the view of another.

The electron density distribution and the applied static **B** field are obtained from data files. The format of these data files will depend on the *Geometry extrapolation* mode selected (see Figure 3.8 and its description in the next subsection). Examples of the different file formats can be found in the problemtype folder `\Documents\Plasma_Files` or in Section 3.9.3.

For the 1D line file format, as a general rule, if the line starts with a number then, that line will be read and processed by ERMES. On the other hand, if a line starts with anything different from a number (e.g., a blank space, the symbol #) then, that line will be considered a comment line and it will be ignored by ERMES. The numerical data columns must be separated by one or more blank spaces and the main precaution when writing the files is to avoid blank spaces at the beginning of a numerical data line (as mentioned before, a blank space at the beginning of a line will be interpreted as if the line is a comment).

For the 3D file format, the folder `\Documents\Plasma_Files` contains practical examples of Python scripts that can be used to transform *eqdsk* equilibrium plasma files into the format required by ERMES 20.0.

The *Plasma* window is divided in three tabs: *Plasma geometry*, *Species densities*, and *Problem settings*. The *Plasma geometry* tab is shown in Figure 3.8 and it used for specifying plasma extrapolation geometry types and the paths to files containing electron density distributions and the applied static **B** fields. The *Species densities* tab is shown in Figure 3.9 and is where the plasma composition is defined. The *Problem settings* tab is shown in Figure 3.10 and it is used for setting damping modes and tuning numerical parameters. The properties defined in each tab are described in the following subsections. Similar descriptions can be obtained inside GiD by hovering the mouse pointer over each data field after clicking on the help button located on the same *Plasma* window.

Plasma geometry - Figure 3.8

- **Geometry extrapolation**: plasma geometry extrapolation type. It can be of three different types:
  - **Flat 1D**: a 1D measurement line is read from the files specified in the *ne file path* and *Be file path* fields. The 1D line is extrapolated into the 3D geometry by extrusion in the toroidal direction, and applying a curvature in the poloidal direction defined by the *Poloidal ne curvature* and *Poloidal Be curvature* fields. The first point of the 1D measurement line will be placed at the coordinates specified in the *L1 00 XYZ* fields. The values of *ne* and *Be* outside the defined line will correspond to those at the endpoints of the line.
  - **Curved**: a 1D measurement line is read from the files specified in the *ne file path* and *Be file path*. The 1D line is extrapolated into the 3D geometry by extrusion, applying a poloidal curvature defined by the *Poloidal ne curvature* and *Poloidal Be curvature* fields, and a toroidal curvature determined by the distance to the tokamak centre. The tokamak centre coordinates are specified in the *TK 00 XYZ* fields. The first point of the 1D measurement line is placed at the coordinates defined in the *L1 00 XYZ* fields. The values of *ne* and *Be* outside the defined line will correspond to those at the endpoints of the line.
  - **Full 3D**: pointwise nodal 3D distributions are read from the files specified in the *ne file path* and *Be file path* fields. The distributions are applied directly to the 3D geometry on a node-by-node basis.
  - **Tensor**: a permittivity tensor defined in the binary file `Tensor_K_CPGE.bin` is applied to the same mesh nodes used in the density and magnetic field 3D data files provided in the text boxes *ne file path* and *Be file path*. The tensor components must be relative to the vacuum permittivity and defined in the external **B** field coordinate system. The tensor file must be located in the problem folder. An example Python script demonstrating how to generate the required binary format can be found in the problem type directory `\Documents\Plasma_Files\Python_Scripts`.
- **Poloidal ne curvature**: poloidal curvature applied to *ne* density profile in the *Flat 1D* and *Curved* modes. The centre of curvature is located at a distance equal to the *Poloidal ne curvature* from the first point of the 1D measurement line. The first point of the 1D measurement line is located at the coordinates specified in the *L1 00 XYZ* fields.
- **Poloidal Be curvature**: poloidal curvature applied to the *Be* field profile in the *Flat 1D* and *Curved* modes. The centre of curvature is located at a distance equal to the *Poloidal Be curvature* from the first point of the 1D measurement line. The first point of the 1D measurement line is located at the coordinates specified in the *L1 00 XYZ* fields.

- **L1 00 X**: X coordinate of the first point of the 1D measurement line.
- **L1 00 Y**: Y coordinate of the first point of the 1D measurement line.
- **L1 00 Z**: Z coordinate of the first point of the 1D measurement line.

- **TK 00 X**: X coordinate of the centre of the tokamak.
- **TK 00 Y**: Y coordinate of the centre of the tokamak.
- **TK 00 Z**: Z coordinate of the centre of the tokamak.

- **TK axis X**: X component of the main axis of the tokamak.
- **TK axis Y**: Y component of the main axis of the tokamak.
- **TK axis Z**: Z component of the main axis of the tokamak.

- **ne file path**: electron density data file path.
- **Be file path**: applied external static **B** field data file path.

Plasma composition - Figure 3.9

- **H1 1+ / ne**: Hydrogen 1 (protium) ion q=+1e density relative to electron density (ne).
- **H2 1+ / ne**: Hydrogen 2 (deuterium) ion q=+1e density relative to electron density (ne).
- **H3 1+ / ne**: Hydrogen 3 (tritium) ion q=+1e density relative to electron density (ne).

- **He3 1+ / ne**: Helium 3 ion q=+1e density relative to electron density (ne).
- **He3 2+ / ne**: Helium 3 ion q=+2e density relative to electron density (ne).

- **He4 1+ / ne**: Helium 4 ion q=+1e density relative to electron density (ne).
- **He4 2+ / ne**: Helium 4 ion q=+2e density relative to electron density (ne).

- **Be9 1+ / ne**: Beryllium 9 ion q=+1e density relative to electron density (ne).
- **Be9 2+ / ne**: Beryllium 9 ion q=+2e density relative to electron density (ne).
- **Be9 3+ / ne**: Beryllium 9 ion q=+3e density relative to electron density (ne).
- **Be9 4+ / ne**: Beryllium 9 ion q=+4e density relative to electron density (ne).

- **Ne20 10+ / ne**: Neon 20 ion q=+10e density relative to electron density (ne).
- **Ni60 20+ / ne**: Nickel 60 ion q=+20e density relative to electron density (ne).
- **W184 25+ / ne**: Tungsten 184 ion q=+25e density relative to electron density (ne).

Plasma problem settings - Figure 3.10

- **Matrix format**: available storage formats for the cold plasma system matrix:
  - **Full matrix**: stores a non-symmetric matrix containing the contribution of the volumetric and the Robin boundary condition elements altogether.
  - **Herm-Full**: stores two separate matrices containing the diagonal and the upper diagonal of the volumetric elements contribution and, a non-symmetric matrix with the contribution of the Robin boundary condition elements.
  - **Herm-Symm**: stores two separate matrices containing the diagonal and the upper diagonal of the volumetric elements contribution and, the diagonal and the upper diagonal of the Robin boundary condition elements contribution.
- **Kij tolerance**: minimum value of the elements of the matrix. If $\mid K_{ij} \mid <= Tol$ then, K_{ij} will not be added to the global system matrix.
- **E par tolerance**: tolerance applied to the electric field parallel to the magnetic flux density: $\mathbf{E}_{\parallel} = 0$ when $P < 0$ and $\mid P \mid > Tol$. This condition will be ignored in the edge element formulations.
- **Damping mode**: available damping modes:
  - **No damping**: no damping is applied to the cold plasma tensor.
  - **Background ctc**: a constant damping value ν is added to the diagonal terms of the cold plasma tensor ($R + i\nu$, $L + i\nu$, $P + i\nu$, $S + i\nu$) everywhere the cold plasma is defined.
  - **Background den**: as in the *Background ctc* mode, a constant damping value ν is added to the diagonal terms of the cold plasma tensor ($R + i\nu$, $L + i\nu$, $P + i\nu$, $S + i\nu$) but, in this case, the damping value ν is made zero when the electron density is zero.
  - **ComplexFreq all**: a collision frequency ν is added to the frequency terms of all the species ($\omega + i\nu$), as shown in [75]
  - **ComplexFreq ele**: as in the *ComplexFreq all* mode, a collision frequency ν is added to the frequency terms ($\omega + i\nu$) but, in this case, only in the electrons contribution to the cold plasma tensor.
- **Damping value**: damping value ν. It has a different meaning depending on the damping mode selected.

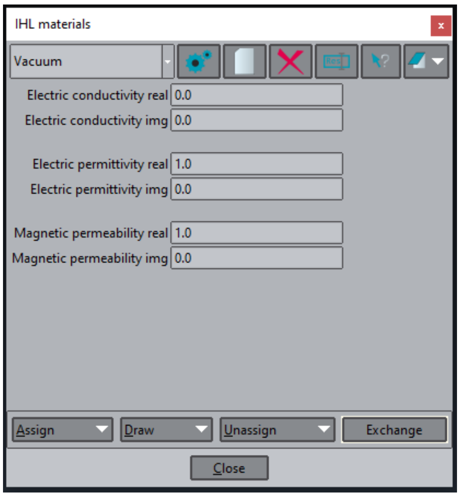

Figure 3.7: Isotropic Homogeneous Linear (IHL) material properties window.

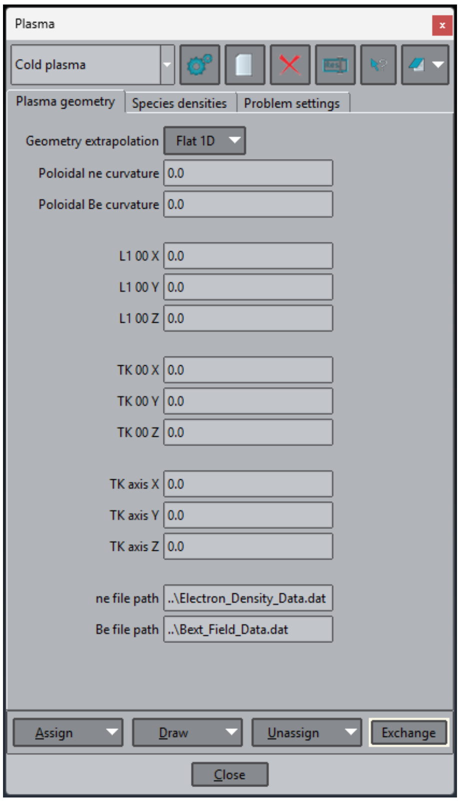

Figure 3.8: Cold plasma properties window - Tab: Plasma geometry.

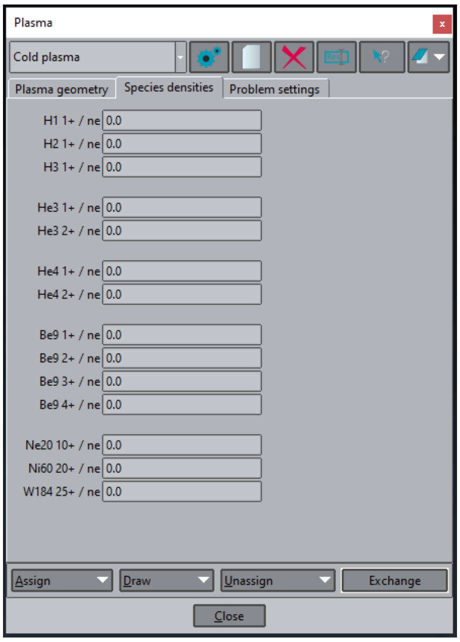

Figure 3.9: Cold plasma properties window - Tab: Species densities.

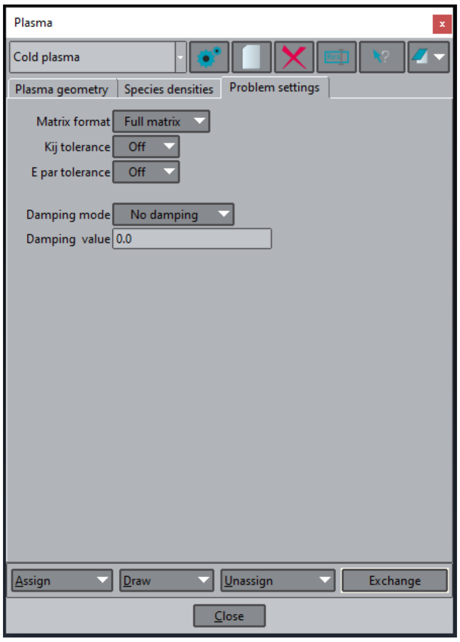

Figure 3.10: Cold plasma properties window - Tab: Problem settings.

3.4 Sources

In ERMES 20.0 there are implemented two types of electromagnetic field sources:

1. Volumetric sources (electric current density vectors **J**).
2. Boundary sources (plane waves, Gaussian beams, waveguide modes, current flux).

Volumetric sources are electric current densities **J** applied to volumes inside the problem domain. Boundary sources are electromagnetic fields applied as Robin conditions to the boundary surfaces of the problem domain (e.g., plane waves, Gaussian beams, waveguide modes, current flux).

All the sources must be first defined on its corresponding properties window and then applied to volumes (or surfaces) using its associated conditions window. The sources assignment process is different from the materials assignment process explained in Section 3.3. Materials are defined and assigned from the same window. On the other hand, sources are defined in a properties definition window and assigned from a conditions window. Further details will be given in the following sections.

3.4.1 Volumetric sources J

Current density **J** sources properties are defined in the window *Current sources properties* shown in Figures 3.15, 3.16, 3.17, 3.18, and 3.19, which can be opened from the menu bar of Figure 3.4 by clicking on the icon *Current density properties* 3.11. To create a new **J**, click on the white paper sheet icon button at the upper part of the *Current sources properties* window.

Once the properties of **J** have been defined in the *Current sources properties* window, the **J** source must be assigned to a volume from the *Current sources* window of Figure 3.13, which can be opened from the ERMES menu bar by clicking on the *Current density sources* icon 3.12.

Current density **J** sources are applied to volumes following a procedure similar to the one explained in Section 3.3 and shown in Figure 3.5, but using the window *Current sources* of Figure 3.13 instead. The **J** source assigned to the volume will be the one selected in the drop-down menu of Figure 3.13.

For checking the correct application of **J** sources, click on the **Draw** → **Colors** menu located at the bottom of the *Current sources* window of Figure 3.13. If properly assigned, the selected volume will appear in a solid colour along with the other **J** sources assigned, as shown in Figure 3.14.

It is important to remember that the current sources properties window must only be used to define the J source properties, not to assign the J source to a volume. Assigning a current density to a volume from the *Current sources properties* window will raise an error in ERMES 20.0. In the following subsections will be described the parameters required to define the complex electric current density vector **J** on different coordinate systems from the window *Current sources properties*.

Figure 3.11: ERMES menu bar icon to access the *Current sources properties* windows shown in 3.15, 3.16, 3.17, 3.18, and 3.19.

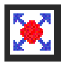

Figure 3.12: ERMES menu bar icon to access the *Current sources* assignment window 3.13.

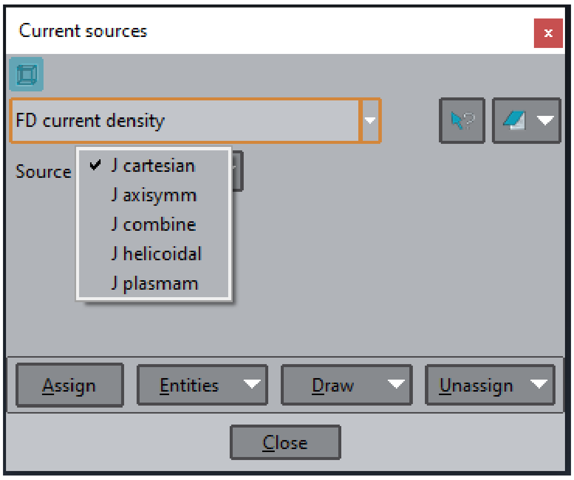

Figure 3.13: Electric current density **J** assignment window. **J** sources must be assigned to volumes using the **Assign** button located on the lower left corner of the *Current sources* window.

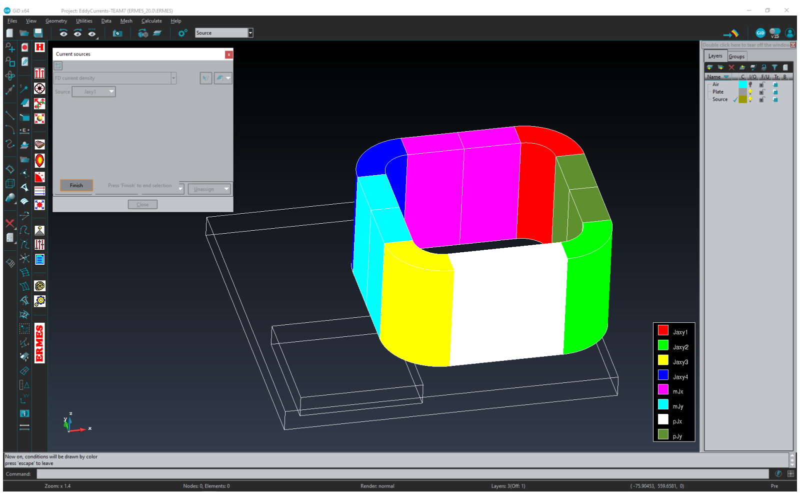

Figure 3.14: Assigned electric current densities **J**. To check the correct assignment of **J** sources, click on the **Draw** → **Colors** menu located at the bottom of the *Current sources* window of Figure 3.13.

J Cartesian - Figure 3.15

Complex electric current density vector $\mathbf{J}_{xyz}$ in Cartesian coordinates:

- **Jx** : modulus of the X component of the electric current density vector (A/m^2).
- **Phx**: phase of the X component of the electric current density (rad).

- **Jy** : modulus of the Y component of the electric current density vector (A/m^2).
- **Phy**: phase of the Y component of the electric current density (rad).

- **Jz** : modulus of the Z component of the electric current density vector (A/m^2).
- **Phz**: phase of the Z component of the electric current density (rad).

J axisymmetric - Figure 3.16

Complex electric current density vector $\mathbf{J}_a$ in axisymmetric coordinates. $\mathbf{J}_a$ is axisymmetric with respect to the axis defined in *AX axis XYZ* with centre of curvature on *AX 00 XYZ*:

- **Ja** : modulus of the angular component of the axisymmetric current density vector (A/m^2).
- **Pha**: phase of the angular component of the axisymmetric current density vector (rad).

- **AX 00 X**: X coordinate of the centre of curvature.
- **AX 00 Y**: Y coordinate of the centre of curvature.
- **AX 00 Z**: Z coordinate of the centre of curvature.

- **AX axis X**: X component of the axisymmetric axis.
- **AX axis Y**: Y component of the axisymmetric axis.
- **AX axis Z**: Z component of the axisymmetric axis.

J Cartesian and axisymmetric - Figure 3.17

Combination of a complex electric current density vector $\mathbf{J}_{xyz}$ in Cartesian coordinates plus a complex electric current density vector $\mathbf{J}_a$ in axisymmetric coordinates. Both current densities will be assigned to the same volume and added to create a total current equal to the combination of the two: $\mathbf{J}_{combine} = \mathbf{J}_{xyz} + \mathbf{J}_a$. The definitions of properties for $\mathbf{J}_{combine}$ in Figure 3.17 are the same as the ones shown in Figures 3.15 and 3.16. To access to the properties of $\mathbf{J}_a$ click on the *Axisymmetric* tab of Figure 3.17.

J helicoidal - Figure 3.18

This input window allows to set a volumetric current density in both poloidal and toroidal directions $\mathbf{J}_{heli} = \mathbf{J}_{pol} + \mathbf{J}_{tor}$. The current density $\mathbf{J}_{heli}$ will be applied to a torus volume with a major radius *HL R*, centered at *HL 00 XYZ*, around the axis *HL axis XYZ*:

- **Jpol**: module of the poloidal component of the helicoidal current density (A/m^2).
- **Php** : phase of the poloidal component of the helicoidal current density (rad).

- **Jtor**: module of the toroidal component of the helicoidal current density (A/m^2).
- **Pht** : phase of the toroidal component of the helicoidal current density (rad).

- **HL R**: major radius of the torus in where the helicoidal current is applied.

- **HL 00 X**: X coordinate of the torus centre.
- **HL 00 Y**: Y coordinate of the torus centre.
- **HL 00 Z**: Z coordinate of the torus centre.

- **HL axis X**: X component of the torus axis.
- **HL axis Y**: Y component of the torus axis.
- **HL axis Z**: Z component of the torus axis.

J plasma modes - Figure 3.19

This input window sets a volumetric current density $\mathbf{J}_{plasm} = \mathbf{J}_{pol} + \mathbf{J}_{tor}$ defined in a torus volume with a major radius *PM R*, centered at *PM 00 XYZ*, around the axis *PM axis XYZ*. The current densities $\mathbf{J}_{pol}$ and $\mathbf{J}_{tor}$ satisfy the following equations:

$$\begin{aligned}\mathbf{J_{pol}} &= |\mathbf{A}_0|\, e^{i\varphi_0} \sin\left(M\theta + \phi\right) \\ \mathbf{J_{tor}} &= |\mathbf{B}_0|\, e^{i\psi_0} \left\{-M\left[(R/r) + \cos\left(\theta\right)\right] \sin\left(M\theta + \phi\right) - \sin(\theta)\cos\left(M\theta + \phi\right)\right\}\end{aligned} \tag{3.3}$$

where $\mathbf{A}_0 = |\mathbf{A}_0|\, e^{i\varphi_0}$ and $\mathbf{B}_0 = |\mathbf{B}_0|\, e^{i\psi_0}$ are complex constants scaling $\mathbf{J_{pol}}$ and $\mathbf{J_{tor}}$, respectively, M is an integer number representing the current density mode, θ is the poloidal angle, ϕ is the toroidal angle, R is the major radius of the torus and, r is its minor radius. Figures 3.20 and 3.21 present examples of $\mathbf{J}_{plasm}$ for various mode numbers. The parameters required to configure $\mathbf{J}_{plasm}$ using the interface shown in Figure 3.19 are described below:

- **Jpol**: module of the constant complex number $\mathbf{A}_0$ (A/m^2).
- **Php** : phase φ_0 (rad) of the constant complex number $\mathbf{A}_0$.

- **Jtor**: module of the constant complex number $\mathbf{B}_0$ (A/m^2).
- **Pht** : phase ψ_0 (rad) of the constant complex number $\mathbf{B}_0$.

- **PM M**: plasma mode index.
- **PM R**: major radius of the torus in where the plasma mode is applied.

- **PM 00 X**: X coordinate of the torus centre.
- **PM 00 Y**: Y coordinate of the torus centre.
- **PM 00 Z**: Z coordinate of the torus centre.

- **PM axis X**: X component of the torus axis.
- **PM axis Y**: Y component of the torus axis.
- **PM axis Z**: Z component of the torus axis.

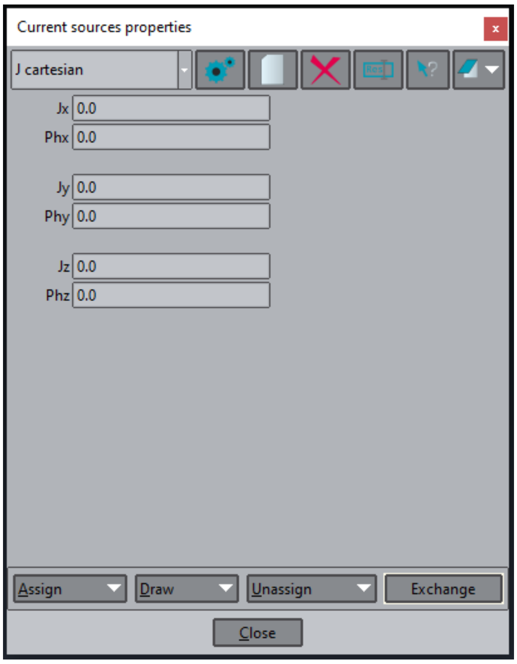

Figure 3.15: Current sources properties window - **J** Cartesian.

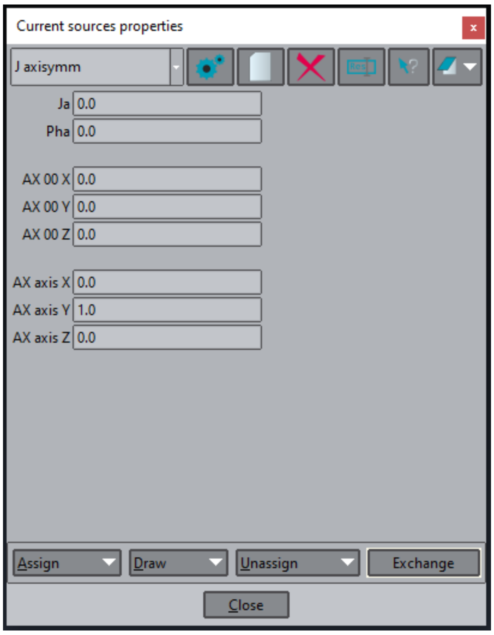

Figure 3.16: Current sources properties window - **J** axisymm.

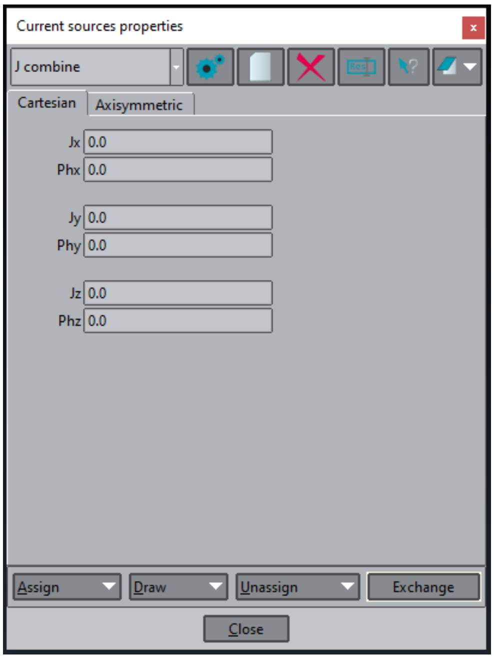

Figure 3.17: Current sources properties window - **J** combine.

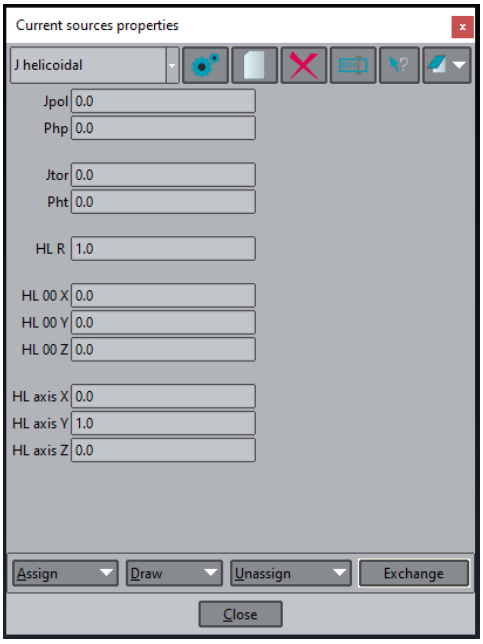

Figure 3.18: Current sources properties window - **J** helicoidal.

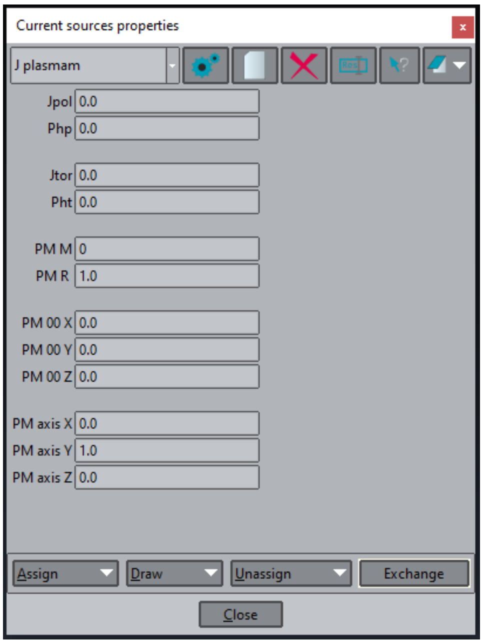

Figure 3.19: Current sources properties window - **J** plasmam.

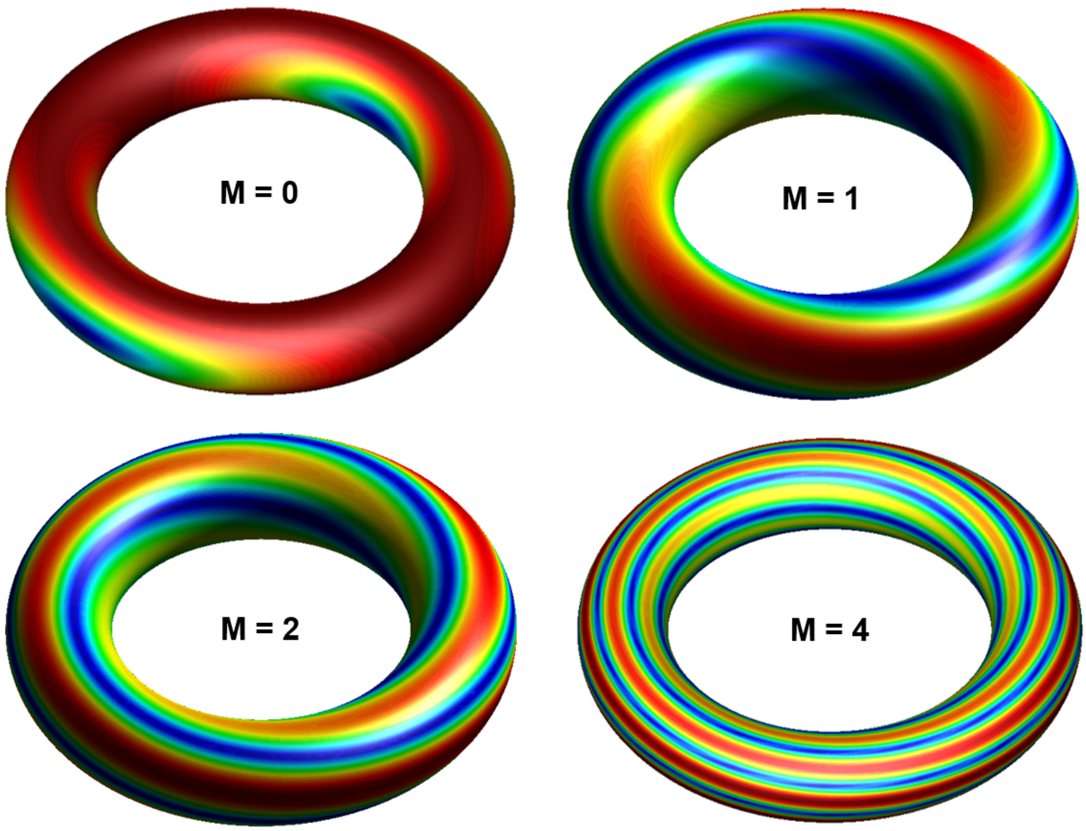

Figure 3.20: Module of $\mathbf{J}_{plasm}$ for M = 0, 1, 2 and, 4.

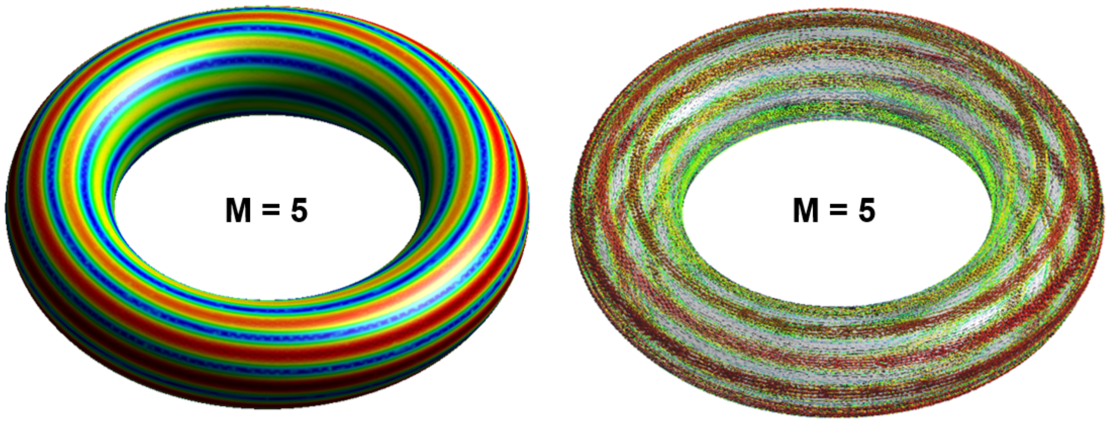

Figure 3.21: Module and vectors of $\mathbf{J}_{plasm}$ for M = 5.

3.4.2 Boundary sources

Boundary sources are fields applied at the boundaries of the problem domain using Robin boundary conditions. This section outlines the properties required by ERMES 20.0 to define these sources. Appendix A provides theoretical background on these conditions in the context of electromagnetics. Appendix B describes their adaptation to the various FEM formulations implemented in ERMES 20.0. The boundary sources available in ERMES 20.0 include:

1. TE_{10} mode for rectangular waveguides,
2. TEM mode for coaxial waveguides,
3. Plane wave,
4. Gaussian beam,
5. Current flux.

Boundary sources properties are defined in the windows: *RWPort TE10*, *CoaxialPort TEM*, and *Robin condition coefficients*, which are shown in Figures 3.26 to 3.31. These can be accessed from the ERMES menu bar icons 3.4:

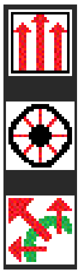

Figure 3.22: ERMES menu bar icons (from top to bottom) for the windows: *RWPort TE10*, *CoaxialPort TEM* and, *Robin condition coefficients*.

Once the properties of the boundary sources have been defined in 3.26-3.31, they must be applied to the surfaces of the geometry using the *Robin conditions* window 3.23, which can be accessed by clicking on the icon *Robin boundary conditions* of 3.4. The appropriate condition must be selected from the drop-down menu of 3.23. Afterwards, to assign the condition to a surface, click the **Assign** button and then select a surface, as shown in Figure 3.24.

To verify the correct application of the condition, navigate to the **Draw** → **Colors** menu located at the bottom of 3.23. If the condition has been properly assigned, the selected surface will appear in a solid colour along with other assigned Robin boundary conditions, as shown in Figure 3.25.

It is important to note that the boundary source properties windows must only be used to define the source properties, not to assign the source to a surface. Attempting to assign a boundary source to a surface from the windows 3.26-3.31 will result in an error in ERMES 20.0. Boundary sources must be assigned to a surface exclusively through the *Robin Conditions* window 3.23.

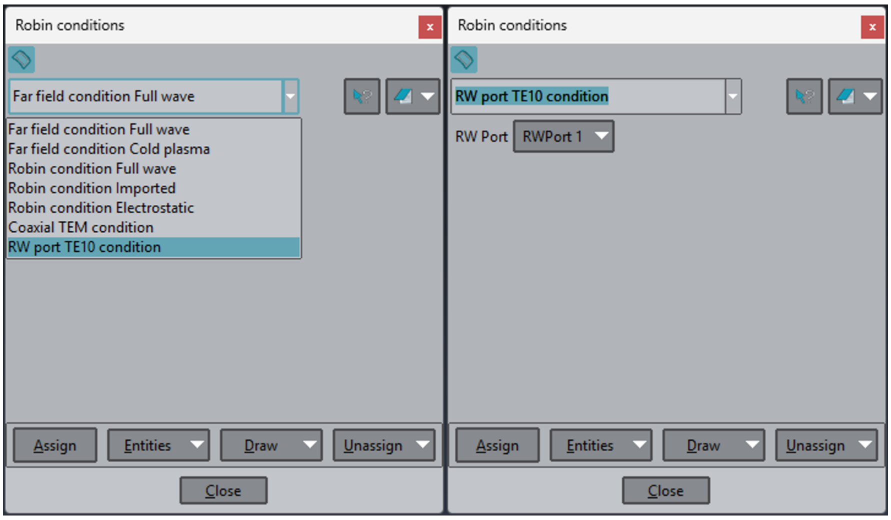

Figure 3.23: *Robin conditions* window. This window can be accessed from the ERMES menu bar 3.4 by clicking the icon *Robin boundary conditions*. From the drop-down menu, select the condition to apply. Next, choose the specific port, wave type, or current flux to assign.

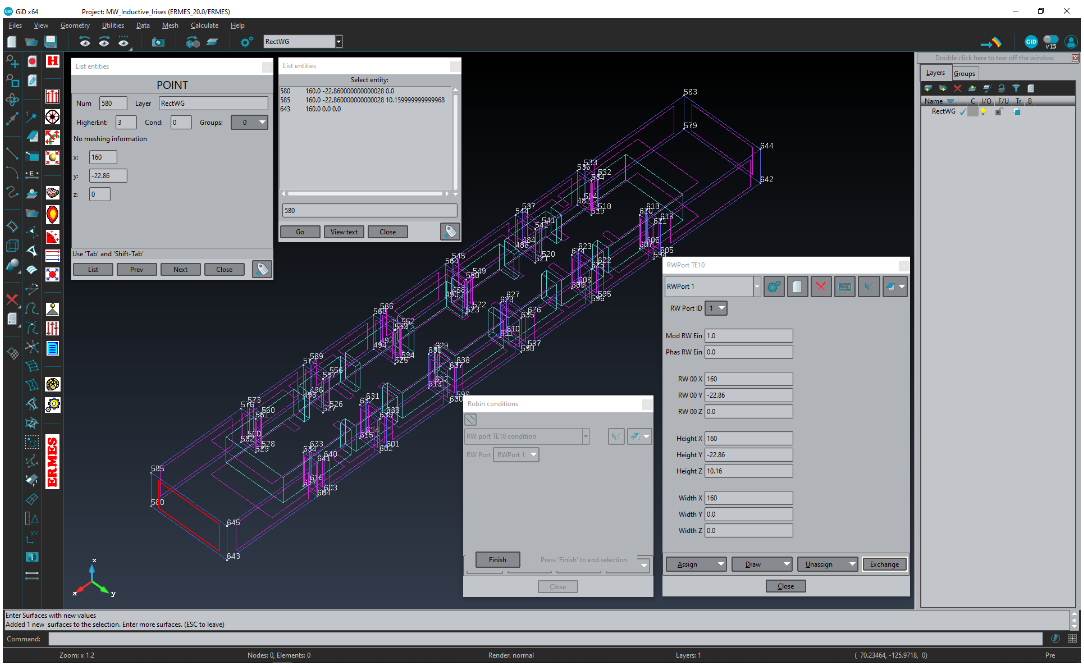

Figure 3.24: From the *Robin conditions* window 3.23, click on the **Assign** button and select a surface. The surface will turn red as shown in the figure. Then, click on the central mouse button, the keyboard key *Escape*, or the *Finish* button to confirm the application.

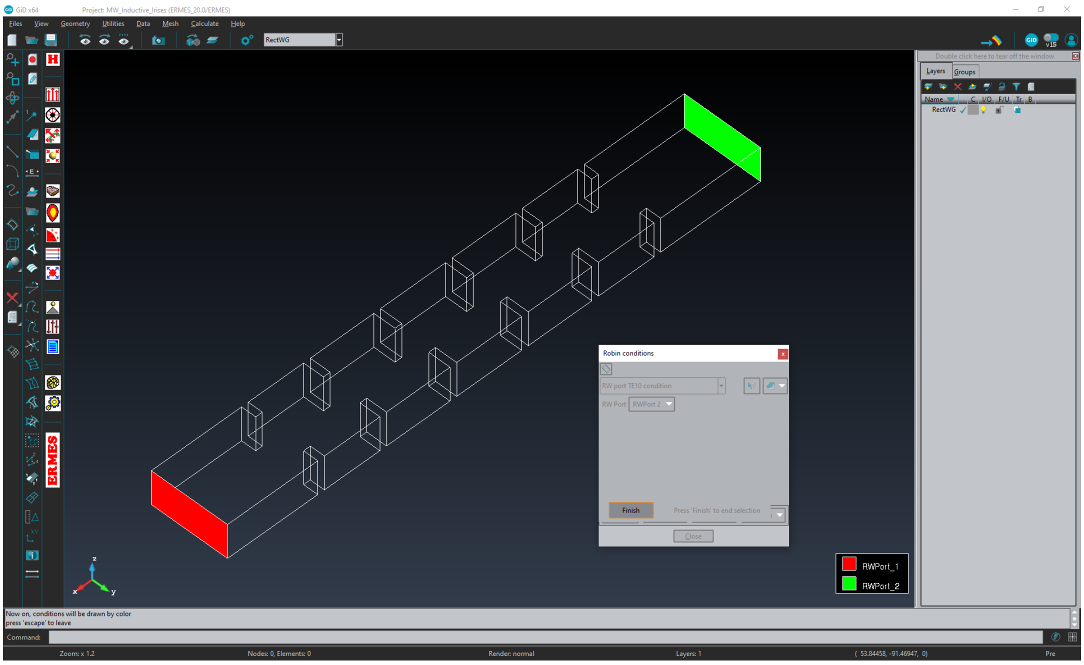

Figure 3.25: To verify the correct application of the condition, navigate to the **Draw** → **Colors** menu at the bottom of the *Robin conditions* window 3.23. If the condition has been correctly assigned, the selected surface will appear in a solid colour, along with other assigned Robin boundary conditions.

Rectangular waveguide port - Figure 3.26

The rectangular waveguide port boundary source assumes a transversal electromagnetic mode TE_{10} at the port surface. The implemented electric field $\mathbf{E}_{10}$ of the fundamental mode TE_{10} is:

$$\mathbf{E}_{10} = -|E_0|e^{i\theta}\sqrt{\frac{2i\omega\mu}{ab\gamma}}\,\sin(k_c x)\,e^{\gamma z}\,\hat{\mathbf{y}}, \tag{3.4}$$

where $|E_0|e^{i\theta}$ is a complex scale factor, a is the width of the rectangular waveguide, b is its height, $k_c = \pi/a$ and, γ is the propagation constant of the mode TE_{10}, which is $\gamma = i\sqrt{k_0^2 - k_c^2}$ when $k_0 > k_c$ and $\gamma = -\sqrt{k_c^2 - k_0^2}$ when $k_0 < k_c$, being $k_0 = \omega\sqrt{\epsilon_0\mu_0}$. In ERMES 20.0 the port surface can have any arbitrary orientation but, for clarity, in expression (3.4) is assumed that the x-axis is along the width of the rectangular waveguide, the y-axis is along its height and, the z-axis is perpendicular to the xy-plane, pointing inward to the waveguide volume. It is assumed z = 0 at the input port. For more information about rectangular waveguides, see references [57, 3].

The time-averaged input power P_0 of the TE_{10} mode on the surface of the port is $P_0 = |E_0|^2/2$, which allows the control of the input power with $|E_0|$. The port is assumed to be in vacuum and the parameters $|E_0|e^{i\theta}$, γ, k_c, a and, b are calculated from the information provided in the window 3.26:

- **RW Port ID**: rectangular waveguide port identification number. This number is required for identifying the ports when calculating the S_{ij} parameters.

- **Mod RW Ein**: module of the complex scale factor E_0 multiplying the input TE_{10} field. For an output port, use *Mod RW Ein* = 0. For an input port, with a time-averaged input power equal to P_0, use *Mod RW Ein* = $\sqrt{2P_0}$.
- **Phas RW Ein**: phase of the complex scale factor E_0 multiplying the input TE_{10} field. For an output port, use *Phas RW Ein* = 0. For an input port, with a phase equal to θ (rad), use *Phas RW Ein* = θ.

- **RW 00 X**: X coordinate of the lower left corner of the port.
- **RW 00 Y**: Y coordinate of the lower left corner of the port.
- **RW 00 Z**: Z coordinate of the lower left corner of the port.

- **Height X**: X coordinate of the upper left corner of the port.
- **Height Y**: Y coordinate of the upper left corner of the port.
- **Height Z**: Z coordinate of the upper left corner of the port.

- **Width X**: X coordinate of the lower right corner of the port.
- **Width Y**: Y coordinate of the lower right corner of the port.
- **Width Z**: Z coordinate of the lower right corner of the port.

where:

$$
\begin{aligned}
|E_0| &= Mod\,RW\ Ein, \\
\theta &= Phas\,RW\ Ein, \\
a &= \sqrt{(Width\ X - RW\,00\,X)^2 + (Width\ Y - RW\,00\,Y)^2 + (Width\ Z - RW\,00\,Z)^2}, \\
b &= \sqrt{(Height\,X - RW\,00\,X)^2 + (Height\,Y - RW\,00\,Y)^2 + (Height\,Z - RW\,00\,Z)^2}.
\end{aligned}
$$

The mathematical expressions of the rectangular waveguide port boundary condition is:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} - \gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) = -2\,\gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}_{10}). \tag{3.5}$$

This expression is properly tailored in ERMES 20.0 to suit the various FEM formulations implemented in it, as outlined in Appendix B. For the regularized and LL2P formulations, it is advisable to add a TEC Dirichlet condition 3.5.1 ($\hat{\mathbf{n}} \cdot \mathbf{E} = 0$) on the port surfaces to enhance convergence.

Coaxial waveguide port - Figure 3.27

The coaxial waveguide port boundary source assumes a transversal electromagnetic mode TEM at the port surface. The implemented electric field $\mathbf{E}_{\text{TEM}}$ of the fundamental coaxial TEM mode is:

$$\mathbf{E}_{\text{TEM}} = |E_0|e^{i\theta}\sqrt{\frac{\eta}{2\pi\ln(b/a)}}\left(\frac{e^{\gamma z}}{\text{r}}\right)\hat{\mathbf{r}}, \tag{3.6}$$

where $|E_0|e^{i\theta}$ is a complex scale factor, $\eta = \sqrt{\mu/\varepsilon}$, $\gamma = i\omega\sqrt{\varepsilon\mu}$ is the propagation constant of the TEM mode, a is the diameter of the inner cylinder of the coaxial waveguide, b is the diameter of the exterior cylinder of the coaxial waveguide, r is the radial coordinate $r = \sqrt{x^2 + y^2}$ and, $\hat{\mathbf{r}} = (x/r, y/r)$ is the unitary radial vector. In ERMES 20.0 the port surface can have any arbitrary orientation but, for clarity, the port plane in (3.6) is assumed to be in the xy-plane and the direction of propagation is along the z-axis, pointing inward to the waveguide volume. It is assumed z = 0 at the input port. For more information about coaxial waveguides, see references [57, 3].

The time-averaged input power P_0 of the fundamental TEM mode (3.6) is $P_0 = |E_0|^2/2$, which allows the control of the input power with $|E_0|$. The port is assumed to be in a lossless IHL material and the parameters $|E_0|e^{i\theta}$, γ, η, a and, b are obtained from the information provided in the window 3.27:

- **CC Port ID**: coaxial port identification number. This number is required for identifying the ports when calculating the S_{ij} parameters.

- **Mod CC Ein**: module of the complex scale factor E_0 multiplying the input TEM field. For an output port, use *Mod CC Ein* = 0. For an input port, with a time-averaged input power equal to P_0, use *Mod CC Ein* = $\sqrt{2P_0}$.
- **Phas CC Ein**: phase of the complex scale factor E_0 multiplying the input TEM field. For an output port, use *Phas CC Ein* = 0. For an input port, with a phase equal to θ (rad), use *Phas CC Ein* = θ.

- **CC 00 X**: X coordinate of the centre of the coaxial waveguide port.
- **CC 00 Y**: Y coordinate of the centre of the coaxial waveguide port.
- **CC 00 Z**: Z coordinate of the centre of the coaxial waveguide port.

- **Inner D**: diameter of the inner cylinder of the coaxial waveguide.
- **Exter D**: diameter of the exterior cylinder of the coaxial waveguide.

- **Epr coax**: relative electric permittivity of the medium inside the coaxial waveguide.
- **Mu coax**: relative magnetic permeability of the medium inside the coaxial waveguide.

The mathematical expressions of the coaxial waveguide port boundary condition is:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} - \gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) = -2\,\gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}_{\text{TEM}}). \tag{3.7}$$

This expression is properly tailored in ERMES 20.0 to suit the various FEM formulations implemented in it, as outlined in Appendix B. For the regularized and LL2P formulations, it is advisable to add a TEC Dirichlet condition 3.5.1 ($\hat{\mathbf{n}} \cdot \mathbf{E} = 0$) on the port surfaces to enhance convergence.

Plane waves - Figure 3.28

The plane wave boundary source assumes a transversal plane wave at the boundary surface. The implemented electric field $\mathbf{E}_{\text{PW}}$ of the incident plane wave is:

$$\mathbf{E}_{\text{PW}} = |E_0|\, e^{i\theta}\, e^{\gamma\, \hat{\mathbf{k}} \cdot \mathbf{r}}\, \hat{\mathbf{e}}_0, \tag{3.8}$$

where $|E_0|e^{i\theta}$ is the complex amplitude of the plane wave, $\gamma = i\omega\sqrt{\varepsilon\mu}$ is the propagation constant, $\hat{\mathbf{k}}$ is the unit wave vector and, $\hat{\mathbf{e}}_0$ is the unit polarization vector. The parameters $|E_0|e^{i\theta}$, γ, $\hat{\mathbf{k}}$, and $\hat{\mathbf{e}}_0$ are derived from the information provided in the window 3.28:

- **Multiple waves** [**On** | **Off**]: enables the application of multiple plane waves on a boundary. All the plane waves with this option activated (*Multiple waves* = *On*) will be added coherently to the same boundary and with the same γ as the assigned plane wave.

- **Module E field**: module of the plane wave complex amplitude ($|E_0|$).
- **Phase E field**: phase of the plane wave complex amplitude (θ).

- **Polarization X**: X component of the polarization vector $\hat{\mathbf{e}}_0$.
- **Polarization Y**: Y component of the polarization vector $\hat{\mathbf{e}}_0$.
- **Polarization Z**: Z component of the polarization vector $\hat{\mathbf{e}}_0$.

- **Wave vector X**: X component of the wave vector $\hat{\mathbf{k}}$.
- **Wave vector Y**: Y component of the wave vector $\hat{\mathbf{k}}$.
- **Wave vector Z**: Z component of the wave vector $\hat{\mathbf{k}}$.

- **IBC conductivity real**: real part of the electric conductivity (S/m).
- **IBC conductivity img**: imaginary part of the electric conductivity (S/m).

- **IBC permittivity real**: real part of the relative electric permittivity.
- **IBC permittivity img**: imaginary part of the relative electric permittivity.

- **IBC permeability real**: real part of the relative magnetic permeability.
- **IBC permeability img**: imaginary part of the relative magnetic permeability.

It is not necessary to provide the vectors *Polarization XYZ* and *Wave vector XYZ* normalized, as they are internally normalized within ERMES 20.0; however, they must be provided perpendicular to each other. The data entered in the IBC material properties fields should match the properties of the medium where the plane wave is incident. By setting *Module E field* equal to zero ($|E_0| = 0$), the plane wave boundary condition can be used as an Impedance Boundary Condition (IBC) (A.87). The mathematical expression of the plane wave boundary condition is:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} - \gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) = -\,\gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}_{\mathrm{PW}}) + \gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{k}} \times \mathbf{E}_{\mathrm{PW}}), \tag{3.9}$$

which differs slightly from the previous cases of rectangular and coaxial waveguides. The difference arises because the plane wave can be applied to curved domains, where the boundary normal is not necessarily parallel to the wave propagation vector of the incident field. The right-hand term of (3.9) reduces to its equivalent (3.5) or (3.7) when $\hat{\mathbf{k}} = -\hat{\mathbf{n}}$, where $\hat{\mathbf{n}}$ is the unitary boundary normal pointing outward from the domain. For the regularized and LL2P formulations, the following extension of the divergence boundary condition is automatically applied inside ERMES 20.0:

$$\nabla \cdot \mathbf{E} - \gamma\,(\hat{\mathbf{n}} \cdot \mathbf{E}) = -\,\gamma\,\hat{\mathbf{n}} \cdot \mathbf{E}_{\mathrm{PW}} + \gamma\,\hat{\mathbf{k}} \cdot \mathbf{E}_{\mathrm{PW}} \tag{3.10}$$

In Appendix B can be consulted the adaptation of the above boundary conditions to all the different formulations implemented in ERMES 20.0.

Gaussian beams - Figure 3.29

The Gaussian beam boundary source assumes the following electric field at the boundary:

$$\mathbf{E}_{\mathrm{GB}} = \sqrt[4]{\frac{z_\varrho\, z_\eta}{|q_\varrho|\,|q_\eta|}}\,\left[|E_\varrho| e^{i\theta_\varrho}\,\hat{\varrho} + |E_\eta| e^{i\theta_\eta}\,\hat{\eta}\right] e^{i\Phi} \tag{3.11}$$

being:

$$z_\varrho = \frac{k_0}{2}\,\omega_\varrho^2, \;\; z_\eta = \frac{k_0}{2}\,\omega_\eta^2, \tag{3.12}$$

$$q_\varrho = \delta - iz_\varrho, \;\; q_\eta = \delta - iz_\eta, \tag{3.13}$$

$$\Phi = k_0\delta - \frac{1}{2}\left[\arctan\left(\frac{\delta}{z_\varrho}\right) + \arctan\left(\frac{\delta}{z_\eta}\right)\right] + \frac{k_0}{2}\left(\frac{\varrho^2}{q_\varrho} + \frac{\eta^2}{q_\eta}\right) \tag{3.14}$$

where $|E_\varrho|e^{i\theta_\varrho}$ and $|E_\eta|e^{i\theta_\eta}$ are the transversal components of electric field at the beam waist centre, $k_0 = \omega\sqrt{\epsilon_0\mu_0}$ is the angular wave number in vacuum, and ω_ϱ and ω_η are the semi-axis lengths of the elliptical cross-section of the beam at its waist. In expressions (3.11) to (3.14), the origin of the coordinate system is placed at the centre of the beam waist. The ϱ- and η-axis are aligned with the principal axes of the elliptical cross-section of the beam, while the δ-axis is aligned with the direction of propagation. The unit vectors $\hat{\varrho}$, $\hat{\delta}$, and $\hat{\eta}$ satisfy the relations $\hat{\varrho} \perp \hat{\delta}$ and $\hat{\eta} = \hat{\delta} \times \hat{\varrho}$. The unit vectors $\hat{\varrho}$, $\hat{\eta}$, and $\hat{\delta}$, as well as the parameters $|E_\varrho|$, θ_ϱ, $|E_\eta|$, θ_η, ω_ϱ, and ω_η are determined from the information provided in window 3.29:

- **GB module E par**: module of the electric field component $\hat{\varrho}$ at the waist centre ($|E_\varrho|$).
- **GB phase E par**: phase of the electric field component $\hat{\varrho}$ at the waist centre (θ_ϱ).

- **GB module E per**: module of the electric field component $\hat{\eta}$ at the waist centre ($|E_\eta|$).
- **GB phase E per**: phase of the electric field component $\hat{\eta}$ at the waist centre (θ_η).

- **GB radius par**: waist radius in the $\hat{\varrho}$ direction (ω_ϱ).
- **GB radius per**: waist radius in the $\hat{\eta}$ direction (ω_η).

- **GB 00 X**: X coordinate of the centre of the Gaussian beam waist.
- **GB 00 Y**: Y coordinate of the centre of the Gaussian beam waist.
- **GB 00 Z**: Z coordinate of the centre of the Gaussian beam waist.

- **GB polarization X**: X component of the polarization vector $\hat{\varrho}$.
- **GB polarization Y**: Y component of the polarization vector $\hat{\varrho}$.
- **GB polarization Z**: Z component of the polarization vector $\hat{\varrho}$.

- **GB wave vector X**: X component of the wave propagation vector $\hat{\delta}$.
- **GB wave vector Y**: Y component of the wave propagation vector $\hat{\delta}$.
- **GB wave vector Z**: Z component of the wave propagation vector $\hat{\delta}$.

It is not necessary to provide the vectors *GB polarization XYZ* and *GB wave vector XYZ* normalized, as they are internally normalized within ERMES 20.0; however, they must be provided perpendicular to each other. The mathematical form of the Gaussian beam boundary condition is:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} - ik_0 \left(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}\right) = -ik_0 \left(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}_{\text{GB}}\right) + ik_0 \left(\hat{\mathbf{n}} \times \hat{\delta} \times \mathbf{E}_{\text{GB}}\right), \tag{3.15}$$

For the regularized and LL2P formulations, the following extension of the divergence boundary condition is automatically applied inside ERMES 20.0:

$$\nabla \cdot \mathbf{E} - ik_0 \left(\hat{\mathbf{n}} \cdot \mathbf{E}\right) = -ik_0\, \hat{\mathbf{n}} \cdot \mathbf{E}_{\text{GB}} + ik_0\, \hat{\delta} \cdot \mathbf{E}_{\text{GB}} \tag{3.16}$$

In Appendix B can be consulted the adaptation of the above boundary conditions to all the different formulations implemented in ERMES 20.0.

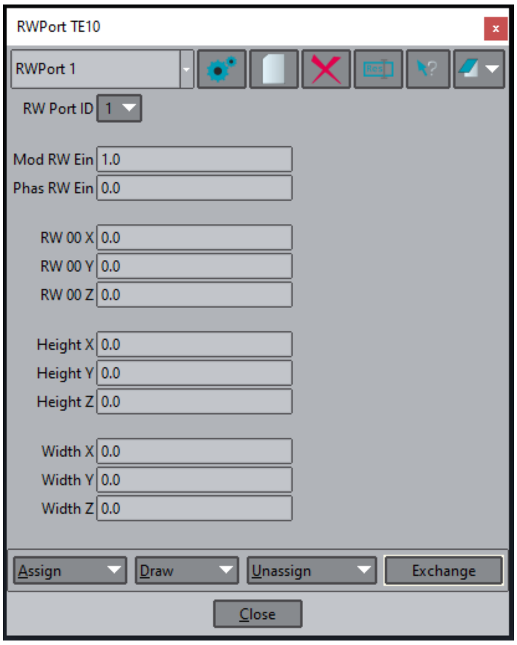

Figure 3.26: Rectangular waveguide port properties window.

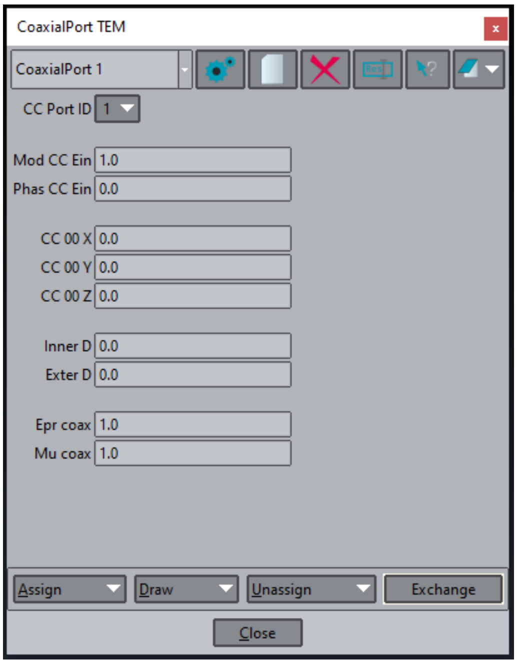

Figure 3.27: Coaxial waveguide port properties window.

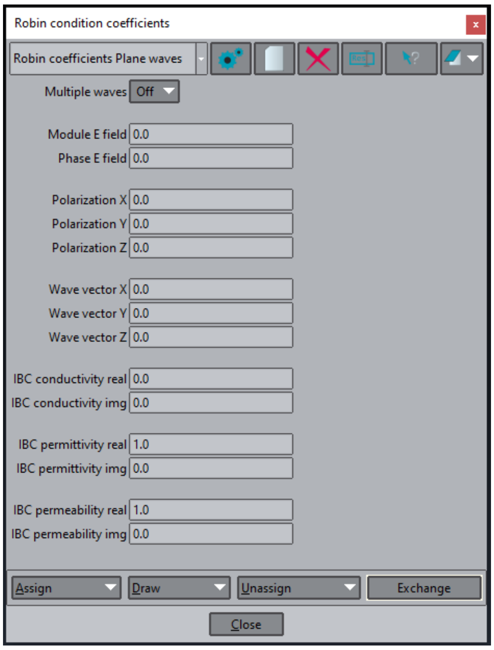

Figure 3.28: Plane waves properties window.

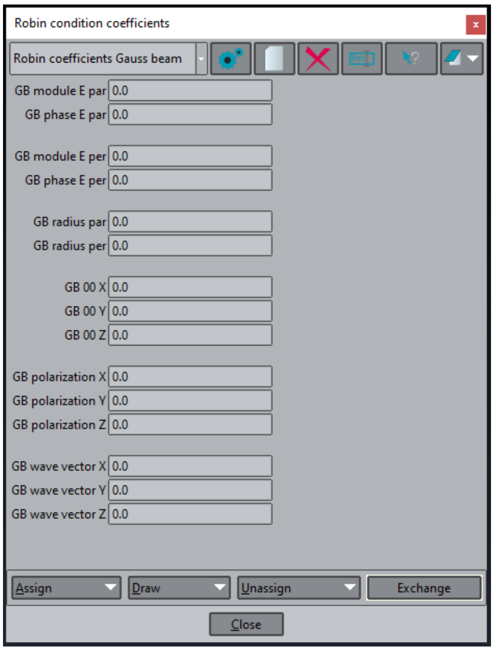

Figure 3.29: Gaussian beams properties window.

Quasi-static current flux - Figure 3.30

The quasi-static current flux source imposes a normal flux on the boundary through the following boundary condition:

$$\begin{aligned} \hat{\mathbf{n}} \times \nabla \times \mathbf{A} - \gamma(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{A}) =&\ 0, \\ \hat{\mathbf{n}} \cdot \omega^2 \varepsilon \left(\mathbf{A} + \nabla \phi^{'}\right) =&\ |F_n| e^{i\theta_n}, \end{aligned} \tag{3.17}$$

where γ is a complex number, and $|F_n|e^{i\theta_n}$ is the imposed normal flux. For the regularized and LL2P formulations, ERMES 20.0 automatically extends condition (3.17) with the following divergence term (see Appendix B):

$$\nabla \cdot \mathbf{A} - \gamma\,(\hat{\mathbf{n}} \cdot \mathbf{A}) = 0. \tag{3.18}$$

The values of γ, $|F_n|$, and θ_n are obtained from the data provided in window 3.30:

- **Robin coeff real**: real part of γ.
- **Robin coeff img**: imaginary part of γ.
- **Module Flux**: module of the imposed normal flux ($|F_n|$).
- **Phase Flux**: phase of the imposed normal flux (θ_n).

This boundary source is specifically designed for the $\mathbf{A} - \phi$ potentials formulations (see Appendix B). It is particularly effective at low frequencies when $\omega^2\varepsilon \approx i\omega\sigma^{'}$ or, equivalently, $\sigma^{'} \gg \epsilon^{'}\omega$ (assuming $\sigma^{''}$ and $\epsilon^{''}$ equal to zero, see definition (3.1)). Under these conditions, with $\gamma = 0$, the boundary source (3.17) directly imposes a normal current density $J_n = |F_n|e^{i\theta_n}$ on the boundary. This result follows from applying the approximation $\omega^2\varepsilon \approx i\omega\sigma^{'}$ in (3.17) and the fact that, in the $\mathbf{A} - \phi$ potentials formulations, $\mathbf{E} = i\omega(\mathbf{A} + \nabla\phi^{'})$ and $\mathbf{J} = \sigma^{'}\mathbf{E}$.

Additionally, condition (3.17) can also be used as a generic Robin boundary condition when $\gamma \neq 0$. As an example, it can be used to impose a quasi-static far-field condition on a boundary with a curvature radius ρ. This is achieved by setting $|F_n| = 0$ and defining the parameter γ as:

$$\gamma = \frac{1}{\rho\mu_0}. \tag{3.19}$$

Electrostatic current flux - Figure 3.31

The electrostatic current flux source imposes a normal flux on the boundary through the following boundary condition:

$$\hat{\mathbf{n}} \cdot (\eta \nabla \phi) + \gamma\phi = q, \tag{3.20}$$

where $\eta = \sigma^{'}$ when $\sigma^{'} > 0$ in the domain, and $\eta = \epsilon^{'}$ otherwise. The coefficients γ and q are real numbers obtained from the data provided in window 3.31:

- **Robin coeff**: Robin coefficient γ.
- **Flux**: Robin flux q.

This boundary source is only defined when the *Electrostatic* mode is activated (see Section 3.6). In *Electrostatic* mode, ERMES 20.0 solves the equation:

$$-\nabla \cdot (\eta \nabla \phi) = 0. \tag{3.21}$$

Recalling that in electrostatics, $\mathbf{E} = -\nabla \phi$, the equation is equivalent to $\nabla \cdot \mathbf{D} = 0$ or $\nabla \cdot \mathbf{J} = 0$, depending on whether $\eta = \epsilon^{'}$ or $\eta = \sigma^{'}$, respectively.

The boundary condition (3.20) can be used to inject a current density q into the domain by setting $\gamma = 0$, or as a far-field electrostatic condition by setting $q = 0$ and $\gamma = 1/\rho$, where ρ is the radius of curvature of the domain.

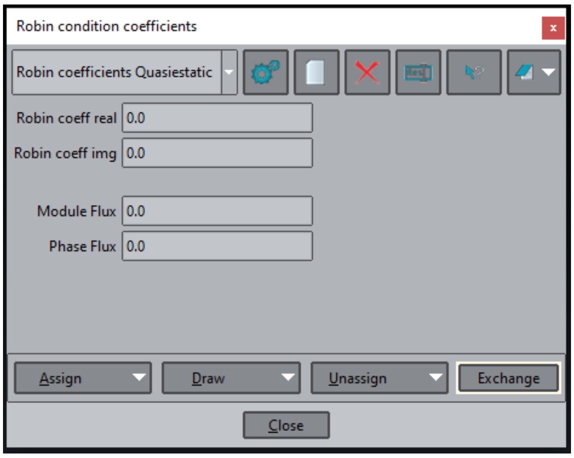

Figure 3.30: Current flux properties window for quasi-statics.

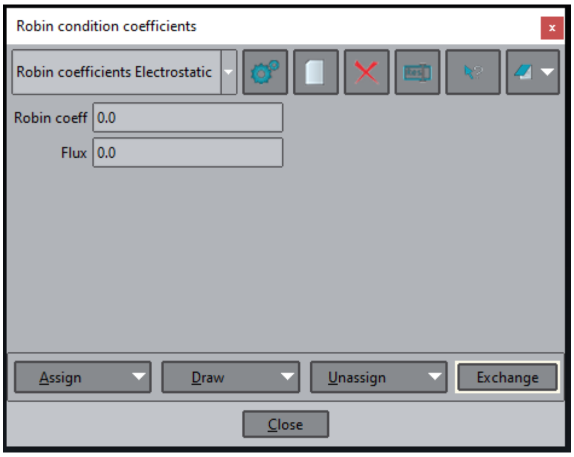

Figure 3.31: Current flux properties window for electrostatics.

3.5 Boundary conditions

In ERMES 20.0, two types of boundary conditions are available:

- Dirichlet boundary conditions,
- Robin boundary conditions.

Dirichlet boundary conditions impose constraints on the field values, while Robin boundary conditions impose constrains on the derivatives of the fields. This section provides a detailed explanation of all the available boundary conditions.

3.5.1 Dirichlet boundary conditions

The Dirichlet boundary conditions window 3.32 can be accessed by clicking on the *Dirichlet boundary conditions* icon in the ERMES menu bar 3.4:

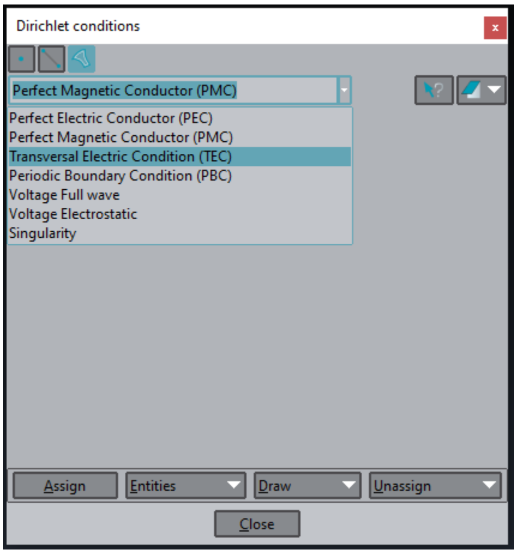

Figure 3.32: Dirichlet boundary conditions window.

In ERMES 20.0, Dirichlet conditions can be assigned to points, lines, and surfaces. The desired geometric object can be selected by choosing one of the icons in the upper-left corner of 3.32. Once the geometric object to which the condition will be applied is chosen, the appropriate boundary condition for that object must be selected. After selecting the condition, any additional options (which will be explained in the following) should be configured. Finally, the boundary condition is assigned to a point, line, or surface following a procedure similar to that illustrated in Figures 3.24 and 3.25, using the **Assign** button located in the lower-left corner of the Dirichlet boundary conditions window 3.32. The available Dirichlet conditions are the following:

- **Perfect Electric Conductor (PEC)** - [*Surfaces*]:
  For $\mathbf{E}$-field formulations, it imposes $\hat{\mathbf{n}} \times \mathbf{E} = 0$ on the surface. For $\mathbf{A} - \phi$ potential formulations, it sets $\hat{\mathbf{n}} \times \mathbf{A} = 0$ and $\phi = 0$ on the surface. For edge-stabilized formulations, it additionally sets the Lagrange multiplier φ equal to zero on the surface.

- **Perfect Magnetic Conductor (PMC)** - [*Surfaces*]:
  For regularized and LL2P $\mathbf{E}$-field formulations, it imposes $\hat{\mathbf{n}} \cdot \mathbf{E} = 0$ on the surface. For regularized and LL2P $\mathbf{A} - \phi$ potential formulations, it sets $\hat{\mathbf{n}} \cdot \mathbf{A} = 0$ on the surface. It is redundant for double-curl edge formulations. This condition is specifically designed for symmetry surfaces.

- **Transversal Electric Condition (TEC)** - [*Surfaces*]:
  For regularized and LL2P $\mathbf{E}$-field formulations, it imposes $\hat{\mathbf{n}} \cdot \mathbf{E} = 0$ on the surface. For regularized and LL2P $\mathbf{A} - \phi$ potential formulations, it sets $\hat{\mathbf{n}} \cdot \mathbf{A} = 0$ on the surface. It is redundant for double-curl edge formulations. This condition is specifically designed for waveguide port surfaces.

- **Periodic Boundary Condition (PBC)** - [*Surfaces*]:
  It enforces equal fields on two separate surfaces. The two surfaces must have identical meshes in order to apply this condition. To generate the same mesh on two separate surfaces, click on the GiD upper menu: **Geometry** → **Create** → **Contact** → **Separated Volume**, and select two surfaces. GiD will create the same triangular mesh on both surfaces and it will connect the triangles with prismatic elements. These prismatic elements provide information about how the triangles on the two separate surfaces are coupled.

  The meshing of the contact volumes can be disabled by clicking on the GiD upper menu: **Mesh** → **Mesh criteria** → **No mesh** → **Volumes**, and then selecting the contact volume to exclude from meshing. Turning the meshing for contact volumes off will prevent the creation of prismatic elements, while keeping identical meshes on the surfaces.

  Once the separate contact volumes are defined and its meshing disable, the PBCs can be applied. There are two pairs of PBC surfaces: **Front**-**Back** and **Left**-**Right**. The *Front-Back* surfaces must always lie parallel to the XY-plane. For cyclic periodic symmetries, the *Left* surface must be positioned in the 00-YZ plane and the axis of rotation must align with the Z-axis.

  If PBCs are only required between parallel surfaces, separate contact volume elements can be used instead. If the contact volume mesh is retained, ERMES 20.0 will automatically enforce equal field values on the opposing facing nodes.

  This condition is not defined for edge element formulations. To enforce equal fields on two separate surfaces with edge elements, define separate contact volumes and ensure that the contact volumes are meshed to retain the coupling information. This approach will work for both parallel and cyclic periodic symmetry surfaces when using edge elements.

- **Voltage Full wave** - [*Points* | *Lines* | *Surfaces*]:
  It imposes a time-harmonic voltage $|V_0|e^{i\theta_0}$ on a point, line, or surface. It is defined for the *Full wave* and *Cold plasma* modes.

- **Voltage Electrostatic** - [*Points* | *Lines* | *Surfaces*]:
  It imposes a electrostatic voltage V_0 on a point, line, or surface. It is defined for the *Electrostatic* mode.

- **Singularity** - [*Points* | *Lines* | *Surfaces*]:
  It sets the number of ungauged layers around a point, line, or surface (see Section B.3.2). In the regularized formulations, the divergence term of the elements inside the ungauged layers is removed. In the LL2P formulations, both the divergence term and the bubble contribution of the elements inside the ungauged layers are removed.

3.5.2 Robin boundary conditions

The Robin boundary conditions window 3.23 can be accessed by clicking on the *Robin boundary conditions* icon in the ERMES menu bar 3.4. In ERMES 20.0, Robin conditions can only be assigned to surfaces. Section 3.4.2 explains how to assign Robin boundary conditions to surfaces and how to define their coefficients. The following Robin conditions are available in ERMES 20.0:

- **Far field condition Full wave** - [*Surfaces*]:
  It applies the following time-harmonic far-field condition to a boundary placed in vacuum:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} - i\omega\sqrt{\epsilon_0\mu_0}\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) = 0. \tag{3.22}$$

  Appendix B describes the adaptation of (3.22) to the various FEM formulations implemented in ERMES 20.0.

- **Far field condition Cold plasma** - [*Surfaces*]:
  It applies the following time-harmonic far-field condition to a boundary enclosing a cold plasma:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} - ik_{cp}\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) = 0, \tag{3.23}$$

  where $k_{cp} = \omega\sqrt{\eta\,\epsilon_0\mu_0}$, with η being a position-dependent parameter calculated within ERMES 20.0. The parameter η can have different values depending on the type of cold plasma wave applied to the boundary. There are four possible choices:

  - **FW**: $\eta = RL/S$ (extraordinary wave).
  - **RW**: $\eta = R$ (right-handed polarized wave).
  - **LW**: $\eta = L$ (left-handed polarized wave).
  - **PW**: $\eta = P$ (ordinary wave).

where R, L, S, and P are components of the cold plasma permittivity tensor [73, 75]. If (3.23) is applied in vacuum, it becomes equal to (3.22). Appendix B shows the adaptation of (3.23) to the various FEM formulations implemented in ERMES 20.0.

- **Robin condition Full wave** - [*Surfaces*]:
  It applies a Robin boundary condition using the coefficients defined in:
  - **Robin coefficients Plane waves** 3.4.2 - 3.28.
  - **Robin coefficients Gauss beam** 3.4.2 - 3.29.
  - **Robin coefficients Quasiestatic** 3.4.2 - 3.30.

- **Robin condition Imported** - [*Surfaces*]:
  It applies the following generic Robin boundary condition [25]:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} + P(\mathbf{E}) = \mathbf{U}, \tag{3.24}$$

  where $P(\mathbf{E})$ and $\mathbf{U}$ are computed externally and subsequently imported into ERMES 20.0 from the files `Vector_U_IRBC.bin` and `Matrix_P_IRBC.bin`. These files must be located in the problem directory. An example illustrating both their generation and required format is provided in the problemtype folder `\Documents\Import_Scripts`. This boundary condition extends the capabilities of ERMES 20.0 by enabling multimode waveguide conditions [25], incorporating spatial Fourier transforms for coupling with other codes, and supporting advanced techniques such as domain decomposition methods.

- **Robin condition Electrostatic** - [*Surfaces*]:
  It applies a Robin boundary condition using the coefficients defined in *Robin coefficients Electrostatic* 3.4.2 - 3.31.

- **Coaxial TEM condition** - [*Surfaces*]:
  It applies a Robin boundary condition using the coefficients defined in *CoaxialPort TEM* 3.4.2 - 3.27.

- **RW port TE10 condition** - [*Surfaces*]:
  It applies a Robin boundary condition using the coefficients defined in *RWPort TE10* 3.4.2 - 3.26.

3.6 Problem settings

Problem parameters such as frequency, FEM formulation, and solver type are set in the *Solving parameters* window, which can be accessed via the icon of the same name on the ERMES menu bar 3.4. The *Solving parameters* window is divided into the five tabs shown in Figures 3.33 to 3.37. A detailed description of each parameter within each tab is provided below.

FEM formulation - Figure 3.33

- **Problem mode**: the available problem modes are:
  - **Full wave**: for solving problems in frequency domain with IHL materials.
  - **Electrostatic**: for solving electrostatic problems.
  - **Cold plasma**: for solving time-harmonic problems with a cold plasma.

- **Symmetry**: symmetry conditions applied to the main field across the entire problem domain. The main field can be either **E** or **A**, depending on the selected FEM formulation. Available options are:
  - **3D**: for solving a general three-dimensional problem.
  - **XY**: for solving only the X- and Y-components of the main field, while setting the Z-component to zero throughout the entire domain. This symmetry is only available in *Full wave* mode and is not supported for edge element formulations.
  - **ZZ**: for solving only the Z-component of the main field, while setting the X- and Y-component to zero throughout the entire domain. The main field is set to zero on all PEC surfaces. This symmetry is only available in *Full wave* mode and is not supported for edge element formulations.
  - **AY**: for solving an axisymmetric problem around the Y axis, with the θ- and r-components of the main field set to zero throughout the entire domain. The main field is set to zero on all PEC surfaces. This symmetry is only available in *Full wave* mode and is not supported for edge element formulations.

- **Element type**: this setting specifies the FEM formulation and the element order to be used in that formulation (see Appendix B):
  - **EDG 1st** : double-curl edge formulation with 1st order elements B.2.
  - **EDG 2nd**: double-curl edge formulation with 2nd order elements B.2.
  - **RME 1st** : regularized Maxwell equations method with 1st order nodal elements B.3.
  - **RME 2nd**: regularized Maxwell equations method with 2nd order nodal elements B.3.
  - **LL2P 3sb**: local L^2 projection method with 1st order nodal elements enriched with three surface bubbles plus an inner volumetric bubble B.4.

- **Potentials**: it selects between two different versions of the chosen FEM formulation:
  - **On** : the problem is solved using the $\mathbf{A} - \phi$ potentials version: B.2.1, B.3.3, B.4.1.
  - **Off**: the problem is solved using the **E**-field version: B.2, B.3, B.4.

- **RME stabilize**: it maintains or removes the divergence term throughout the entire domain in the regularized B.3 and LL2P B.4 formulations:
  - **On** : maintains the divergence term.
  - **Off**: removes the divergence term throughout the entire domain.

- **EDG stabilize**: it adds a stabilization term to the double-curl formulation B.2:
  - **On** : the problem is solved with the stabilized term B.2.2.
  - **Off**: the problem is solved without the stabilization term B.2-B.2.1.

- **LL2P stabilize**: it adds a stabilization term to the LL2P formulation B.4:
  - **On** : the problem is solved with the stabilized term B.4.2 with $h_k = 1$.
  - **Off**: the problem is solved without the stabilization term B.4-B.4.1.

- **Results mode**: it selects between three different results visualization modes:
  - **Nodes**: the results are given at the nodes.
  - **GP 1**: the results are given at the central Gauss points of the elements.
  - **GP 4**: the results are given at four internal Gauss points of the elements.

- **LL2P smooth**: it smooths the solution of the LL2P method. This option is useful for mitigating numerical oscillations that can arise because of oversampling when using bubble elements:
  - **On** : the smoothing is applied to the LL2P solution.
  - **Off**: the smoothing is not applied to the LL2P solution.

- **Solving mode**: it selects between three different solving modes:
  - **Release**: the problem is solved normally.
  - **Debug**: the problem is not solved, but some results are generated for debugging purposes. This option is useful for verifying that the cold plasma is properly set up or ensuring that the contact elements are well defined.
  - **Read**: the problem is not solved, but new results are generated after reading the `Vector_Xo.bin` file calculated previously. This option will create a new results file and is useful for adding results that were previously overlooked.

Geometry settings - Figure 3.34

- **Length units**: a multiplicative factor applied to all length values.
- **Geo error tol**: error tolerance for GiD geometric objects. This setting can help ensure accurate axis alignment in axisymmetric simulations.
- **Normals type**: criteria for computing the normals on PEC, PMC, and contact surfaces:
  - **Area weighted**: normals are calculated using a surface-weighted average.
  - **Geometric average**: normals are calculated as the average of the surrounding element normals, ignoring their relative sizes.
- **AV continuity**: allows for continuous or discontinuous normal components of the vector potential **A** across material interfaces:
  - **On** : the normal component of **A** is continuous across interfaces.
  - **Off**: the normal component of **A** can be discontinuous across interfaces.
- **LL2P keep div**: preserves or removes the divergence term of the LL2P formulation B.4 inside the ungauged layers when collapsing bubbles with the *Singularities* condition 3.5.1:
  - **On** : the divergence term is preserved inside the ungauged layers.
  - **Off**: the divergence term is removed inside the ungauged layers.

  This condition applies only to ungauged layers and is independent of the *RME stabilize* setting 3.6, which can remove the divergence term throughout the entire domain.
- **Contact detect**: this setting provides the option to consider or disregard meshed contact volumes in the analysis:
  - **Manual**: all the meshed contact volumes are considered.
  - **Ignore**: all the contact volumes are ignored.

  This option only applies to meshed contact volumes. To exclude any volume from the meshing process, navigate to the GiD upper menu: **Mesh** → **Mesh criteria** → **No mesh** → **Volumes** and select the desired volume.
- **PBC symmetry**: type of periodic boundary condition 3.5.1 used in the simulation:
  - **Cyclic**: the right face is tilted relative to the left face.
  - **Periodic**: the right face is parallel to the left face.

Import/Export files - Figure 3.35

- **Import Robin flux**: imports a generic Robin boundary condition from a set of files. These files must be located in the problem directory and named `Vector_U_IRBC.bin` and `Matrix_P_IRBC.bin`. An example illustrating how to generate them is provided in the folder `\Documents\Import_Scripts`. Two options are available:
  - **On** : applies the boundary condition using the provided files.
  - **Off**: the boundary condition is not applied.

- **Import J currents**: imports a volumetric current distribution **J** from a set of files. The files must be located inside the problem folder `\Export_J_Sources` and saved with the name: `Jsource-*.dat`, being * an integer number. The mesh used for the calculation of **J** should be the same as the mesh used in the ongoing simulation. Two options are available:
  - **On** : adds all **J** in `\Export_J_Sources\Jsource-*.dat`.
  - **Off**: no file is read.

- **Import ele matrix**: imports volumetric element matrices and vectors from a set of files. These files must be located in the problem folder and named `Matrix_K_IVEM.bin` and `Vector_R_IVEM.bin`. An example of how to generate these files is provided in the folder `\Documents\Import_Scripts`. Two options are available:
  - **On** : adds the volumetric element systems defined in the files.
  - **Off**: does not add any imported volumetric element system.

- **Export fields**: the complex results for **E**, **H**, **B** and **J** are saved in individual files within the problem folder `\Fields`. These files store the Cartesian components of the fields at all nodes within the surfaces and volumes selected in *Field integrals* 3.7.2. Additionally, two more files are saved, providing the list of nodes and elements associated with each selected surface and volume. Two options are available:
  - **On** : the fields, along with the lists of nodes and elements for each selected surface and volume, are written to separate files inside the problem folder `\Fields`.
  - **Off**: no files are generated to store the fields data.

- **Output format**: there are two possible formats of the results files:
  - **Binary**: a single binary file `.flavia.res` is written, containing both results and mesh data. This format saves disk space and is faster to read by GiD, but requires custom scripts for human user interpretation.
  - **Ascii**: two ASCII files are written: `.post.res` for results and `.post.msh` for mesh data. These files can be easily read by the user, but they are less space-efficient and slower to read by GiD.

Problem frequency - Figure 3.36

- **Angular frequency mode**: if this option is selected, the problem is solved for one angular frequency $\omega = 2\pi f$, being f the value provided in *Frequency*. All the fields are assumed to be time-harmonic as in A.3. This is the default mode.

- **Complex frequency mode**: if this option is selected, the problem is solved for one complex frequency $s = a + ib$, being a = *Real* s and b = *Imag* s. All the fields are assumed to vary with time as in A.3, but replacing $i\omega$ by s as, for instance, in $\mathbf{F}(\mathbf{r}, t) = Real\left[\mathbf{F}_c(\mathbf{r})e^{-st}\right]$. This option overrides *Angular frequency mode*, is limited to the *Full wave* mode, and only allows the use of Dirichlet and quasi-static Robin boundary conditions.

- **Sweep frequency mode**: if this option is selected, the problem is solved for a set of angular frequencies within the interval $[\omega_{ini}, \omega_{end}]$, with a step size of ω_{step}, being $\omega_{ini} = 2\pi f_{ini}$, $\omega_{end} = 2\pi f_{end}$, and $\omega_{step} = 2\pi f_{step}$. The values of f_{ini}, f_{end}, and f_{step} are obtained from the input fields: **Initial frequency**, **Final frequency**, and **Step frequency**, respectively. *Sweep frequency mode* takes precedence over *Angular frequency mode* and *Complex frequency mode*. Visual post-processing results will not be provided, but files containing fields, integrals, and scattering parameters for each frequency will be stored inside the problem folder.

Solver settings - Figure 3.37

- **Solver type**: this setting determines the solver used to solve the linear system. Three options are available:

  - **Quasi Minimal Residual**
  - **Bi Conjugate Gradient**
  - **External solver**

  The *Quasi Minimal Residual* and the *Bi Conjugate Gradient* are iterative solvers implemented inside ERMES 20.0, whose parameters are specified in *Internal solvers settings*. Although these solvers are characterized by low memory consumption, their convergence may be compromised for certain problems. For faster solving times of ill-conditioned matrices, the *External solver* option is recommended instead.

  If the *External solver* option is selected, the linear system will be saved in binary files within the problem folder 3.11. In *Full wave* and *Electrostatic* modes, the stiffness matrix will be saved in symmetric format. In *Cold plasma* mode, the stiffness matrix will be saved in the same format as specified in 3.3.2.

  After the matrix and vector are saved, the external solver will be called using the parameters specified in *External solvers settings*. Once the external solver completes its execution, ERMES 20.0 will be called again to read the solution and generate result files suitable for visualization. If the textboxes in *External solvers settings* are left blank, ERMES 20.0 will end immediately after saving the linear system. The saved system can then be read and solved at a later time.

- **Internal solvers settings**:
  - **Processors**: number of OpenMP threads used by the iterative solver.
  - **Tolerance**: convergence is achieved when $||Ax - b|| \,/\, ||b|| < Tolerance$.
  - **Preconditioner**: internal solver pre-conditioner.
  - **Max iterations**: maximum number of iterations permitted.
  - **Iteration step**: the residual $||Ax - b|| \,/\, ||b||$ will be printed to the *.info* file at the conclusion of each *Iteration step*. If the option *Every step* is selected in *Results in file*, the solution is saved in a file every *Iteration step*.
  - **Initial guess**: a solution file is read and used as an initial guess.
  - **Results in file**: saves the solution at *Every step* or only at the *Final step*.
- **External solvers settings**:
  - **Solver executable**: external solver executable or script to call it.
  - **Solver parameters**: input parameters for the external solver.

The problemtype folder `\External_Solvers` contains examples of how to use external solvers, including batch files, documentation, and scripts for calling the solvers and solve the linear system.

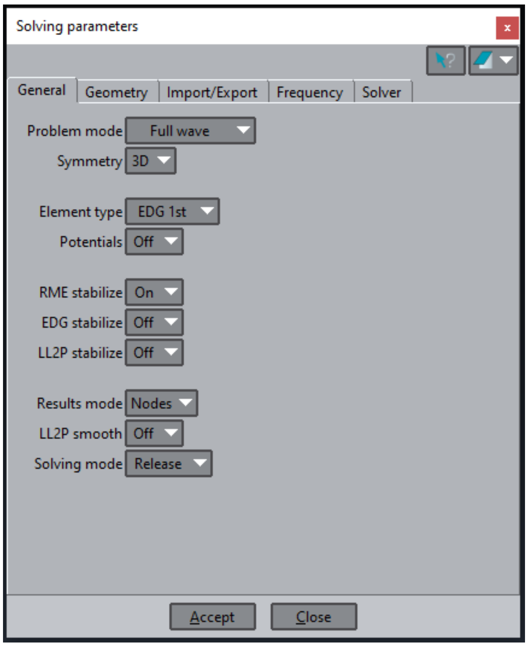

Figure 3.33: Solving parameters window - Tab: General.

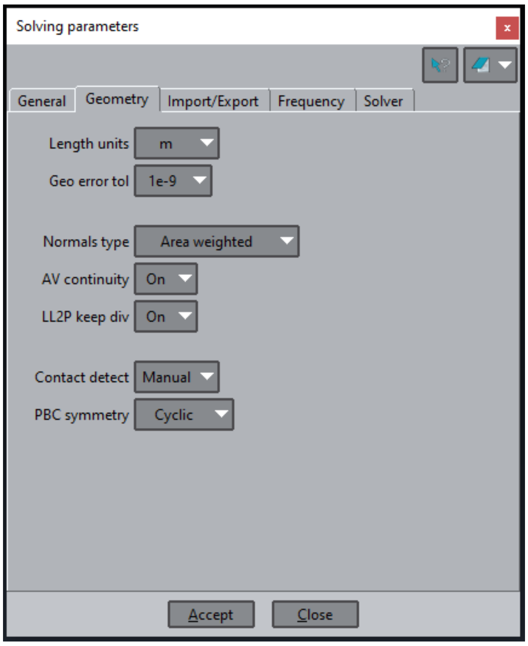

Figure 3.34: Solving parameters window - Tab: Geometry.

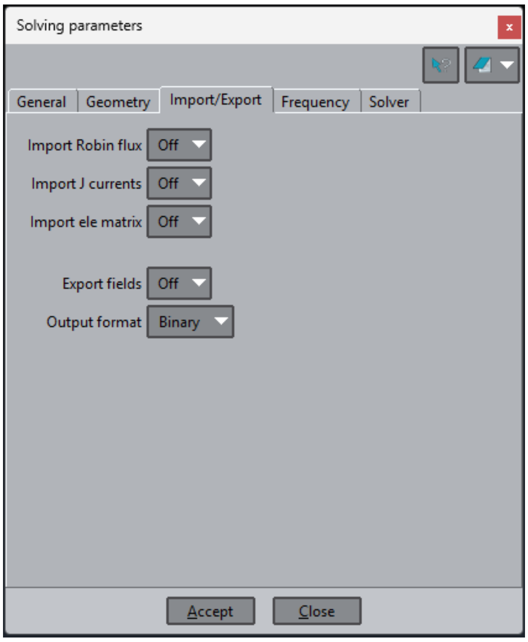

Figure 3.35: Solving parameters window - Tab: Import/Export.

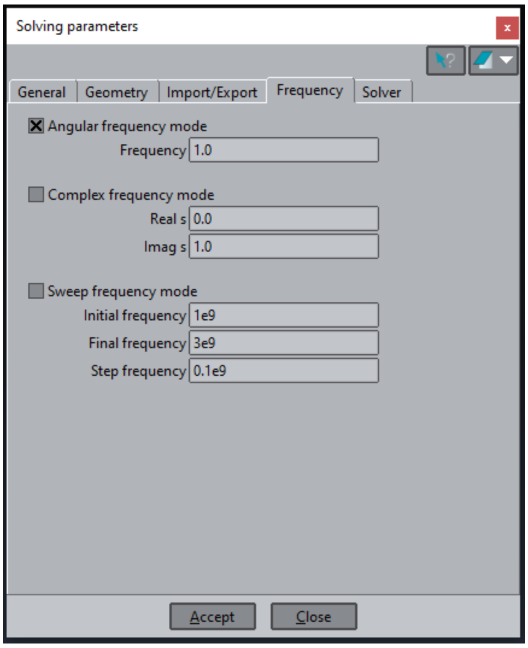

Figure 3.36: Solving parameters window - Tab: Frequency.

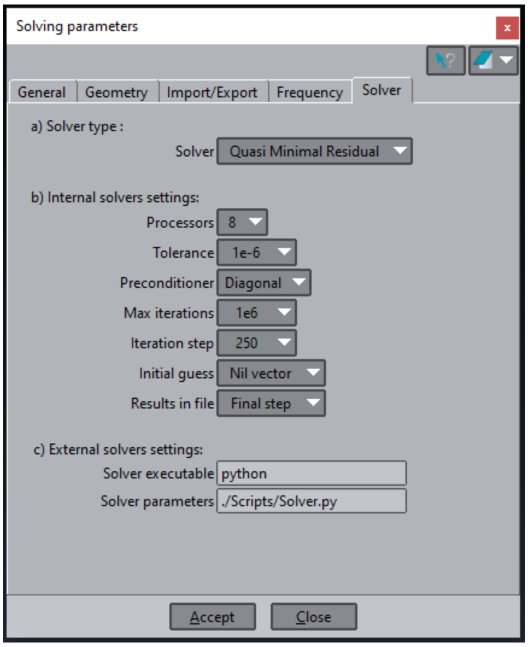

Figure 3.37: Solving parameters window - Tab: Solver.

3.7 Output results selection

The output results of ERMES 20.0 can be presented in three different formats:

1. Visualization,
2. Integrals,
3. Fields files.

The results for visualization are saved in the problem folder in one binary file `.flavia.res`, or in two ASCII files `.post.res` and `.post.msh`, depending on the output format selected in 3.6-3.35. These files can be directly read by the GiD post-processor to display the results, as presented in Chapter 4. The results to be visualized must be specified in the GiD pre-processor windows 3.38 to 3.41 before running the simulation. See Section 3.7.1 for further details.

The results can also be integrated over a surface or volume. This feature is particularly useful for computing parameters such as inductance, capacitance, resistance, total energy dissipation, or scattering parameters [48, 62, 45]. The surface or volume for field integration must be selected before running the simulation, using windows 3.42 and 3.43. The values of the integrals will be displayed in the *.info* file and saved in files within the folder `\Integrals`, located inside the problem folder. See Sections 4.1.1 and 3.7.2 for further details.

Finally, the output results can be saved in ASCII files with a simple format that is easy to read with other applications. These applications can further post-process the results to generate new data tailored for specific purposes. Results from specific regions of the problem domain can be saved in this format by selecting surfaces or volumes using windows 3.42 and 3.43 and enabling the *Export fields* option in 3.6-3.35. The files will be saved in a folder named `\Fields`, located inside the problem folder, along with information about the nodes and elements in those regions. See Section 3.7.3 for further details.

3.7.1 Visualization

As mentioned above, the results to be visualized in the GiD post-processor must be specified in the GiD pre-processor before running the simulation. This is done from the *Results* window, which can be opened by clicking the corresponding icon on the ERMES menu bar 3.4. The *Results* window is divided into four tabs, as shown in Figures 3.38 to 3.41. It is recommended to read Appendix A in advance to become familiar with the notation.

Frequency domain results - Figure 3.38

The *Frequency domain* tab is used to select the complex results and their time averages for visualization (see A.3 for further details). These results are stored at the time step t = 0.0 in GiD post-processor. The available options are:

- **E**: $Real\,[\,\mathbf{E}_c(\mathbf{r})\,]$, $Imag\,[\,\mathbf{E}_c(\mathbf{r})\,]$, $\|\,\mathbf{E}_c(\mathbf{r})\,\|$.
- **H**: $Real\,[\,\mathbf{H}_c(\mathbf{r})\,]$, $Imag\,[\,\mathbf{H}_c(\mathbf{r})\,]$, $\|\,\mathbf{H}_c(\mathbf{r})\,\|$.

- **B**: $Real\,[\,\mathbf{B}_c(\mathbf{r})\,]$, $Imag\,[\,\mathbf{B}_c(\mathbf{r})\,]$, $\|\,\mathbf{B}_c(\mathbf{r})\,\|$.
- **J**: $Real\,[\,\mathbf{J}_c(\mathbf{r})\,]$, $Imag\,[\,\mathbf{J}_c(\mathbf{r})\,]$, $\|\,\mathbf{J}_c(\mathbf{r})\,\|$, being $\mathbf{J}_c = \sigma_c \mathbf{E}_c + \mathbf{J}_c^{imp}$.
- **Poynting vector**: $\langle\,\mathbf{S}\,\rangle = \frac{1}{2}\,Real\,\left[\,\mathbf{E}_c \times \bar{\mathbf{H}}_c\,\right]$.
- **Joule heating**: $\langle\,Q\,\rangle = \frac{1}{2}\,\left(\sigma' + \omega\epsilon''\right)\,|\,\mathbf{E}_c\,|^2$.
- **Lorentz force**: $\langle\,\mathbf{f}\,\rangle = \frac{1}{2}\,Real\,\left[\,\mathbf{J}_c \times \bar{\mathbf{B}}_c\,\right]$.

Time domain results - Figure 3.39

The *Time Domain* tab selects the fields to visualize in time domain. These fields are calculated from the frequency domain results, as in A.3. These results are stored at the GiD post-processor time steps ranging from t = T/32 to t = T, where T $= 2\pi/\omega$ is the period of the harmonic oscillation and T/32 is the time step. The textbox showing the time interval in 3.39 is purely informative. The interval is fixed and cannot be changed from the interface. However, if desired, it can be modified in the template file `18-Solve_and_Print.bas` or its corresponding input file `Problem_Name-18.dat` (see Sections 3.9.1 and 3.9.2 for more details). Time domain results can generate large files on problems with large meshes, so use this options with care. The available options are:

- **E(t)**: $\mathbf{E}\,(\mathbf{r},t) = Real\,\left[\mathbf{E}_c(\mathbf{r})e^{-i\omega t}\,\right]$.
- **H(t)**: $\mathbf{H}\,(\mathbf{r},t) = Real\,\left[\mathbf{H}_c(\mathbf{r})e^{-i\omega t}\right]$.
- **B(t)**: $\mathbf{B}\,(\mathbf{r},t) = Real\,\left[\mathbf{B}_c(\mathbf{r})e^{-i\omega t}\right]$.
- **J (t)**: $\mathbf{J}\,\,(\mathbf{r},t) = Real\,\left[\,\mathbf{J}_c(\mathbf{r})e^{-i\omega t}\,\right]$.

Cold plasma results - Figure 3.40

The *Cold Plasma* tab adds magnitudes specific to the *Cold Plasma* mode to the *Frequency Domain* and *Time Domain* results. The cold plasma frequency domain results are stored at the GiD post-processor time step t = 0.0, while the cold plasma time domain results are stored at time steps ranging from t = T/32 to t = T. In this context, parallel ($\|$) and perpendicular ($\perp$) components are defined with respect to the direction of the external magnetic flux density B_e. For further details, it is recommended to consult the references [75, 73]. The available options when the *Cold plasma* mode is selected in 3.6 are:

- **Electron density**: electron density n_e, as provided in 3.3.2.
- **External B field**: applied external magnetic flux density B_e, as provided in 3.3.2.
- **Permittivity tensor**: real part of the cold plasma tensor components S, P, D, R, and L.
- **E perpendicular**: $Real\,[\,\mathbf{E}_{c\perp}(\mathbf{r})\,]$, $Imag\,[\,\mathbf{E}_{c\perp}(\mathbf{r})\,]$, $\|\,\mathbf{E}_{c\perp}(\mathbf{r})\,\|$.
- **E parallel**: $Real\,\left[\,\mathbf{E}_{c\|}(\mathbf{r})\,\right]$, $Imag\,\left[\,\mathbf{E}_{c\|}(\mathbf{r})\,\right]$, $\left\|\,\mathbf{E}_{c\|}(\mathbf{r})\,\right\|$.
- **E perpendicular (t)**: $\mathbf{E}_{\perp}(\mathbf{r},t) = Real\,\left[\mathbf{E}_{c\perp}(\mathbf{r})e^{-i\omega t}\,\right]$.
- **E parallel (t)**: $\mathbf{E}_{\|}(\mathbf{r},t) = Real\,\left[\mathbf{E}_{c\|}(\mathbf{r})e^{-i\omega t}\,\right]$.

Electrostatic results - Figure 3.41

The *Electrostatic* tab specifies the magnitudes to visualize when the *Electrostatic* mode is selected in 3.6. It also exports the calculated static current density to a file. This file can later be imported using the *Import currents* option in 3.6 to impose the same volumetric current density distribution in *Full Wave* or *Cold Plasma* mode calculations. This feature is particularly useful, for example, when computing quasi-static fields generated by coils with complex geometries. The available options when the *Electrostatic* mode is selected in 3.6 are:

- **Electrostatic potential**: ϕ.
- **Electrostatic E field**: $\mathbf{E} = -\nabla\phi$.
- **Electrostatic current density**: $\mathbf{J} = \sigma^{'}\mathbf{E}$.
- **Electrostatic Joule heating**: $Q = \sigma^{'} \mid \mathbf{E} \mid^2$.
- **Export electrostatic currents**: it writes the calculated electrostatic current density to the file `Jsource-nId.dat`, being `nId` the number provided in the textbox **Export Id**. This file will be stored inside the folder `\Export_J_Sources` within the problem folder. The current distribution will be multiplied by the phase $e^{i\varphi}$, being φ the phase provided in the textbox **Export Ph**, when imported into *Full Wave* or *Cold Plasma* mode simulations.

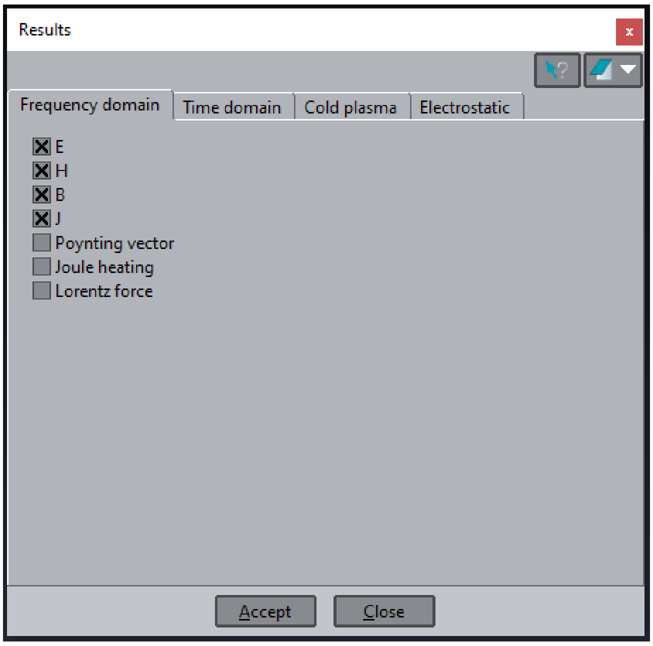

Figure 3.38: Results window - Tab: Frequency domain.

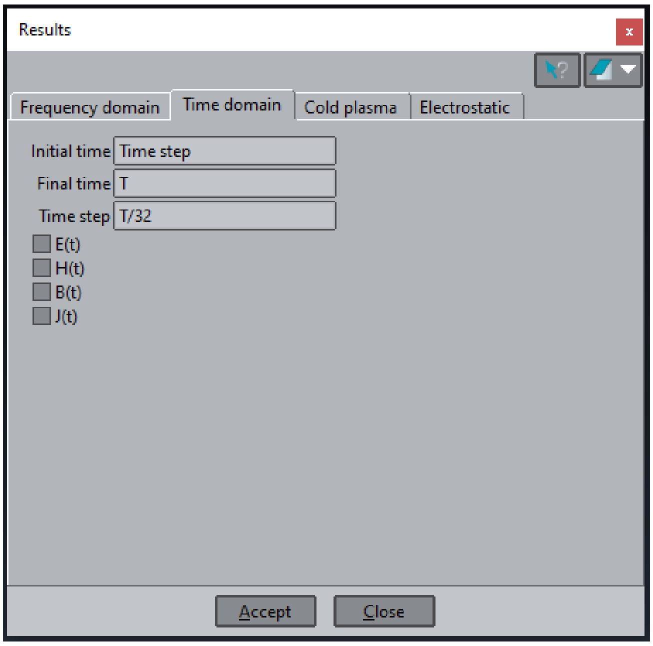

Figure 3.39: Results window - Tab: Time domain.

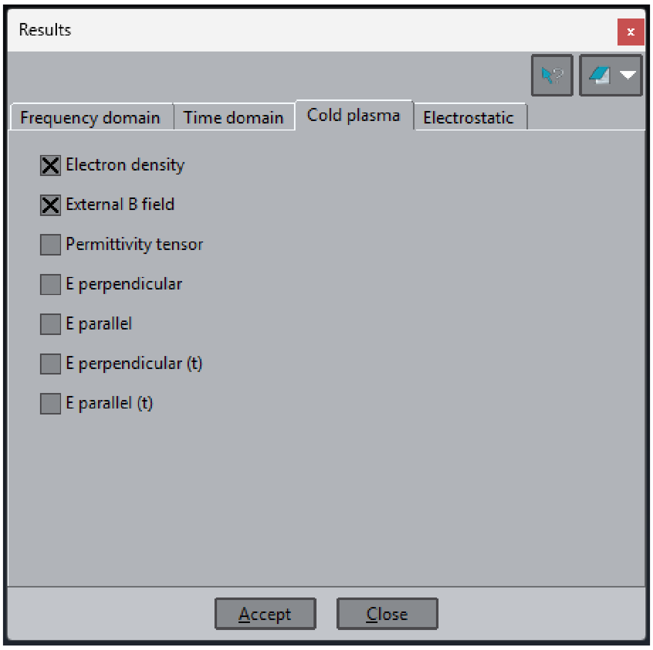

Figure 3.40: Results window - Tab: Cold plasma.

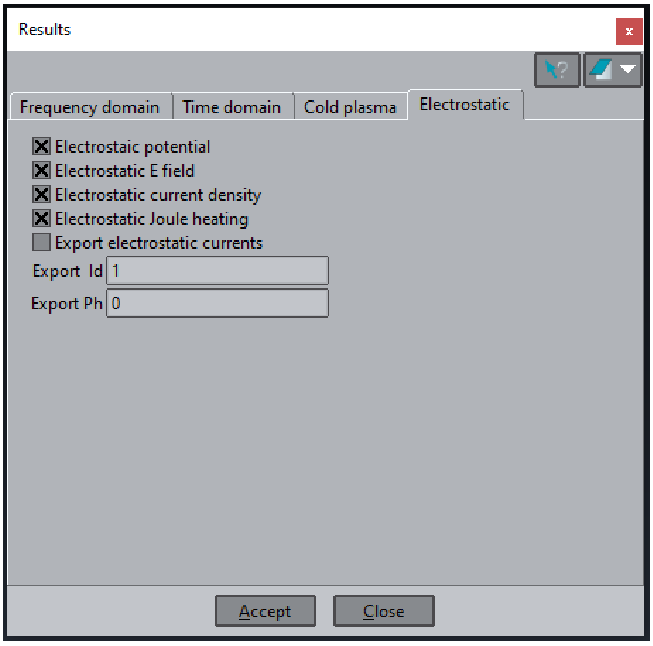

Figure 3.41: Results window - Tab: Electrostatic.

3.7.2 Integrals

From the *Fields integrals* window of Figures 3.42 and 3.43, a surface, volume, or waveguide port can be selected over which the fields will be integrated. This window can be opened by clicking on the *Results integrals* icon in the ERMES menu bar 3.4. The selection of a surface, volume, or waveguide port follows a procedure similar to that illustrated in Figures 3.5, 3.6, 3.24, and 3.25, but using the *Fields integrals* window 3.42-3.43 instead. The results of the calculated integrals will be shown for each frequency in the `.info` file and stored in files within the `\Integrals` folder, located inside the problem folder. Each surface, volume, or waveguide port will have its own file, named according to its identification number. The type of integrals calculated after selecting a surface, volume, or waveguide port are detailed in the next subsections.

Surface integrals

If the option *Field surface integral* in 3.42 is assigned to a surface, the following integrals will be automatically calculated:

- $\iint_S E_{cx}\, dS,\ \iint_S E_{cy}\, dS,\ \iint_S E_{cz}\, dS.$
- $\iint_S H_{cx}\, dS,\ \iint_S H_{cy}\, dS,\ \iint_S H_{cz}\, dS.$
- $\iint_S B_{cx}\, dS,\ \iint_S B_{cy}\, dS,\ \iint_S B_{cz}\, dS.$
- $\iint_S J_{cx}\, dS,\ \iint_S J_{cy}\, dS,\ \iint_S J_{cz}\, dS,\ \iint_S \mathbf{J}_c \cdot d\mathbf{S}.$

- $\iint_S |\mathbf{E}_c|\ dS,\ \iint_S |\mathbf{H}_c|\ dS,\ \iint_S |\mathbf{B}_c|\ dS,\ \iint_S |\mathbf{J}_c|\ dS.$
- $\iint_S |\mathbf{E}_c|^2\, dS,\ \iint_S |\mathbf{H}_c|^2\, dS,\ \iint_S |\mathbf{B}_c|^2\, dS,\ \iint_S |\mathbf{J}_c|^2\, dS.$
- $\iint_S \left(\mathbf{E}_c \times \bar{\mathbf{H}}_c\right)_x dS,\ \iint_S \left(\mathbf{E}_c \times \bar{\mathbf{H}}_c\right)_y dS,\ \iint_S \left(\mathbf{E}_c \times \bar{\mathbf{H}}_c\right)_z dS,\ \iint_S \left(\mathbf{E}_c \times \bar{\mathbf{H}}_c\right) \cdot d\mathbf{S}.$

The surface integral results will be available in the `Problem_Name.info` file located within the problem folder, where `Problem_Name` corresponds to the name of the problem being solved. The results will be also available in files named `Surface-sId-Integrals.dat`, where `sId` represents the identification number of the surface. These files will be stored in the `\Integrals` folder, located within the problem folder. The format of the `Surface-sId-Integrals.dat` files will be detailed in Section 4.1.3. If the *Sweep frequency mode* is selected, the surface integrals will be calculated for each frequency.

Scattering parameters

The scattering parameters of a waveguide are automatically calculated if one of the options, *Projection RWTE10* or *Projection CoaxialTEM*, in 3.42 is applied to a waveguide port surface. The scattering parameters are obtained using the following equations:

$$S_{ii} = \frac{\int_{\Gamma_i} (\,\mathbf{E}_c \ \times \mathbf{H}_m\,) \cdot \hat{\mathbf{n}}\ \mathrm{d}\Gamma_i}{\int_{\Gamma_i} (\,\mathbf{E}_m \times \mathbf{H}_m\,) \cdot \hat{\mathbf{n}}\ \mathrm{d}\Gamma_i} - 1, \tag{3.25}$$

$$S_{ji} = \frac{\int_{\Gamma_j} (\,\mathbf{E}_c \ \times \mathbf{H}_m\,) \cdot \hat{\mathbf{n}}\ \mathrm{d}\Gamma_j}{\int_{\Gamma_i} (\,\mathbf{E}_m \times \mathbf{H}_m\,) \cdot \hat{\mathbf{n}}\ \mathrm{d}\Gamma_i}, \tag{3.26}$$

where i and j are the identification number of the ports (see 3.26 and 3.27), Γ_i is the surface of the input port, Γ_j is the surface of the output port, $\mathbf{E}_c$ represents the total complex electric field at the port, $\mathbf{E}_m$ denotes the electric field of the imposed waveguide mode at the port (see 3.4.2 and 3.4.2), and $\mathbf{H}_m$ corresponds to the magnetic field of the imposed waveguide mode at the port. The field $\mathbf{H}_m$ is obtained from $\mathbf{E}_m$ using the time-harmonic Faraday's law (see Appendix A):

$$\mathbf{H}_m = \frac{1}{i\omega\mu}\ \nabla \times \mathbf{E}_m. \tag{3.27}$$

The scattering parameters results will be available in the `Problem_Name.info` file located within the problem folder, where `Problem_Name` corresponds to the name of the problem being solved. The results will be also available in files named `SParameter-S_j_i.dat`, where `j` and `i` represent the identification numbers of the ports. These files will be stored in the `\Integrals` folder, located within the problem folder. The format of the `SParameter-S_j_i.dat` files will be detailed in Section 4.1.5. If the *Sweep frequency mode* is selected, the scattering parameters will be calculated for each frequency.

Volume integrals

If the option *Field volume integral* in 3.43 is assigned to a volume, the following integrals will be automatically calculated:

- $\iiint_V E_{cx}\, dV$, $\iiint_V E_{cy}\, dV$, $\iiint_V E_{cz}\, dV$.
- $\iiint_V H_{cx}\, dV$, $\iiint_V H_{cy}\, dV$, $\iiint_V H_{cz}\, dV$.
- $\iiint_V B_{cx}\, dV$, $\iiint_V B_{cy}\, dV$, $\iiint_V B_{cz}\, dV$.
- $\iiint_V J_{cx}\, dV$, $\iiint_V J_{cy}\, dV$, $\iiint_V J_{cz}\, dV$.
- $\iiint_V |\mathbf{E}_c|\, dV$, $\iiint_V |\mathbf{H}_c|\, dV$, $\iiint_V |\mathbf{B}_c|\, dV$, $\iiint_V |\mathbf{J}_c|\, dV$.
- $\iiint_V |\mathbf{E}_c|^2\, dV$, $\iiint_V |\mathbf{H}_c|^2\, dV$, $\iiint_V |\mathbf{B}_c|^2\, dV$, $\iiint_V |\mathbf{J}_c|^2\, dV$.
- $\iiint_V (\mathbf{J}_c \times \mathbf{B}_c)_x\, dV$, $\iiint_V (\mathbf{J}_c \times \mathbf{B}_c)_y\, dV$, $\iiint_V (\mathbf{J}_c \times \mathbf{B}_c)_z\, dV$, $\iiint_V |\mathbf{J}_c \times \mathbf{B}_c|\, dV$.
- $\iiint_V (\mathbf{E}_c \cdot \bar{\mathbf{J}}_c)\, dV$.

The scattering parameters results will be available in the `Problem_Name.info` file located within the problem folder, where `Problem_Name` corresponds to the name of the problem being solved. The results will be also available in files named `Volume-vId-Integrals.dat`, where `vId` represents the identification number of the volume. These files will be stored in the `\Integrals` folder, located within the problem folder. The format of this file will be detailed in Section 4.1.4. If the *Sweep frequency mode* is selected, the volume integrals will be calculated for each frequency.

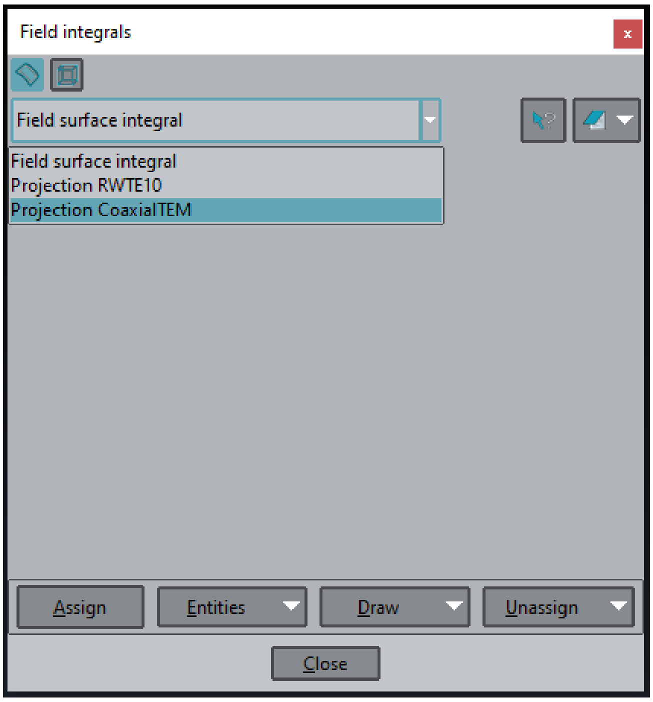

Figure 3.42: Field integrals window - Surfaces.

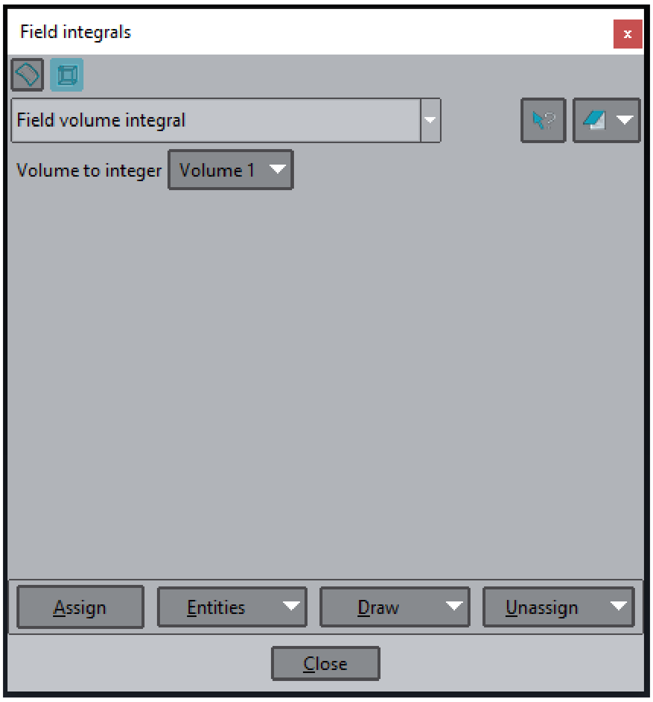

Figure 3.43: Field integrals window - Volumes.

3.7.3 Fields files

If the *Export Fields* option in 3.6-3.35 is set to *On*, and there are surfaces or volumes with the *Field Surface Integral* or *Field Volume Integral* options assigned to them, the fields $\mathbf{E}c$, $\mathbf{B}c$, $\mathbf{H}c$, and $\mathbf{J}c$ for those surfaces or volumes will be saved for each node on those surfaces or volumes in files within the `\Fields` folder, located in the problem folder.

If surfaces are assigned, the fields will be saved to a set of files named `Surface_sId_E.dat`, `Surface_sId_B.dat`, `Surface_sId_H.dat`, and `Surface_sId_J.dat`, where `sId` represents the surface's identification number. Similarly, if volumes are assigned, the fields will be saved to files named `Volume_vId_E.dat`, `Volume_vId_B.dat`, `Volume_vId_H.dat`, and `Volume_vId_J.dat`, where `vId` represents the volume's identification number.

Additionally, the following files will be stored in the same folder, providing information about the nodes and elements of the mesh for the selected surfaces and volumes: `Surface_sId_Nodes.dat`, `Surface_sId_Elements.dat`, `Volume_vId_Nodes.dat`, and `Volume_vId_Elements.dat`. The format of all these files will be detailed in Section 4.1.6.

3.8 Meshing

The last step before clicking the *Calculate* button is to mesh the geometry. This must be done after assigning sources, boundary conditions, and materials. Any reassignment of sources, boundary conditions, or materials will require re-meshing of the geometry. This does not include changes in its values; re-meshing is only required if they are reassigned to a different geometric entity. The meshing process can be started by clicking on the *Generate mesh* icon in the ERMES menu bar 3.4 or from the upper menu of GiD: **Mesh** → **Generate mesh...**. The size of the mesh elements must take into account that almost ten elements per wavelength may be necessary to obtain a well defined solution, and that the wavelength changes depending on the medium under consideration according to the following formula for IHL materials:

$$\lambda = \frac{c}{f\sqrt{\varepsilon_r \mu_r}} \tag{3.28}$$

where f is the problem frequency, c is the speed of light in vacuum ($\sim 3x10^8$m/s), ε_r is the relative electric permittivity of the IHL material, and μ_r is the relative magnetic permeability of the IHL material. For cold plasmas, the wavelength depends on the location, plasma conditions and the wave modes that propagate in each region (see references [75, 73] for further details).

Different mesh sizes can be assigned to points, lines, surfaces, or volumes from the GiD upper menu: **Mesh** → **Unstructured** → **Assign sizes on** [**points** | **lines** | **surfaces** | **volumes**]. The transition speed from one size to another can be modified in the *Preferences Window* 3.44, which can be opened from the GiD upper menu: **Utilities** → **Preferences...**. Then, in the left menu of the *Preferences Window*, click on: **Meshing** → **General**. Additional mesh parameters can be modified from the *Meshing* menu of the *Preferences Window*, as shown in Figure 3.45. For further details on meshing with GiD, refer to the tutorials in **Help** → **Tutorials...** or online [15].

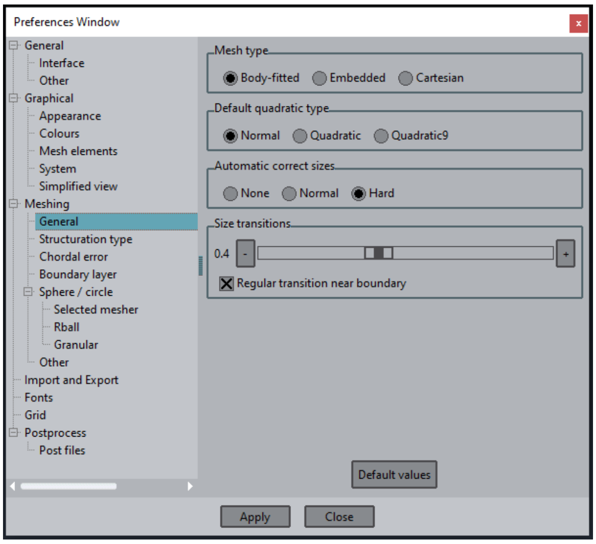

Figure 3.44: Preferences window - Meshing - General.

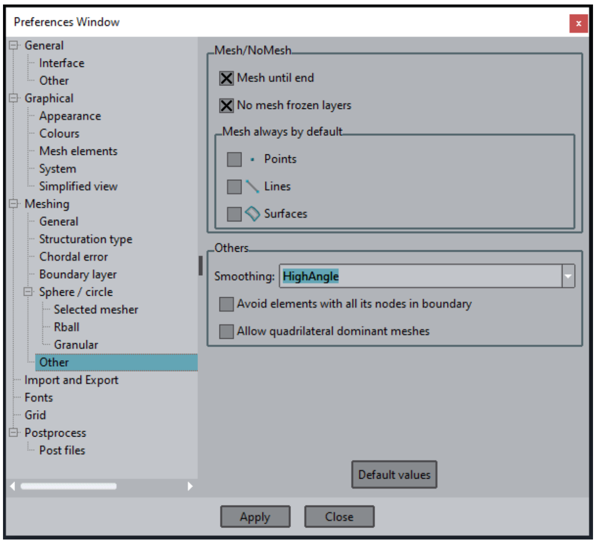

Figure 3.45: Preferences window - Meshing - Other.

3.9 ERMES input files

ERMES 20.0 input files are automatically generated when you click the *Calculate* icon in the ERMES menu bar 3.4. These files are saved in the problem folder and are re-generated each time you click on the *Calculate* icon. ERMES 20.0 is launched immediately after creating the input files. Alternatively, you can generate the input files without launching ERMES 20.0, via the GiD upper menu: **Files** → **Export** → **Calculation file...**. This option is especially useful for batch calculations, enabling the user to modify directly input files and run parametrized ERMES simulations without the need of re-opening GiD each time a parameter is changed.

ERMES 20.0 input files are created from a set of templates located in the problem type folder `\ERMES.gid`. This set of templates extract data from the GiD pre-processor and generate input files in the problem folder according to the format defined by the templates. The process of creating input files is completely automatic and transparent for a general user. However, advanced users can benefit from a detailed description of this process to further customize ERMES 20.0. This section will describe the templates and input files, as well as additional files required by ERMES 20.0, such as those containing cold plasma electron density and magnetic field distributions, and pre-calculated electric current densities.

3.9.1 Template files

The problem type folder `\ERMES.gid` contains all the essential files required to create the GiD-ERMES 20.0 interface. These files can be edited, allowing for the addition of new windows, data fields, or custom actions to be executed before or after invoking ERMES 20.0. Below is a list of the files that define and control the functionality of the GiD-ERMES 20.0 interface:

- `ERMES.cnd`: defines boundary conditions windows,
- `ERMES.mat`: defines materials windows,
- `ERMES.prb`: defines parameters windows,
- `ERMES.tcl`: defines ERMES menu bar,
- `ERMES.uni`: defines units,
- `ERMES.win.bat`: calls ERMES on Windows,
- `ERMES.unix.bat`: calls ERMES on Linux systems.

The directory `\ERMES.gid` also contains a set of `.bas` template files that process the data provided by the GiD-ERMES interface to create `.dat` input files in the specific format required by ERMES 20.0. Below is a list of these template files, along with a brief description of the input data file they generate:

- `ERMES.bas`: problem definition and global settings.
- `01-Nodes_IdCoords.bas`: list of all mesh nodes.
- `02-Nodes_Voltages.bas`: list of nodes with a voltage assigned.
- `03-Nodes_Singular.bas`: list of nodes with a singularity assigned.
- `04-Volume_Elements.bas`: list of all mesh volumetric tetrahedral elements.
- `05-SourceJ_Elements.bas`: list of **J** source volumetric elements.
- `06-Dirichlet_Elements.bas`: list of PEC, PMC, and TEC surface elements.
- `07-RobinBC_Elements.bas`: list of Robin condition surface elements.
- `08-Periodic_Elements.bas`: list of PBC surface elements.
- `09-Contact_Elements.bas`: list of contact elements.
- `10-Projector_Elements.bas`: list of projection elements.
- `11-Materials_Properties.bas`: list of materials and their properties.
- `12-Cold_Plasma.bas`: cold plasma parameters.
- `13-Plane_Waves.bas`: plane waves parameters
- `14-External_Solver.bas`: external solver parameters.
- `15-Export_Current.bas`: electrostatic export current parameters.
- `16-Angular_Freq.bas`: angular frequency value.
- `17-Cmplex_Freq.bas`: complex frequency value.
- `18-Solve_and_Print.bas`: list of results to visualize.
- `19-Import_Robin_Elements.bas`: list of imported Robin condition elements.

These `.bas` template files are thoroughly commented to facilitate easy reading and modification. If an interface other than GiD is preferred, these files serve as the starting point. The `.bas` files describe how information is extracted from the GiD interface and how it should be formatted for ERMES 20.0 to read. A new interface would only need to replicate this data acquisition and formatting process.

3.9.2 Problem files

The `.bas` template files generate `.dat` input files in the problem folder, which are subsequently read by ERMES 20.0 to initiate the calculations. If a problem named `Problem_Name` is being solved, the generated input files will be saved in the folder `\Problem_Name.gid` and will be named according to the following convention:

- `ERMES.bas` → `Problem_Name.dat`,
- `01-Nodes_IdCoords.bas` → `Problem_Name-1.dat`,
- `02-Nodes_Voltages.bas` → `Problem_Name-2.dat`,
- `03-Nodes_Singular.bas` → `Problem_Name-3.dat`,
- `04-Volume_Elements.bas` → `Problem_Name-4.dat`,
- `05-SourceJ_Elements.bas` → `Problem_Name-5.dat`,
- `06-Dirichlet_Elements.bas` → `Problem_Name-6.dat`,
- `07-RobinBC_Elements.bas` → `Problem_Name-7.dat`,
- `08-Periodic_Elements.bas` → `Problem_Name-8.dat`,
- `09-Contact_Elements.bas` → `Problem_Name-9.dat`,
- `10-Projector_Elements.bas` → `Problem_Name-10.dat`,
- `11-Materials_Properties.bas` → `Problem_Name-11.dat`,
- `12-Cold_Plasma.bas` → `Problem_Name-12.dat`,
- `13-Plane_Waves.bas` → `Problem_Name-13.dat`,
- `14-External_Solver.bas` → `Problem_Name-14.dat`,
- `15-Export_Current.bas` → `Problem_Name-15.dat`,
- `16-Angular_Freq.bas` → `Problem_Name-16.dat`,
- `17-Cmplex_Freq.bas` → `Problem_Name-17.dat`,
- `18-Solve_and_Print.bas` → `Problem_Name-18.dat`.
- `19-Import_Robin_Elements.bas` → `Problem_Name-19.dat`.

These files contain all the information required by ERMES 20.0 to perform the simulation. Once created, they can be modified to launch new simulations without the need of further intervention from GiD.

3.9.3 Cold plasma files

The *Cold Plasma* mode requires information about the electron density distribution and the applied static **B** field. This information is provided through files that must be formatted as described in this section. Examples of the various file formats can also be found in the problem type folder `\Documents\Plasma_Files\Plasma_Samples`.

The file format will depend on the *Geometry ext type* selected (see Section 3.3.2). As a general rule, lines that start with a number are read and processed by ERMES. Conversely, lines beginning with anything other than a number (e.g., a blank space or the symbol #) are treated as comment lines and ignored by ERMES. Numerical data columns must be separated by one or more blank spaces. When writing these files, it is important to avoid blank spaces at the start of numerical data lines, as a blank space at the beginning of a line will be interpreted as a comment.

If the *Geometry ext type* option selected is *Flat 1D* or *Curved*, the electron density file must have the following general format:

```
# Comments
L(m)     ne(m-3)
```

where *L(m)* represents the column with the distance in meters from the point *L1 00 XYZ* provided in 3.8, and *ne(m-3)* represents the column with the electron density in m^{-3}. For more details on how this file is projected onto the 3D geometry, see Section 3.3.2. An example of this type of file is provided below:

```
# JPN      95645
# Time     48.7
# L(m)     ne(m-3)
0.000     0.00E+00
0.013     0.00E+00
0.027     0.55E+15
0.040     0.96E+16
0.054     1.01E+16
...
```

For the applied static **B** field, if the *Geometry ext type* option selected is *Flat 1D* or *Curved*, the static **B** field file must have the following general format:

```
# Comments
L(m)     B(T)     teta(deg)
```

where *L(m)* represents the column with the distance in meters from the point *L1 00 XYZ* using the same distance points as the electron density file above, *B(T)* represents the column with the magnitude of the static **B** field, and *teta(deg)* represents the column with the angle in degrees formed by the **B** vector with the plane perpendicular to the tokamak axis *TK axis XYZ*, as viewed from *TK 00 XYZ* towards *L1 00 XYZ*. An example of this type of file is provided below:

```
# JPN      95645
# Time     48.7
# L(m)   B(T)    teta(deg)
0.000     2.55    8.974
0.013     2.56    9.013
0.027     2.57    9.053
```

```
0.040    2.58    9.096
0.054    2.59    9.142
...
```

If a point falls outside the interval defined by the above files, the last closest values of the electron density distribution and the applied static **B** field will be assigned to that point.

If the *Geometry ext type* option is set to *Full 3D*, the electron density and the static **B** field must be provided at the mesh nodes. The indices and coordinates of these nodes can be found in the problem file `Problem_Name-1.dat`. If a node does not have an assigned density or **B**, it will be assumed to have a zero value. The file for the electron density must have the following general format:

```
# Comments
NodeId    ne(m-3)
```

where the column *NodeId* represents the identification number of the mesh nodes, and the column *ne(m-3)* represents the electron density in m^{-3} at the nodes. An example of this type of file is provided below:

```
# Electron density 3D format
# NodeId     ne(m-3)
1     0
2     1.994428193439744e+18
3     6.612102289630953e+18
4     2.230372750985475e+19
5     2.564435389731727e+19
6     2.461668655832341e+19
...
```

For the applied static **B** field, if the *Geometry ext type* option selected is *Full 3D*, the static **B** field file must have the following general format:

```
# Comments
NodeId   Bx(T)   By(T)   Bz(T)
```

where *NodeId* represents the column with the identification number of the mesh nodes, and *Bx(T)*, *By(T)*, and *Bz(T)* represents the columns with the Cartesian components of the applied static **B** field at that node in Tesla. An example of this type of file is provided below:

```
# External B 3D format
# NodeId   Bx(T)   By(T)   Bz(T)
1    -2.097315788269043  -0.8615391254425049   0.32111769914627080
2    -2.060964345932007  -0.9514395594596863   0.32016107439994810
3    -2.141749382019043  -0.8083917498588562   0.33365988731384280
4    -2.279344797134399  -0.6750744581222534   0.36515107750892640
5    -2.329360485076904  -0.8477968573570251   0.37611362338066100
6    -2.203813076019287  -1.0339540243148800   0.38065534830093380
...
```

Equilibrium plasma files in *eqdsk* format can be converted into the ERMES 20.0 format using Python scripts. Example scripts for converting *eqdsk* files into the corresponding *Full 3D* format are provided in `\Documents\Plasma_Files` along with sample *eqdsk* files.

3.9.4 Current density files

ERMES 20.0 can import an externally computed electric current density distribution, **J**, and use it as a volumetric source (see Section 3.6). The current density **J** must be stored in files located in the problem folder `\Export_J_Sources` and saved with the name `Jsource-*.dat`, where * is an integer. The mesh used to export **J** must match the mesh used in the simulation that imports the current. One option is to export **J** directly from the ERMES 20.0 *Electrostatic* mode, as described in Section 3.7.1; in this case, the output is automatically formatted to meet the required file specifications. Alternatively, the Python scripts available in the problemtype folder `\Documents\Import_Scripts\Current_J_Scripts` can be used to convert current density data from an *eqdsk* plasma equilibrium file into the ERMES 20.0 format. Three different file formats are supported, depending on whether **J** is applied element-wise or nodal-wise. The element-wise format supports either a constant or a variable phase for each element. The element-wise constant-phase format is as follows:

```
Phase(rad)
NodeId_1  NodeId_2  NodeId_3  NodeId_4  Jx(A/m2)  Jy(A/m2)  Jz(A/m2)
```

where *Phase(rad)* represents the phase φ in radians of $\mathbf{J}_c = \mathbf{J}e^{i\varphi}$, which is the complex current density imported into ERMES 20.0. *NodeId_1*, *NodeId_2*, *NodeId_3*, and *NodeId_4* are the identification numbers of the nodes in a tetrahedral mesh element, and *Jx(A/m2)*, *Jy(A/m2)*, and *Jz(A/m2)* represent the Cartesian components of the **J** assigned to that element in A/m^2. An example of this type of file is provided below:

```
0
36  23  32  22  1678.92  1.58440  0.0593253
73  53  54  59  1680.52  10.2272  -0.446468
97  89  92  75  1678.81  11.4579  0.1569060
73  70  77  75  1678.71  9.60589  -0.068501
74  61  59  83  1683.99  12.3864  -0.247362
50  32  34  48  1679.22  5.09186  0.0544152
...
```

The format above assigns the same phase to all elements in the file. To assign a different phase to each element, the following format must be used:

```
V
NodeId_1  NodeId_2  NodeId_3  NodeId_4  |Jx|  Ph_x  |Jy|  Ph_y  |Jz|  Ph_z
```

where the character *V* in the first line indicates the element-wise variable-phase format, $|J_x|$, $|J_y|$, and $|J_z|$ denote the magnitudes of the Cartesian components of **J** in A/m^2, and *Ph_x*, *Ph_y*, and *Ph_z* represent the phases in rad of each Cartesian component. An example of this file format is provided below:

```
V
36  23  32  22  1678.92  0.00  1200.54  3.14  0.1000  1.15
73  53  54  59  1000.00  0.00  1250.26  6.14  0.1500  2.15
97  89  92  75  1500.80  0.00  1330.56  3.16  0.1020  1.30
73  70  77  75  1400.30  0.00  1200.37  2.14  0.1000  1.15
74  61  59  83  1111.32  0.00  1207.26  1.14  0.1300  1.40
50  32  34  48  1600.58  0.00  1102.58  3.10  0.1600  2.15
...
```

The third supported format for imported current sources is the nodal-wise format, which is specified as follows:

```
N
NodeId   |Jx|  Ph_x   |Jy|  Ph_y   |Jz|  Ph_z
```

where the symbol *N* in the first line indicates the nodal-wise format; $|J_x|$, $|J_y|$, and $|J_z|$ denote the magnitudes of the Cartesian components of **J** in A/m^2; and *Ph_x*, *Ph_y*, and *Ph_z* represent the phases in rad of each Cartesian component. The indices and coordinates of the mesh nodes can be found in the problem file `Problem_Name-1.dat`. If a node does not have an assigned **J**, it will be assumed to have a zero value. An example of this file format is provided below:

```
N
1  1678.92  0.00  1200.54  3.14  0.1000  1.15
2  1000.00  0.00  1250.26  6.14  0.1500  2.15
3  1500.80  0.00  1330.56  3.16  0.1020  1.30
4  1400.30  0.00  1200.37  2.14  0.1000  1.15
5  1111.32  0.00  1207.26  1.14  0.1300  1.40
6  1600.58  0.00  1102.58  3.10  0.1600  2.15
...
```

3.10 Batch scripts

ERMES 20.0 can be called from batch scripts to automate parametrized calculations. Since batch scripts are highly dependent on the operating system, it is recommended to consult the system documentation for proper script syntax, grant executable rights when needed, and address any other configuration requirements before using this option.

Examples of batch scripts used with ERMES 20.0 in High Performance Computer (HPC) clusters can be found in the problemtype folder: `\Documents\Batch_Scripts`. These scripts are well-commented, making them easy to understand and adapt for your specific setup. Additionally, there are scripts demonstrating how to use GiD off-screen mode to assign materials, define boundary conditions, and generate the mesh without the GiD graphical interface.

3.11 External solvers

ERMES 20.0 gives the possibility of using external libraries to solve the sparse complex linear system generated by its FEM formulations. The workflow when using external solvers within the GiD user interface is as follows:

1. User selects *External_solver* as *Solver_type* option 3.6.
2. User sets the rest of problem parameters and clicks on the *Calculate* button.
3. ERMES 20.0 builds the linear system.
4. ERMES 20.0 saves the system matrix and vector in the problem folder.
5. ERMES 20.0 calls the external library.

6. The external library reads the linear system and solves it.
7. The external library saves the results vector in the problem folder.
8. ERMES 20.0 reads the results vector and generates the results file.
9. GiD reads the results file for the visualization and analysis of results.

The linear system $AX_o = B$ is stored in the problem folder within the following binary files:

1. `Matrix_A_cmplx.bin`: matrix complex coefficients a_{ij},
2. `Matrix_A_int.bin`: matrix indices ij,
3. `Matrix_A_aux_cmplx.bin`: auxiliary matrix complex coefficients c_{mn},
4. `Matrix_A_aux_int.bin`: auxiliary matrix indices mn,
5. `Vector_B.bin`: linear system vector B,
6. `Vector_Xo.bin`: results vector X_o.

The system matrix is stored in different formats depending on the active problem mode. The available formats are:

1. **Symmetric**: stores the diagonal and upper diagonal elements of the system matrix in the binary file `Matrix_A_cmplx.bin` with the corresponding indices for these elements in the binary file `Matrix_A_int.bin`.

2. **Full-matrix**: stores the entire system matrix in the binary file `Matrix_A_cmplx.bin`, along with its corresponding indices in the binary file `Matrix_A_int.bin`.

3. **Hermitic-Symmetric**: stores two separate matrices: the first matrix contains the diagonal and upper-diagonal contributions from the volumetric elements, stored in `Matrix_A_cmplx.bin` and `Matrix_A_int.bin`. The second matrix contains the diagonal and upper-diagonal contributions from the Robin boundary condition elements, stored in `Matrix_A_aux_cmplx.bin` and `Matrix_A_aux_int.bin`.

4. **Hermitic-Full**: stores two separate matrices: the first matrix contains the diagonal and upper-diagonal contributions from the volumetric elements, saved in `Matrix_A_cmplx.bin` and `Matrix_A_int.bin`. The second matrix is a complete representation of the contributions from the Robin boundary condition elements, stored in `Matrix_A_aux_cmplx.bin` and `Matrix_A_aux_int.bin`.

In *Full wave* and *Electrostatic* modes, the stiffness matrix will be always saved in *Symmetric* format. In *Cold plasma* mode, the stiffness matrix can be saved in *Full-matrix*, *Hermitic-Symmetric*, or *Hermitic-Full*, depending on the selection made in 3.3.2.

The linear system files described above can be read and transformed to the format required by any external library using the scripts provided in the problemtype folder `\External_Solvers`. For instance, in the folder `\External_Solvers\Python` there are two examples of Python scripts (`SuperLU.py` and `BiCG.py`) that use the library Python NumPy [44] to read, transform, and solve the linear system. Further details can be found in the comments written inside those scripts.

The folder `\External_Solvers\PETSc` contains all the files required to interface ERMES 20.0 with PETSc [55]. It includes the PETSc manual, a `README.txt` file with installation instructions, a Bash script for invoking PETSc from the Windows Subsystem for Linux (WSL), and a Python script to read, transform, and solve the linear system generated by ERMES 20.0 using PETSc. In addition, the subfolder `\External_Solvers\PETSc\Solver` contains the PETSc executable together with its corresponding C++ source code, `PETScSolver.cpp`, which is called by the Python script. Further details are provided in the comments within these files.

Chapter 4

Post-process

This chapter provides a detailed explanation of all the output files generated by ERMES 20.0 after completing a computation. It also demonstrates how to use the GiD post-processor to visualize and analyse the results.

4.1 Output files

Several output files are generated by ERMES 20.0 upon completing the calculations. These output files can be categorized into the following groups:

- Progress file,
- Visualization files,
- Surface integrals files,
- Volume integrals files,
- Scattering parameters files,
- Fields on nodes files.

In the following subsections, each group of output files will be detailed, including examples and format specifications. These files can be utilized to review the results, process them further using scripts, and conduct analyses. The results can be visualized using the GiD post-processor or, if preferred, with any other post-processing tool after reformatting them to be compatible with the chosen software.

4.1.1 Progress file

The progress of an ERMES 20.0 computation is logged in the `Problem_Name.info` file located within the problem folder, where `Problem_Name` corresponds to the name of the problem currently being executed. This file can be opened directly with any text editor or viewed within the GiD pre-processor by clicking on **Calculate** → **View process info...** in the GiD upper menu.

The Problem_Name.info file contains details about the problem's main settings, the residual error evolution of the solvers, and, upon completion of the calculation, the results of surface and volume integrals, as well as the scattering parameters, if any of these outputs were selected as described in Section 3.7.2. An example of a Problem_Name.info file is shown below:

```
ERMES analysis started...

Problem type: Full wave
Geometry type: 3D
Element type: RME 1st
AV potentials: Off
Stabilization: On
Angular freq: 1e+09 Hz
[...]
--------------------------------------------------------
[...]
|S11| = -1.772862e-01 dB [ 9.797961e-01, -7.970637e-01 rad ]
|S21| = -1.409555e+01 dB [ 1.973433e-01, -2.818825e+00 rad ]
--------------------------------------------------------
[...]
--------------------------------------------------------
Volume[1]: 1.75e-07 m^3
[...]
VolIntg( Ey ) = 6.608089e-04 * exp( j * -3.046219e+00 )
VolIntg( Ez ) = 1.883837e-07 * exp( j * -7.014302e-01 )
[...]
VolIntg( (JxB)x ) = +3.768553e-06 + j * +4.910668e-01
VolIntg( (JxB)y ) = +6.360195e-09 + j * -2.813885e+00
[...]
--------------------------------------------------------
[...]
--------------------------------------------------------
Surface[3]: 5e-05 m^2
[...]
SrfIntg( Bx ) = 2.000487e-09 * exp( j * +3.482370e-01 )
SrfIntg( By ) = 4.660085e-11 * exp( j * -2.664756e+00 )
[...]
SrfIntg( |E| ) = 1.721156e+00
SrfIntg( |H| ) = 7.823491e-02
[...]
--------------------------------------------------------
[...]
Printing vector iE...
Printing vector rE...
Printing double mod(E)...
Printing vector iH...
[...]

ERMES analysis finished.
```

The surface integral results 3.7.2 are denoted as `SrfIntg(.)`, and the surface is identified as `Surface[sId]`, where `sId` represents the unique identification number of the surface. Similarly, the volume integral results 3.7.2 are denoted as `VolIntg(.)`, and the volume is identify as `Volume[vId]`, where `vId` represents the unique identification number of the volume. The scattering parameters 3.7.2 are presented in decibels as `Sji(dB) = 20*log10(|Sji|)` and in polar form as `[|Sji|, phase(Sji) rad]`.

4.1.2 Visualization files

By default, ERMES 20.0 saves the simulation results in a binary file named `Problem_Name.flavia.res`, where `Problem_Name` corresponds to the name of the problem being solved. This file is located in the problem folder and can be read directly by the GiD post-processor. Section 4.2 provides instructions on how to visualize and analyse this file with the GiD post-processor. The file is generated with the help of the `gidpost` library, located in the source code folder `\ERMES_CPP_20.0\external_libraries\gidpost`. Additional details about this library are available in the GiD documentation.

If the *Ascii* option is selected in the *Output format* setting of 3.6-3.35, two files will be generated: `Problem_Name.post.res` and `Problem_Name.post.msh`. The first file contains the results, while the second provides information about the mesh. These files are human-readable and can be easily processed by other applications. However, they are slower to load in the GiD post-processor and consume more disk space compared to the binary format. An example of the mesh file `Problem_Name.post.msh` is provided below:

```
MESH "HOMesh" dimension 3 ElemType Tetrahedra Nnode 4
Coordinates
1 0 0.01 0
2 0 0.01 0.0005
3 0 0.00926775 0
4 0.000760075 0.01 0
[...]
1249 0.045 0.000741483 0.0005
1250 0.045 0 0
1251 0.045 0 0.0005
End Coordinates
Elements
1 36 23 32 22 1
2 73 53 54 59 1
3 97 89 92 75 1
4 73 70 77 75 1
[...]
3396 582 619 616 610 2
3397 598 616 595 579 2
3398 562 600 572 561 2
End Elements
```

The file lists first the coordinates of each node in the mesh, followed by the connectivity of these nodes in the tetrahedral mesh, along with the material identification number assigned to each element. For the results file `Problem_Name.post.res`, an example is provided below:

```
GiD Post Results File 1.0
GaussPoints "GPsEM_01" ElemType Tetrahedra
Number Of Gauss Points: 1
Natural Coordinates: Internal
End GaussPoints
GaussPoints "GPsEM_04" ElemType Tetrahedra
Number Of Gauss Points: 4
Natural Coordinates: Internal
End GaussPoints
Result "iE" "ERMES" 0 Vector OnNodes
Values
1 -354.667 368.144 0
2 -430.484 446.843 0
3 0.188047 0 0
[...]
1250 -0.00123668 0.0012949 0
1251 -0.0010276 0.00107598 0
End Values
Result "rE" "ERMES" 0 Vector OnNodes
Values
1 16.7959 -17.4341 0
2 23.6727 -24.5723 0
[..]
1250 0.0121584 -0.0127307 0
1251 0.00950394 -0.00995131 0
End Values
Result "mod(E)" "ERMES" 0 Scalar OnNodes
Values
1 511.766
2 621.409
[...]
1250 0.0176947
1251 0.0138408
End Values
[...]
Result "E(t)" "ERMES" 1.03125e-09 Vector OnNodes
Values
1 -52.7189 54.7222 0
2 -60.7654 63.0745 0
[...]
1250 0.0116835 -0.0122335 0
1251 0.00912084 -0.00955018 0
End Values
```

In the example above, results are presented for each node in the mesh. Alternatively, results can be provided at Gauss points by selecting either the *GP 1* or *GP 4* options in the *Results mode* setting, as described in 3.6-3.33. For further details about these files and their format, refer to the GiD documentation.

4.1.3 Surface integrals files

When the *Field surface integral* option in 3.42 is assigned to a surface, the integrals described in 3.7.2 are automatically computed and saved in a file named `Surface-sId-Integrals.dat`, where `sId` represents the surface's identification number. This file is stored in the `\Integrals` folder within the problem directory. The format of `Surface-sId-Integrals.dat` is as follows:

```
# Header
Freq  |siEx|  phsiEx  |siEy|  phsiEy  |siEz|  phsiEz  si(|E| )  si(|E|^2)
Freq  |siHx|  phsiHx  |siHy|  phsiHy  |siHz|  phsiHz  si(|H| )  si(|H|^2)
Freq  |siBx|  phsiBx  |siBy|  phsiBy  |siBz|  phsiBz  si(|B| )  si(|B|^2)
Freq  |siJx|  phsiJx  |siJy|  phsiJy  |siJz|  phsiJz  si(|J| )  si(|J|^2)
Freq  |siSx|  phsiSx  |siSy|  phsiSy  |siSz|  phsiSz  si(|Jn|)  si(r(Sn))
```

where `# Header` refers to a block of comments at the beginning of the file, outlining the file format and specifying the identification number of the surface. Each line in the header starts with the symbol `#`. Following the header, the results of the integrals are saved in polar form. `Freq` represents the frequency at which the integrals are calculated. `Freq` can be f or s depending on the frequency mode selected in 3.36-3.6. `si` denotes a complex surface integral, being `|si|` the magnitude of the complex integral and `phsi` its phase. `S` refers to a magnitude twice the time average of the Poynting vector `S` = $2\langle\mathbf{S}\rangle = \mathbf{E}_c \times \bar{\mathbf{H}}_c$. `|Jn|` is the magnitude of the normal component of the current density at the surface, and `r(Sn)` represents the real part of the normal component of `S`. If the *Sweep frequency mode* option in 3.6-3.36 is selected, this block will be repeated for each frequency within the interval specified in 3.36.

4.1.4 Volume integrals files

When the *Field volume integral* option in 3.43 is assigned to a volume, the integrals described in 3.7.2 are automatically computed and saved in a file named `Volume-vId-Integrals.dat`, where `vId` represents the volume's identification number. This file is stored in the `\Integrals` folder within the problem directory. The format of `Volume-vId-Integrals.dat` is as follows:

```
# Header
Freq  |viEx|  phviEx  |viEy|  phviEy  |viEz|  phviEz  vi(|E|)  vi( |E|^2 )
Freq  |viHx|  phviHx  |viHy|  phviHy  |viHz|  phviHz  vi(|H|)  vi( |H|^2 )
Freq  |viBx|  phviBx  |viBy|  phviBy  |viBz|  phviBz  vi(|B|)  vi( |B|^2 )
Freq  |viJx|  phviJx  |viJy|  phviJy  |viJz|  phviJz  vi(|J|)  vi( |J|^2 )
Freq  |viFx|  phviFx  |viFy|  phviFy  |viFz|  phviFz  vi(|F|)  vi(r(J*.E))
```

where `# Header` refers to a block of comments at the beginning of the file, outlining the file format and specifying the identification number of the volume. Each line in the header starts with the symbol `#`. Following the header, the results of the integrals are saved in polar form. `Freq` represents the frequency at which the integrals are calculated. `Freq` can be f or s depending on the frequency mode selected in 3.36-3.6. `vi` denotes a complex volume integral, being `|vi|` the magnitude of the complex integral and `phvi` its phase. `F` represents the cross product $\mathbf{J}_c \times \mathbf{B}_c$, and `r(J*.E)` = $Real[\bar{\mathbf{J}}_c \cdot \mathbf{E}_c]$. If the *Sweep frequency mode* option in 3.6-3.36 is selected, this block will be repeated for each frequency within the interval specified in 3.36.

4.1.5 Scattering parameters files

When the *Projection RWTE10* or *Projection CoaxialTEM* options in 3.42 are assigned to a waveguide port surface, the scattering parameters 3.7.2 are automatically calculated and saved in a file named `SParameter-S_j_i.dat`, where `j` and `i` denote the identification numbers of the ports. This files is stored in the `\Integrals` folder within the problem directory. The format of the `SParameter-S_j_i.dat` file is as follows:

```
# Header
Freq    Sji(dB)    |Sji|    phSji
```

where `# Header` refers to a block of comments at the beginning of the file, outlining the file format and specifying the identification number of the waveguide port. Each line in the header begins with the symbol `#`. Following the header, the scattering parameters are provided in decibels and polar form. In this case, unlike the surface and volume integral files, `Freq` represents only the frequency f at which the scattering parameters are calculated, noting that complex frequencies s are not available for waveguide port boundary conditions. `Sji(dB)` = $20\log_{10}(|Sji|)$, `|Sji|` is the module of the scattering parameter S_{ji}, and `phSji` its phase. If the *Sweep frequency mode* option in 3.6-3.36 is selected, this line will be repeated for each frequency within the specified interval.

4.1.6 Fields on nodes files

If the *Export fields* option in 3.6-3.35 is set to *On*, and there are surfaces or volumes with the *Field surface integral* or *Field volume integral* options assigned to them, the fields $\mathbf{E}_c$, $\mathbf{B}_c$, $\mathbf{H}_c$, and $\mathbf{J}_c$ for those surfaces or volumes will be saved to files in the `\Fields` folder, located within the problem folder. If there are the surfaces assigned, the fields will be saved to files named:

- `Surface_sId_E.dat`,
- `Surface_sId_B.dat`,
- `Surface_sId_H.dat`,
- `Surface_sId_J.dat`,

where `sId` represents identification number of the surface. If there are the volumes assigned, the fields will be saved to files named:

- `Volume_vId_E.dat`,
- `Volume_vId_B.dat`,
- `Volume_vId_H.dat`,
- `Volume_vId_J.dat`,

where `vId` represents the identification number of the volume. Moreover, a set of files containing information about the nodes and elements of the mesh of the selected surfaces and volumes will also be saved alongside the field files in the `\Fields` folder. The names of the files detailing the mesh information are listed below:

- `Surface_sId_Nodes.dat`,
- `Surface_sId_Elements.dat`,
- `Volume_vId_Nodes.dat`,
- `Volume_vId_Elements.dat`.

The format of the files `Surface_sId_Nodes.dat` and `Volume_vId_Nodes.dat` is the following:

```
# Freq(Hz)
NodeId X(m) Y(m) Z(m)
```

where `Freq(Hz)` is the problem frequency in Hz, which can be f or s depending on the frequency mode selected (see 3.36-3.6), `NodeId` is the identification number of a node on the selected surface or volume, and `X(m)`, `Y(m)`, and `Z(m)` are the Cartesian coordinates of the node in m. The format of the file `Surface_sId_Elements.dat` is:

```
# Freq(Hz)
NodeId_1 NodeId_2 NodeId_3
```

where `NodeId_1`, `NodeId_2`, and `NodeId_3` represent the identification numbers of the nodes of a triangular element on the selected surface. A similar format is used for the selected volumes in the files named `Volume_vId_Elements.dat`:

```
# Freq(Hz)
NodeId_1 NodeId_2 NodeId_3 NodeId_4
```

where `NodeId_1`, `NodeId_2`, `NodeId_3`, and `NodeId_4` represent the identification numbers of the nodes of a tetrahedral element on the selected volume. The complex electric field $\mathbf{E}_c$ on the selected surfaces and volumes will be stored in the files `Surface_sId_E.dat` and `Volume_vId_E.dat` with the format:

```
# Freq(Hz)
Real(Ecx) Imag(Ecx) Real(Ecy) Imag(Ecy) Real(Ecz) Imag(Ecz)
```

where `Ecx`, `Ecy`, and `Ecz` are the Cartesian components of the complex electric field $\mathbf{E}_c$ in V/m. The complex magnetic flux density $\mathbf{B}_c$ on the selected surfaces and volumes will be stored in the files `Surface_sId_B.dat` and `Volume_vId_B.dat` with the format:

```
# Freq(Hz)
Real(Bcx) Imag(Bcx) Real(Bcy) Imag(Bcy) Real(Bcz) Imag(Bcz)
```

where `Bcx`, `Bcy`, and `Bcz` are the Cartesian components of the complex magnetic flux density $\mathbf{B}_c$ in T. The complex magnetic field $\mathbf{H}_c$ on the selected surfaces and volumes will be stored in the files `Surface_sId_H.dat` and `Volume_vId_H.dat` with the format:

```
# Freq(Hz)
Real(Hcx) Imag(Hcx) Real(Hcy) Imag(Hcy) Real(Hcz) Imag(Hcz)
```

where `Hcx`, `Hcy`, and `Hcz` are the Cartesian components of the complex magnetic field $\mathbf{H}_c$ in A/m. The complex electric current density $\mathbf{J}_c$ on the selected surfaces and volumes will be stored in the files `Surface_sId_J.dat` and `Volume_vId_J.dat` with the format:

```
# Freq(Hz)
Real(Jcx) Imag(Jcx) Real(Jcy) Imag(Jcy) Real(Jcz) Imag(Jcz)
```

where `Jcx`, `Jcy`, and `Jcz` are the Cartesian components of the complex electric current density $\mathbf{J}_c$ in A/m^2. The order of the field files aligns with the nodes file, such that the first line of a field file corresponds to the value of the field at the first node in the nodes file, and so on. If the *Sweep frequency mode* is selected, the fields for each frequency will be stored in their corresponding files, with each frequency block separated by the line: `# Freq(Hz)`.

4.2 GiD postprocess

The GiD post-processor can be accessed from the GiD pre-processor upper menu by selecting **Files** $\rightarrow$ **Postprocess** or by clicking the rainbow icon located in the upper-right corner of the GiD pre-processor. To return to the GiD pre-processor from the GiD post-processor, select **Files** $\rightarrow$ **Preprocess** or click on the corresponding icon in the upper-right corner of the GiD post-processor. If the problem has been loaded in pre-process, switching to post-process will automatically open the corresponding results file. However, if the post-processor is opened empty, the results files `.flavia.res` or `.post.res` must be manually loaded from **Files** $\rightarrow$ **Open**.

Once the results are loaded in the post-processor, it may be necessary to frame the geometry by clicking on **View** $\rightarrow$ **Zoom** $\rightarrow$ **Frame**. After framing, open the *View Results* window 4.1 from the upper menu **Window** $\rightarrow$ **View results...**. In the *View Results* window, choose a **View** (e.g., *Contour Fill*, *Display Vectors*, etc.) and then select a magnitude to visualize. Complex magnitudes are stored in **Step**: **0.0**, whereas time-domain values are stored in the subsequent time steps.

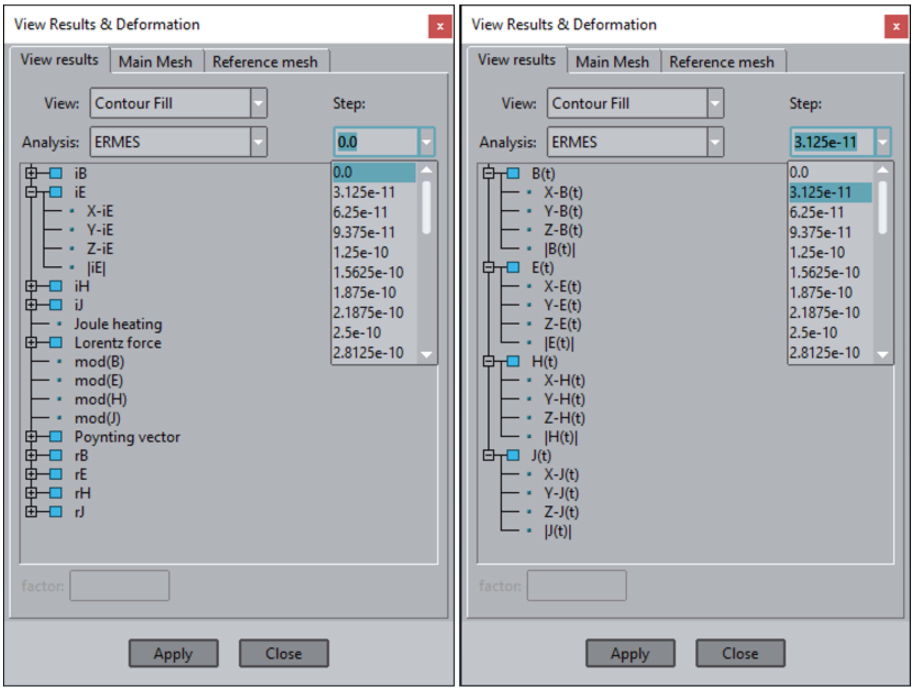

Figure 4.1: *View Results* window displaying the available results for two different time steps.

The upper menu **Window** provides access to various result-related windows, such as:

- **View results...**: selects results to visualize (see Figure 4.1).
- **Animate...**: animates time-domain results and records videos.
- **View graphs...**: displays and creates customized graphs.
- **View style...**: selects layers and adjust rendering styles.

The upper menu **View results** provides access to various result visualization styles and operations, including: **Contour fill**, **Contour lines**, **Show min max**, **Display vectors**, **Iso surfaces**, **Stream lines**, **Graphs**, **Results surface**, and **Integrate**.

Visualization settings can be adjusted through the *Preferences Window*. This window is accessible via the upper menu **Utilities** → **Preferences...**. Visualization settings can be modified in the **Postprocess** section of the *Preferences Window*. For instance, under **Countour fill and lines**, you can configure the colour scale and the number of colours displayed. Under **Vectors**, you can adjust vector density and appearance.

The GiD post-processor also provides tools for slicing the geometry, removing layers to improve results visibility, completing symmetric results, capturing pictures and videos, and more. It is recommended to follow the GiD tutorials available in **Help** → **Tutorials...** or consult the online documentation [15] to maximize the potential of the GiD post-processor. Examples of ERMES 20.0 applications and the GiD post-processing capabilities are shown in the following figures.

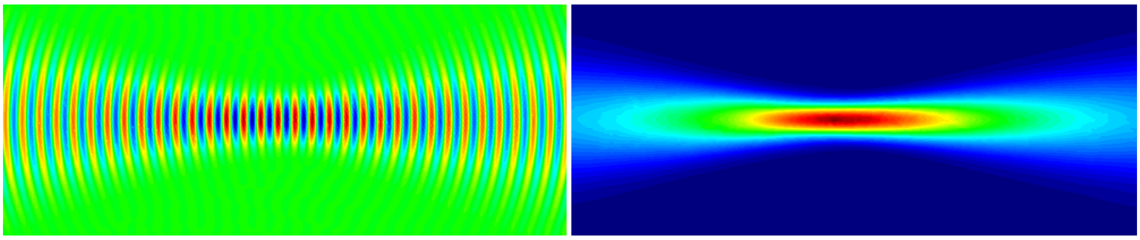

Figure 4.2: Electric field (left) and Poynting vector (right) of a Gaussian beam in vacuum [66].

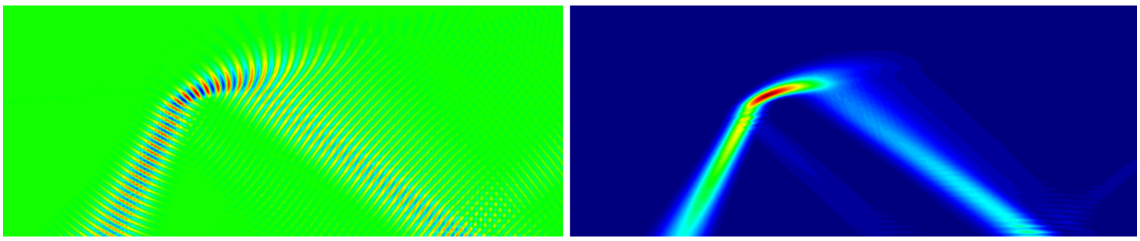

Figure 4.3: Electric field (left) and Poynting vector (right) of a 20 GHz Gaussian beam in cold plasma [66].

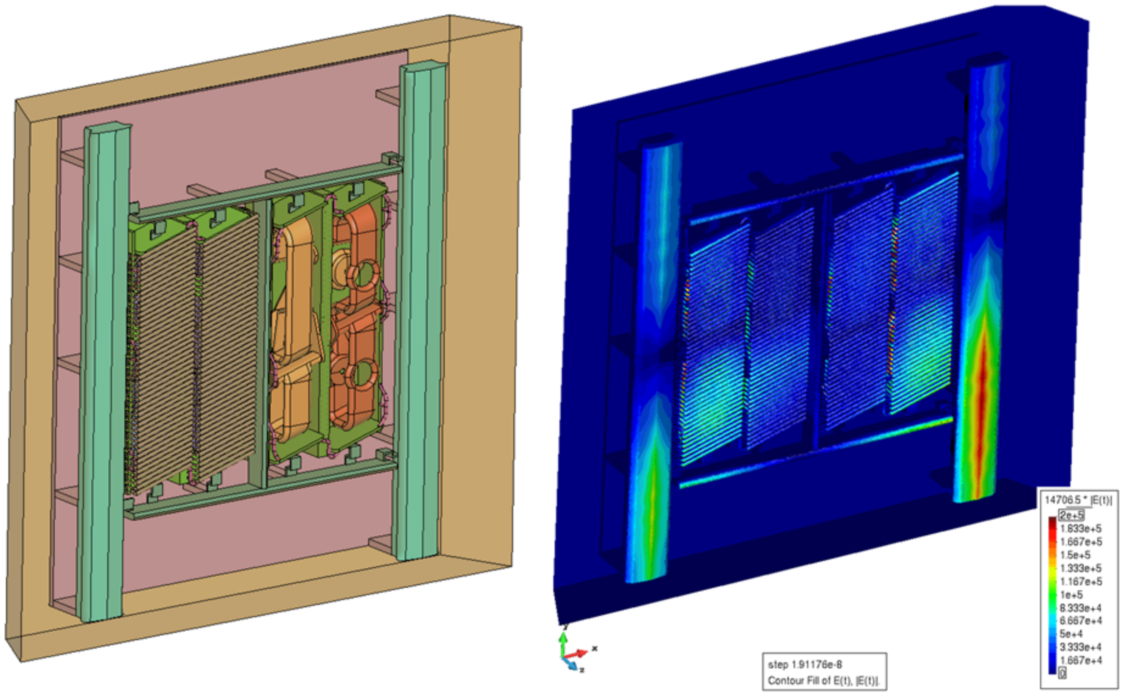

Figure 4.4: Electric field generated by the JET A2 antenna in cold plasma [59].

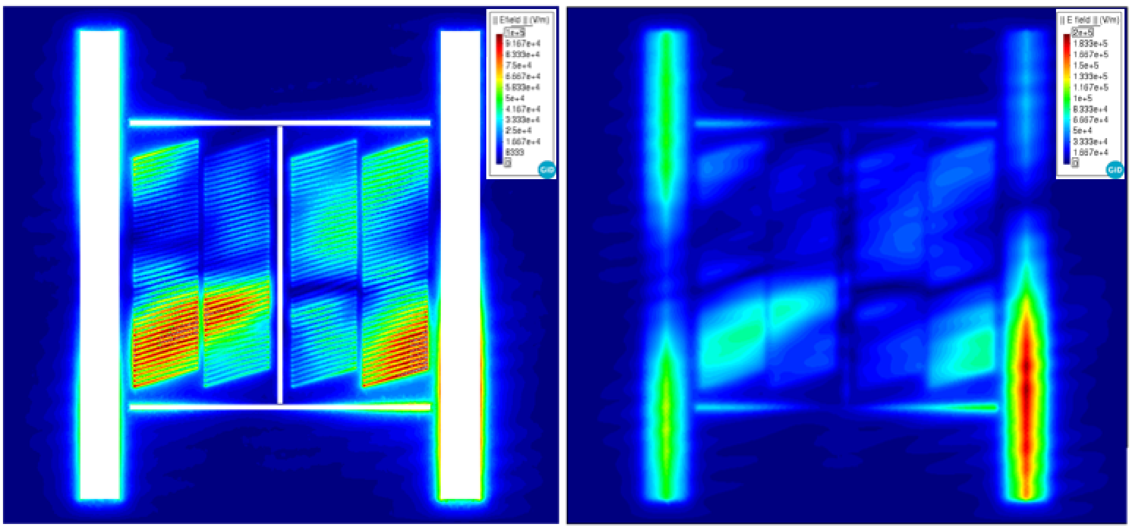

Figure 4.5: Cut planes of the electric field generated by the JET A2 antenna in cold plasma [59].

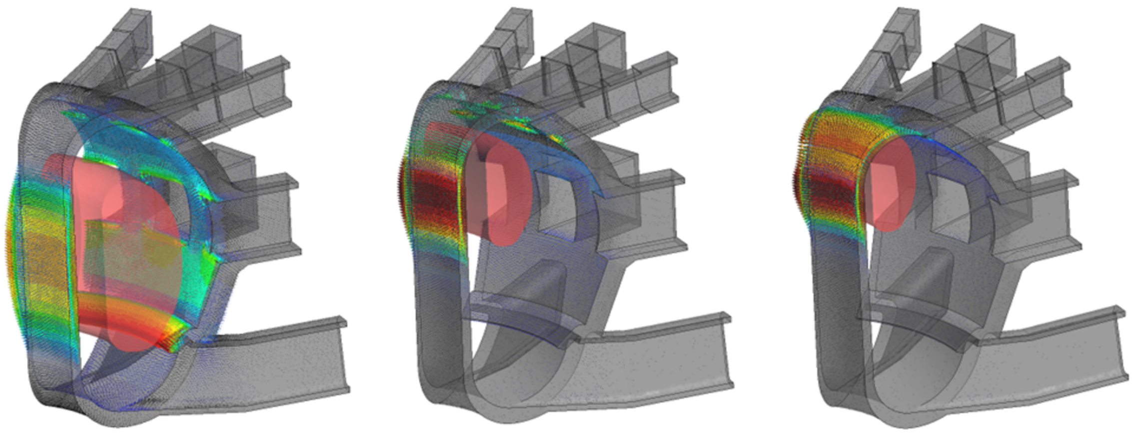

Figure 4.6: Eddy currents induced on ITER walls by plasma disruption [64].

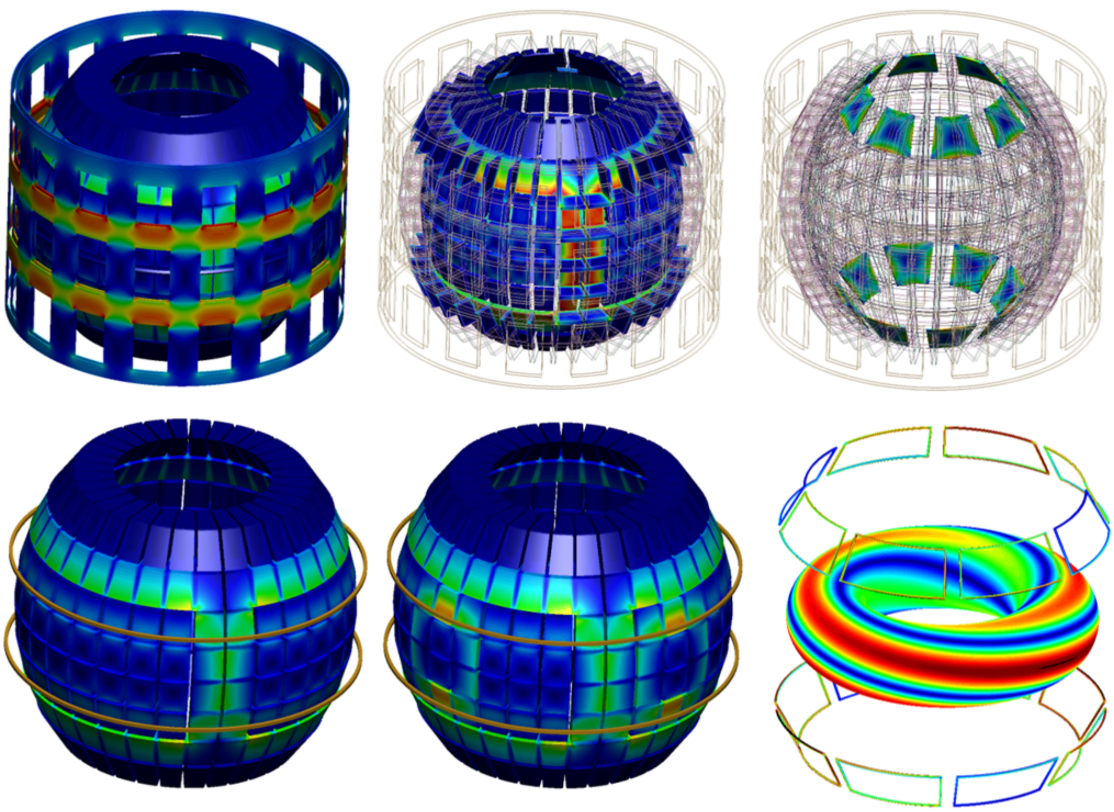

Figure 4.7: Eddy currents induced on STEP components by plasma and control coils [64].

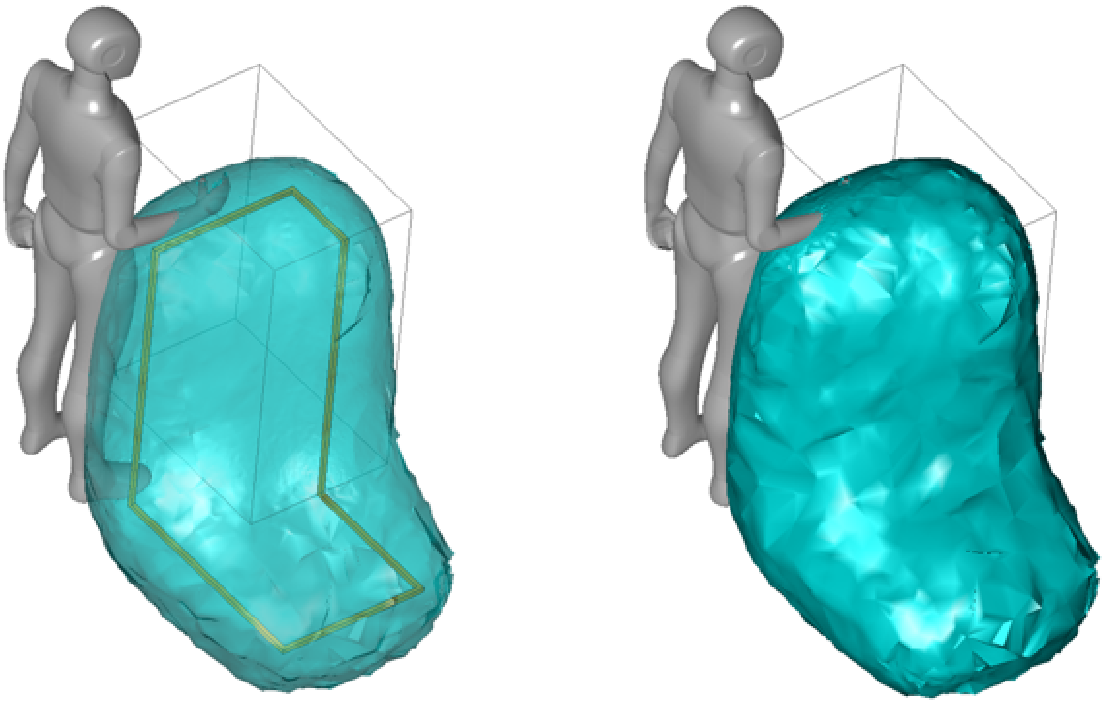

Figure 4.8: Iso-surfaces representing electromagnetic field exposure limits [52].

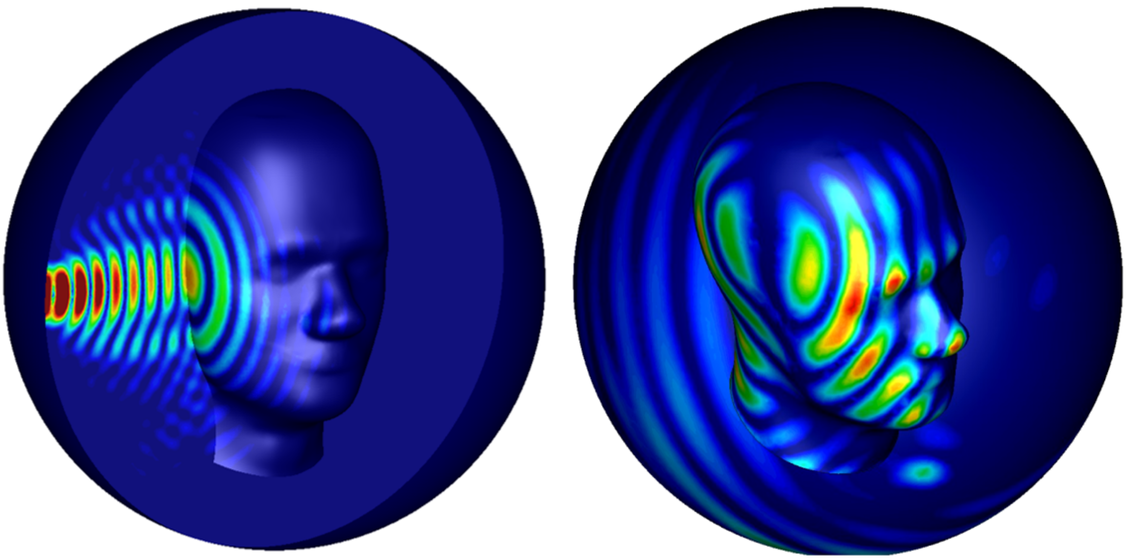

Figure 4.9: Gaussian beams on SAM head.

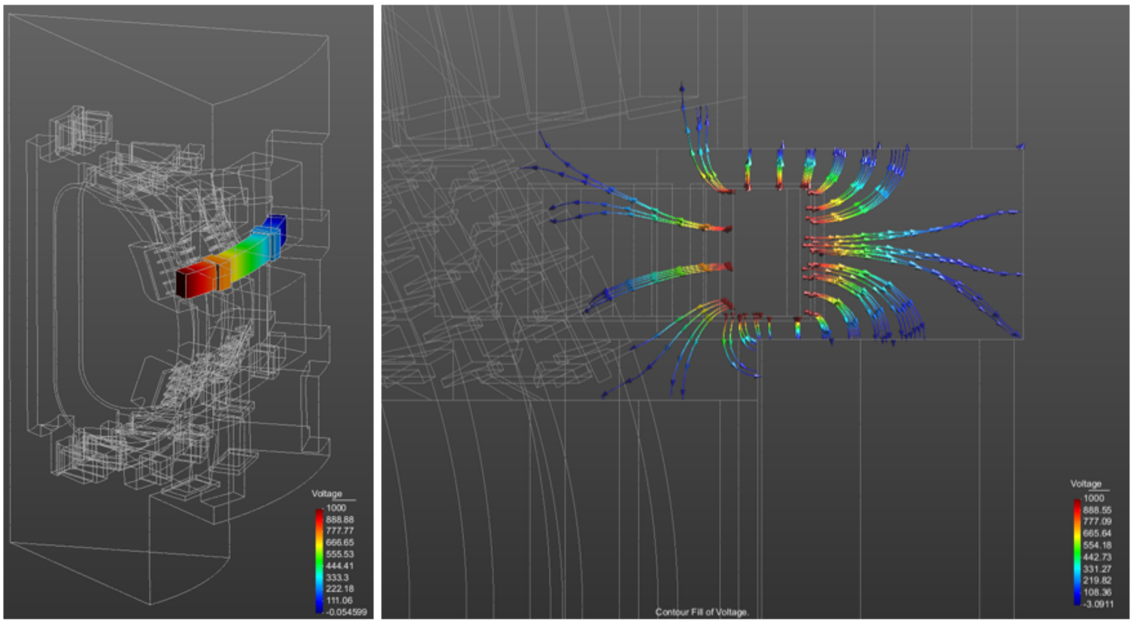

Figure 4.10: Voltage induced by a quench in a superconducting poloidal coil of ITER.

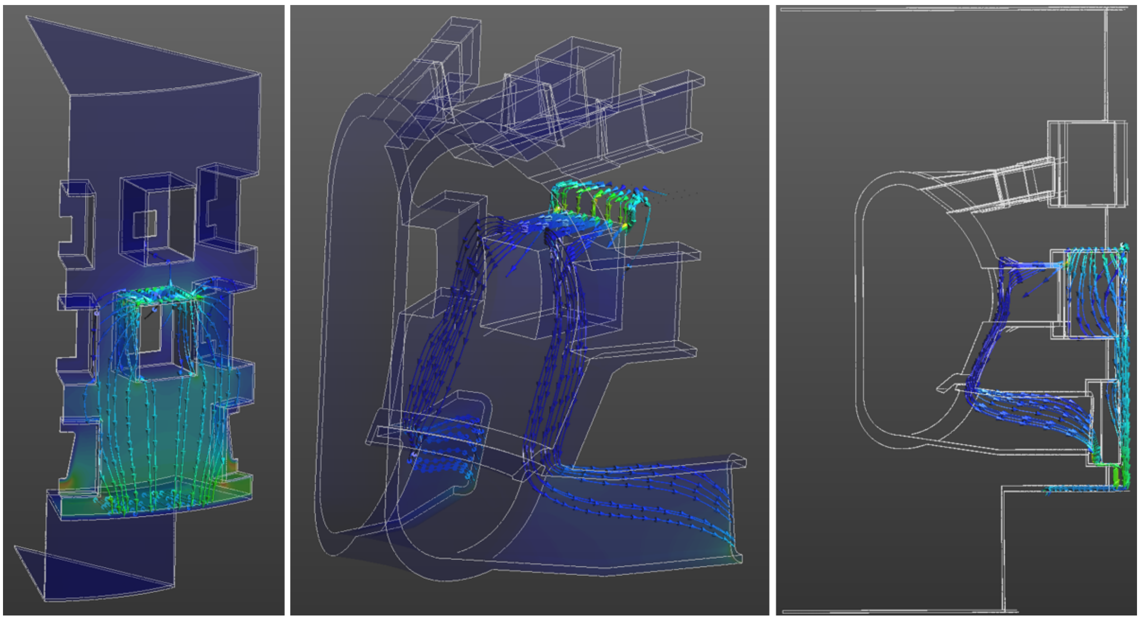

Figure 4.11: Electric currents following an electric arc strike on ITER components.

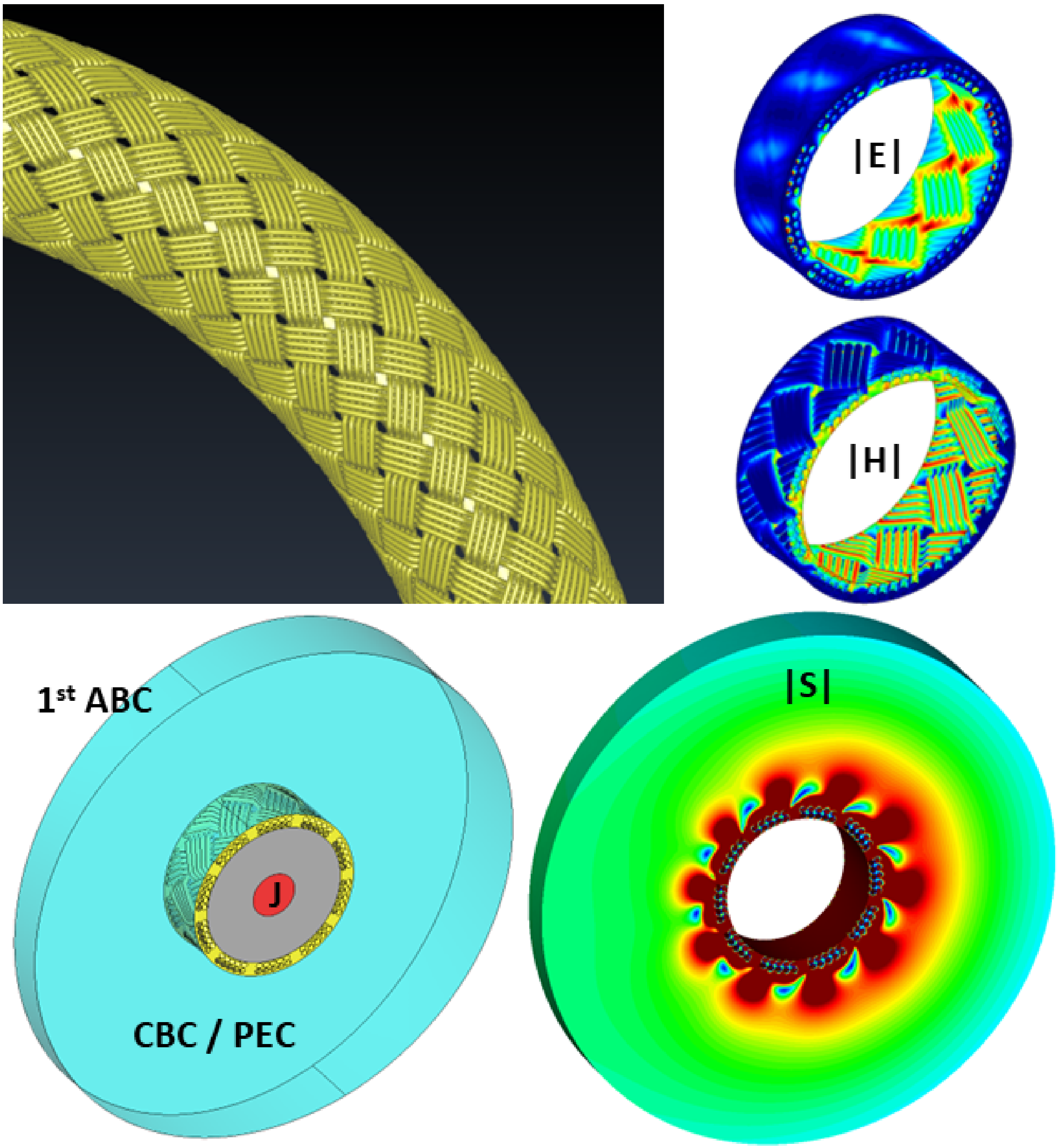

Figure 4.12: Electromagnetic radiation leaking from a curved coaxial cable braided shield.

Appendix A

Electromagnetic theory

This appendix briefly reviews the portions of the electromagnetic theory that are used in ERMES 20.0. For a more detailed information on the fundamentals of electromagnetic theory, look at the references [23, 16, 68, 32, 74, 76, 67]. For more advanced topics and specific applications in frequency domain, look at the references [3, 19, 9].

A.1 Maxwell's equations

In the second half of the 19th century James Clerk Maxwell established in [34, 35] a set of twenty equations with twenty unknowns that summarized all the electromagnetic phenomena known in those days. This set of equations was later reduced by Oliver Heaviside [21, 41], who simplified and transformed the original set with the help of the vector notation developed by himself concurrently with Josiah Willard Gibbs [79, 78]. The term *Maxwell's equations* is nowadays applied to the simplified version due to Heaviside:

$$\nabla \cdot \mathbf{D} = \rho, \tag{A.1}$$

$$\nabla \times \mathbf{H} - \frac{\partial \mathbf{D}}{\partial t} = \mathbf{J}, \tag{A.2}$$

$$\nabla \times \mathbf{E} + \frac{\partial \mathbf{B}}{\partial t} = 0, \tag{A.3}$$

$$\nabla \cdot \mathbf{B} = 0, \tag{A.4}$$

where the vector functions **E**, **D**, **B** and **H** are usually called (although there is no universal agreement and different names can be found in the literature):

E: electric field $(volt/meter)$,

D: electric displacement field $\left(coulomb/meter^2\right)$,

B: magnetic flux density (also magnetic induction) $(tesla)$,

H: magnetic field $(ampere/meter)$.

The sources that generate these electromagnetic fields are:

$\mathbf{J}$: electric current density $\left(ampere/meter^2\right)$,

ρ: electric charge density $\left(coulomb/meter^3\right)$,

which are related through the continuity equation

$$\nabla \cdot \mathbf{J} + \frac{\partial \rho}{\partial t} = 0. \tag{A.5}$$

The above relation express the law of conservation of charge. This law can be deduced from equations (A.2) and (A.1) using the fact that, for any vector function $\mathbf{F}$, it is satisfied:

$$\nabla \cdot (\nabla \times \mathbf{F}) = 0. \tag{A.6}$$

The symbols $\nabla\cdot$ and $\nabla\times$ correspond to the divergence and the curl, respectively. The divergence is defined by

$$\nabla \cdot \mathbf{F} \equiv \lim_{V \to 0} \frac{1}{V} \oint_S \mathbf{F} \cdot \hat{\mathbf{n}} \, dS, \tag{A.7}$$

where $\mathbf{F}$ is a vector function, V is an arbitrary volume, S is the surface surrounding that volume and, the integral is a surface integral with $\hat{\mathbf{n}}$ being the outward normal to the surface S. In Cartesian coordinates, with $\mathbf{F} = F_x \hat{\mathbf{x}} + F_y \hat{\mathbf{y}} + F_z \hat{\mathbf{z}}$, the divergence takes the form:

$$\nabla \cdot \mathbf{F} = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} + \frac{\partial F_z}{\partial z}. \tag{A.8}$$

The curl in the direction given by the unit vector $\hat{\mathbf{n}}$ is defined by

$$(\nabla \times \mathbf{F}) \cdot \hat{\mathbf{n}} \equiv \lim_{S \to 0} \frac{1}{S} \oint_C \mathbf{F} \cdot d\mathbf{r}, \tag{A.9}$$

where $\mathbf{F}$ is a vector function and the integral is a line integral along the boundary C of the area S. The area S is perpendicular to $\hat{\mathbf{n}}$. In Cartesian coordinates, with $\mathbf{F} = F_x \hat{\mathbf{x}} + F_y \hat{\mathbf{y}} + F_z \hat{\mathbf{z}}$, the curl takes the form:

$$\nabla \times \mathbf{F} = \left(\frac{\partial F_z}{\partial y} - \frac{\partial F_y}{\partial z} \right) \hat{\mathbf{x}} + \left(\frac{\partial F_x}{\partial z} - \frac{\partial F_z}{\partial x} \right) \hat{\mathbf{y}} + \left(\frac{\partial F_y}{\partial x} - \frac{\partial F_x}{\partial y} \right) \hat{\mathbf{z}}. \tag{A.10}$$

In principle, it is assumed that the Maxwell's equations are always valid, and that, if a vector function satisfies them, it is a possible electromagnetic field. It must be remembered that the Maxwell's equations are the mathematical expression of thoroughly validated experimental results, that is:

(A.1) summarizes Coulomb's law of forces plus the effects of matter,

(A.2) represents an extension of Ampere's circuital law,

(A.3) represents Faraday's law of induction,

(A.4) proclaims the non-existence of magnetic monopoles.

Under these circumstances, Maxwell's equations can not be demonstrated, but its applicability can be verified experimentally. Therefore, as a result of an extensive experimental work, it is considered that these set of equations can be applied to almost all macroscopic situations and that they can be used as guiding principles in the same way as the conservation of energy or the conservation of the linear momentum are used.

A.1.1 Lorentz force

To complete the description of the electromagnetic phenomena, an additional law must be added. This law is called the *Lorentz force law* and it relates electromagnetism with mechanics:

$$\mathbf{f} = \rho\,(\mathbf{E} + \mathbf{v} \times \mathbf{B}), \tag{A.11}$$

where **f** is the force density $\left(newton/meter^3\right)$, **v** is the velocity of the electric charge density $(meter/second)$ and $\times$ represents the cross product.

Using the continuity equation (A.5), it can be shown that an electric charge density ρ moving at a velocity **v** satisfies:

$$\mathbf{J} = \rho\,\mathbf{v}. \tag{A.12}$$

Therefore, the Lorentz force (A.11) can also be written as:

$$\mathbf{f} = \rho\,\mathbf{E} + \mathbf{J} \times \mathbf{B}, \tag{A.13}$$

and the power per unit volume p $\left(watt/meter^3\right)$ generated by this Lorentz force is

$$p = \mathbf{f} \cdot \mathbf{v} = \rho\,(\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \mathbf{v} = \mathbf{E} \cdot \mathbf{J}. \tag{A.14}$$

Lorentz force acting on a charged particle

The mathematical description of the electric charge density of a charged point particle is:

$$\rho(\mathbf{r}) = q\,\delta(\mathbf{r} - \mathbf{r}_0), \tag{A.15}$$

where q is the total electric charge of the particle, $\delta(\mathbf{r} - \mathbf{r}_0)$ is the Dirac's delta function, and $\mathbf{r}_0$ is the position of the particle. Using (A.15) and (A.11), the total force **F** $(newton)$ on a charged point particle can be written as:

$$\mathbf{F} = q\,(\mathbf{E} + \mathbf{v} \times \mathbf{B}), \tag{A.16}$$

A.1.2 Constitutive relations

Maxwell's equations cannot be solved until the relations **D** = **D**[**E**, **B**] and **H** = **H**[**E**, **B**] are known. These connections are called *constitutive relations* and can be expressed as:

$$\mathbf{D} = \epsilon_0 \mathbf{E} + \mathbf{P}, \tag{A.17}$$

$$\mathbf{H} = \frac{1}{\mu_0}\mathbf{B} - \mathbf{M}, \tag{A.18}$$

where

P: polarization $\left(coulomb/meter^2\right)$,

M: magnetization $(ampere/meter)$,

ϵ_0: vacuum permittivity $\left(\frac{1}{36\pi} \cdot 10^{-9} farad/meter\right)$,

μ_0: vacuum permeability $\left(4\pi \cdot 10^{-7} henry/meter\right)$.

The quantity **P** represents the macroscopically averaged electric dipole moment per unit volume of a material medium. The quantity **M** represents the macroscopically averaged magnetic dipole moment per unit volume of a material medium. Higher order multipole moments are negligible in most materials, therefore, the knowledge of **P** and **M** is usually enough to have a complete characterization of a macroscopic media from the standpoint of the electromagnetic theory. **P** and **M** are functions of the fields, position and time and can not be predicted by the macroscopic electromagnetic theory, but they are accepted as an external information. These relations can be very complex and its determination are left to the experiment or to be calculated theoretically from microscopic models.

To complete the description of matter we should add a third constitutive equation, **J** = **J**[**E**, **B**], which also has to be known experimentally, or theoretically, and represents the electric current density induced on the media.

Material types in electromagnetism

From the electromagnetic theory point of view, materials are classified depending on the form of its constitutive relations **D** = **D**[**E**, **B**], **H** = **H**[**E**, **B**] and **J** = **J**[**E**, **B**]. For instance, linear materials are those with a linear dependency between the fields, isotropic materials are those in which the directions of the field pairs **D**[**E**], **B**[**H**], and **J**[**E**] are aligned, homogeneous materials are those with constant properties over its volume, and so on.

In ERMES 20.0 there are implemented two types of materials: isotropic homogeneous linear materials and cold plasma, being the cold plasma a type of inhomogeneous gyrotropic linear material. In the cold plasma model implemented in ERMES, the field **D** is related with **E** through a position dependant 3x3 Hermitian tensor with off-diagonal complex conjugate terms. The cold plasma permittivity tensor is calculated from a given electron density distribution and a magnetic flux density following the equations provided in [75].

Linear isotropic materials

Inside a linear isotropic material (time-invariant and local in space) the constitutive relations have the general form [9]:

$$\mathbf{D}(\mathbf{r},t) = \int_{-\infty}^{\infty} \epsilon(\mathbf{r},t')\mathbf{E}(\mathbf{r},t-t')\,dt', \tag{A.19}$$

$$\mathbf{B}(\mathbf{r},t) = \int_{-\infty}^{\infty} \mu(\mathbf{r},t')\mathbf{H}(\mathbf{r},t-t')\,dt', \tag{A.20}$$

$$\mathbf{J}(\mathbf{r},t) = \int_{-\infty}^{\infty} \sigma(\mathbf{r},t')\mathbf{E}(\mathbf{r},t-t')\,dt', \tag{A.21}$$

where ϵ, μ and σ are scalars functions denominated

ϵ: electric permittivity $(farad/meter)$,

μ: magnetic permeability $(henry/meter)$,

σ: electric conductivity $(siemens/meter)$.

The assumed time-invariant property reveals a non-instantaneous response of the material to an applied field. This property allows to express the linear constitutive relation by means of a convolution in time. The assumed locality in space is a valid approximation when the spatial variation of the applied fields has a scale that is large compared with the dimensions involved in the creation of the atomic or molecular polarization. For visible light, or electromagnetic radiation of longer wavelength, it is often permissible to neglect the non-locality in space [23]. For conductors or some plasmas, however, the presence of free charges with macroscopic mean free paths makes this assumption to break down at much lower frequencies.

The constitutive relations (A.19)-(A.21) can be reformulated in the frequency domain by means of the Fourier transform, obtaining

$$\mathbf{D}(\mathbf{r},\omega) = \epsilon(\mathbf{r},\omega)\,\mathbf{E}(\mathbf{r},\omega), \tag{A.22}$$

$$\mathbf{B}(\mathbf{r},\omega) = \mu(\mathbf{r},\omega)\,\mathbf{H}(\mathbf{r},\omega), \tag{A.23}$$

$$\mathbf{J}(\mathbf{r},\omega) = \sigma(\mathbf{r},\omega)\,\mathbf{E}(\mathbf{r},\omega), \tag{A.24}$$

where $\mathbf{D}(\mathbf{r},\omega)$, $\mathbf{B}(\mathbf{r},\omega)$, $\mathbf{J}(\mathbf{r},\omega)$, $\mathbf{E}(\mathbf{r},\omega)$ and $\mathbf{H}(\mathbf{r},\omega)$ are the Fourier transform in time of $\mathbf{D}(\mathbf{r},t)$, $\mathbf{B}(\mathbf{r},t)$, $\mathbf{J}(\mathbf{r},t)$, $\mathbf{E}(\mathbf{r},t)$ and $\mathbf{H}(\mathbf{r},t)$, respectively. Also, $\epsilon(\mathbf{r},\omega)$, $\mu(\mathbf{r},\omega)$ and $\sigma(\mathbf{r},\omega)$ are the Fourier transform in time of $\epsilon(\mathbf{r},t)$, $\mu(\mathbf{r},t)$ and $\sigma(\mathbf{r},t)$, respectively. A linear isotropic material is completely characterized, from the standpoint of the macroscopic electromagnetic theory, when ϵ, μ and σ are known, either in time domain or in frequency domain. The Fourier transform definition used to transform (A.19)-(A.21) into (A.22)-(A.24) was:

$$F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t}\,dt. \tag{A.25}$$

A.2 Potentials

Using Helmholtz's theorem and the equations (A.4) and (A.3), it can be shown that there exist a scalar function ϕ, called the electric potential, and a vectorial function $\mathbf{A}$, called magnetic vector potential, such as

$$\mathbf{B} = \nabla \times \mathbf{A}, \tag{A.26}$$

$$\mathbf{E} = -\nabla\phi - \frac{\partial \mathbf{A}}{\partial t}. \tag{A.27}$$

By replacing the above equations into (A.1) and (A.2), and considering the general expressions for the constitutive relations (A.17) and (A.18), it is obtained:

$$\nabla^2\phi + \frac{\partial}{\partial t}\nabla\cdot\mathbf{A} = -\frac{1}{\epsilon_0}\left(\rho - \nabla\cdot\mathbf{P}\right), \tag{A.28}$$

$$\nabla^2\mathbf{A} - \mu_0\epsilon_0\frac{\partial^2\mathbf{A}}{\partial t^2} - \nabla\left(\nabla\cdot\mathbf{A} + \mu_0\epsilon_0\frac{\partial\phi}{\partial t}\right) = -\mu_0\left(\mathbf{J} + \nabla\times\mathbf{M} + \frac{\partial\mathbf{P}}{\partial t}\right), \tag{A.29}$$

where the operator ∇^2, when applied to a scalar function in a cartesian coordinate system, is defined by:

$$\nabla^2\phi = \frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} + \frac{\partial^2\phi}{\partial z^2} \tag{A.30}$$

and, when applied to a vectorial function in a cartesian coordinate system, is defined by:

$$\nabla^2\mathbf{A} = \nabla^2 A_x\,\hat{\mathbf{x}} + \nabla^2 A_y\,\hat{\mathbf{y}} + \nabla^2 A_z\,\hat{\mathbf{z}}. \tag{A.31}$$

Equations (A.28) and (A.29) plus the definitions (A.26) and (A.27) are totally equivalent to the set of equations (A.1)-(A.4). If (A.28) and (A.29) are solved then, the fields **E** and **B** obtained with (A.26) and (A.27) will automatically satisfy all the Maxwell's equations (A.1)-(A.4).

There are different possible **A** and ϕ potentials which give the same **E** and **B**. In fact, if χ is an arbitrary scalar function, any potential of the form

$$\mathbf{A}^+ = \mathbf{A} + \nabla\chi, \tag{A.32}$$

$$\phi^+ = \phi - \frac{\partial\chi}{\partial t}, \tag{A.33}$$

will give the same fields **E** and **B**. This happens because any scalar function χ fulfils:

$$\nabla\times(\nabla\chi) = 0, \tag{A.34}$$

where the operator ∇ represents a gradient which, in cartesian coordinates, is defined by:

$$\nabla\chi = \frac{\partial\chi}{\partial x}\,\hat{\mathbf{x}} + \frac{\partial\chi}{\partial y}\,\hat{\mathbf{y}} + \frac{\partial\chi}{\partial z}\,\hat{\mathbf{z}}. \tag{A.35}$$

To have a unique solution in (A.28) and (A.29), an extra condition over **A** and ϕ must be imposed. This extra condition is call the *gauge* and there are several options. The most used gauge conditions are the *Coulomb gauge*:

$$\nabla\cdot\mathbf{A} = 0, \tag{A.36}$$

and the *Lorenz gauge* [5, 43]:

$$\nabla\cdot\mathbf{A} + \mu_0\epsilon_0\frac{\partial\phi}{\partial t} = 0. \tag{A.37}$$

In some situations, thanks to the gauge freedom, a suitable choice of the $\mathbf{A} - \phi$ gauge can simplify noticeably equations (A.28)-(A.29) and make them easier to solve than the Maxwell's equations (A.1)-(A.4) themselves.

A.3 Time-harmonic Maxwell's equations

Time-harmonic fields are steady-state sinusoidally time varying electromagnetic fields. In ERMES 20.0 time-harmonic fields are represented in phasor notation as:

$$\mathbf{E}(\mathbf{r},t) = Real\left[\mathbf{E}_c(\mathbf{r})e^{-i\omega t}\right], \tag{A.38}$$

$$\mathbf{D}(\mathbf{r},t) = Real\left[\mathbf{D}_c(\mathbf{r})e^{-i\omega t}\right], \tag{A.39}$$

$$\mathbf{B}(\mathbf{r},t) = Real\left[\mathbf{B}_c(\mathbf{r})e^{-i\omega t}\right], \tag{A.40}$$

$$\mathbf{H}(\mathbf{r},t) = Real\left[\mathbf{H}_c(\mathbf{r})e^{-i\omega t}\right], \tag{A.41}$$

where $i = \sqrt{-1}$ is the imaginary unit, $e^{-i\omega t} = cos(\omega t) - i\,sin(\omega t)$ is the Euler's formula modified by a sign, ω is a positive real number, $Real[\,.\,]$ denotes the real part of the expression in brackets, and $\mathbf{E}_c(\mathbf{r})$, $\mathbf{D}_c(\mathbf{r})$, $\mathbf{B}_c(\mathbf{r})$ and $\mathbf{H}_c(\mathbf{r})$ are vector complex functions that only depend on position. It is also assumed a steady-state sinusoidal time dependance for the sources which, in phasor notation, are written as:

$$\mathbf{J}(\mathbf{r},t) = Real\left[\mathbf{J}_c(\mathbf{r})e^{-i\omega t}\right], \tag{A.42}$$

$$\rho(\mathbf{r},t) = Real\left[\rho_c(\mathbf{r})e^{-i\omega t}\right]. \tag{A.43}$$

If expressions (A.38)-(A.43) are substituted into equations (A.1)-(A.4), it is obtained the so-called *time-harmonic Maxwell's equations*:

$$\nabla \cdot \mathbf{D}_c(\mathbf{r}) = \rho_c(\mathbf{r}), \tag{A.44}$$

$$\nabla \times \mathbf{H}_c(\mathbf{r}) + i\omega\mathbf{D}_c(\mathbf{r}) = \mathbf{J}_c(\mathbf{r}), \tag{A.45}$$

$$\nabla \times \mathbf{E}_c(\mathbf{r}) - i\omega\mathbf{B}_c(\mathbf{r}) = 0, \tag{A.46}$$

$$\nabla \cdot \mathbf{B}_c(\mathbf{r}) = 0. \tag{A.47}$$

To reach these equations it was used the fact that, an arbitrary complex function $F_c = F_r + iF_i$, satisfies

$$Real\left[F_c e^{-i\omega t}\right] = 0 \quad \forall\, t \quad \Leftrightarrow \quad F_r = 0\,,\, F_i = 0 \tag{A.48}$$

and

$$\frac{\partial}{\partial t}\left(F_c e^{-i\omega t}\right) = -i\omega F_c e^{-i\omega t}. \tag{A.49}$$

The solutions of equations (A.44)-(A.47) are vector complex functions that depend exclusively on the position. The physical solution in time domain can be recovered using the expressions (A.38)-(A.41):

$$\mathbf{F}(\mathbf{r},t) = Real\left[(\mathbf{F}_r(\mathbf{r}) + i\,\mathbf{F}_i(\mathbf{r}))e^{-i\omega t}\right] = \mathbf{F}_r(\mathbf{r})\cos(\omega t) + \mathbf{F}_i(\mathbf{r})\sin(\omega t), \tag{A.50}$$

where $\mathbf{F}$ is any of the electromagnetic fields in (A.38)-(A.41). The time average of the dot and cross product of the quantities $\mathbf{F}(\mathbf{r},t) = Real\left[\mathbf{F}_c(\mathbf{r})e^{-i\omega t}\right]$ and $\mathbf{G}(\mathbf{r},t) = Real\left[\mathbf{G}_c(\mathbf{r})e^{-i\omega t}\right]$ can be calculated using:

$$\langle\, \mathbf{F}(\mathbf{r},t)\,\cdot\,\mathbf{G}(\mathbf{r},t)\,\rangle = \frac{1}{T}\int_0^T \mathbf{F}(\mathbf{r},t)\,\cdot\,\mathbf{G}(\mathbf{r},t)\;dt = \frac{1}{2}\;Real\left[\mathbf{F}_c(\mathbf{r})\,\cdot\,\bar{\mathbf{G}}_c(\mathbf{r})\right], \quad (A.51)$$

$$\langle\, \mathbf{F}(\mathbf{r},t)\times\mathbf{G}(\mathbf{r},t)\,\rangle = \frac{1}{T}\int_0^T \mathbf{F}(\mathbf{r},t)\times\mathbf{G}(\mathbf{r},t)\;dt = \frac{1}{2}\;Real\left[\mathbf{F}_c(\mathbf{r})\times\bar{\mathbf{G}}_c(\mathbf{r})\right], \quad (A.52)$$

being $\bar{\mathbf{G}}_c$ the complex conjugate of $\mathbf{G}_c$ and $T = 2\pi/\omega$ the period. For the specific case where $\mathbf{F}_c = \mathbf{G}_c$, expression (A.51) can be simplified to:

$$\langle\, \mathbf{F}(\mathbf{r},t)\cdot\mathbf{F}(\mathbf{r},t)\,\rangle = \left\langle\, |\,\mathbf{F}(\mathbf{r},t)\,|^2\,\right\rangle = \frac{1}{2}\;\mathbf{F}_c(\mathbf{r})\cdot\bar{\mathbf{F}}_c(\mathbf{r}) = \frac{1}{2}\;|\,\mathbf{F}_c(\mathbf{r})\,|^2. \quad (A.53)$$

A.3.1 Time-harmonic Maxwell's equations in linear isotropic media

Inside a linear isotropic material (time-invariant and local in space) the constitutive relations have the general form given in (A.19)-(A.21). Assuming the presence of time-harmonic electromagnetic fields as given in (A.38)-(A.41), the constitutive relations (A.19)-(A.21) can be written as:

$$\mathbf{D}_c\,(\mathbf{r}) = \epsilon_c\,(\mathbf{r})\;\mathbf{E}_c\,(\mathbf{r})\,, \quad (A.54)$$

$$\mathbf{B}_c\,(\mathbf{r}) = \mu_c\,(\mathbf{r})\;\mathbf{H}_c\,(\mathbf{r})\,, \quad (A.55)$$

$$\mathbf{J}_c\,(\mathbf{r}) = \sigma_c\,(\mathbf{r})\;\mathbf{E}_c\,(\mathbf{r})\,, \quad (A.56)$$

where $\mathbf{E}_c$, $\mathbf{D}_c$, $\mathbf{B}_c$ and $\mathbf{H}_c$ are the complex valued vectorial functions defined in (A.38)-(A.41) and ϵ_c, μ_c and σ_c are the complex conjugate of the Fourier transform of the functions $\epsilon(\mathbf{r},t)$, $\mu(\mathbf{r},t)$ and $\sigma(\mathbf{r},t)$ defined in (A.19)-(A.21). In other words, if $\epsilon(\mathbf{r},\omega)$, $\mu(\mathbf{r},\omega)$ and $\sigma(\mathbf{r},\omega)$ are the functions given in (A.22)-(A.24) and $\bar{\epsilon}(\mathbf{r},\omega)$, $\bar{\mu}(\mathbf{r},\omega)$ and $\bar{\sigma}(\mathbf{r},\omega)$ are its complex conjugates, then

$$\epsilon_c = \bar{\epsilon}(\mathbf{r},\omega), \quad (A.57)$$

$$\mu_c = \bar{\mu}(\mathbf{r},\omega), \quad (A.58)$$

$$\sigma_c = \bar{\sigma}(\mathbf{r},\omega), \quad (A.59)$$

where ω is the frequency at which the fields are oscillating. Since the response of materials to alternating fields is characterized by a complex value, we can separate its real and imaginary parts in the following way:

$$\epsilon_c = \epsilon^{'} + i\epsilon^{''}, \quad (A.60)$$

$$\mu_c = \mu^{'} + i\mu^{''}, \quad (A.61)$$

$$\sigma_c = \sigma^{'} + i\sigma^{''}. \quad (A.62)$$

The imaginary parts $\epsilon^{''}$ and $\mu^{''}$ and the real part $\sigma^{'}$ accounts for the dissipation (or loss) of energy within a medium. The real parts $\epsilon^{'}$ and $\mu^{'}$ and the imaginary part $\sigma^{''}$ are related to the stored energy within the medium.

The time-harmonic Maxwell's equations for linear isotropic media are obtained after substituting (A.54)-(A.56) into (A.44)-(A.47):

$$\nabla \cdot (\epsilon_c \mathbf{E}_c) = \rho_c, \tag{A.63}$$

$$\nabla \times \mathbf{H}_c + i\omega\epsilon_c \mathbf{E}_c = \mathbf{J}_c, \tag{A.64}$$

$$\nabla \times \mathbf{E}_c - i\omega\mu_c \mathbf{H}_c = 0, \tag{A.65}$$

$$\nabla \cdot (\mu_c \mathbf{H}_c) = 0, \tag{A.66}$$

where $\mathbf{E}_c$, $\mathbf{H}_c$, $\mathbf{J}_c$, ρ_c, ϵ_c and μ_c are complex valued functions that only depend on position. The term $\mathbf{J}_c$ in (A.64) includes all the *free* current densities which are present in the problem domain. Free current densities are all the currents densities except those produced by the polarization and magnetization of the material. The currents densities produced by the polarization and magnetization of the material are implicit in ϵ_c and μ_c. Free current densities are, for instance, the diffusion of charged particles, the induced eddy currents in conductive materials or, the impressed currents that are independent of the fields. Hence, we can write

$$\mathbf{J}_c = \sigma_c \mathbf{E}_c + \mathbf{J}_c^{imp}, \tag{A.67}$$

where $\mathbf{J}_c^{imp}$ represents all the free current densities except those induced by the electric field in materials with conductivity. Substituting (A.67) into (A.64) gives

$$\nabla \times \mathbf{H}_c + i\omega \left(\epsilon_c + i\,\frac{\sigma_c}{\omega}\right) \mathbf{E}_c = \mathbf{J}_c^{imp}. \tag{A.68}$$

Applying the divergence to (A.68) and recalling (A.6), equations (A.63)-(A.66) can be written as:

$$\nabla \cdot (\varepsilon_c \mathbf{E}_c) = \rho_c^{imp}, \tag{A.69}$$

$$\nabla \times \mathbf{H}_c + i\omega\varepsilon_c \mathbf{E}_c = \mathbf{J}_c^{imp}, \tag{A.70}$$

$$\nabla \times \mathbf{E}_c - i\omega\mu_c \mathbf{H}_c = 0, \tag{A.71}$$

$$\nabla \cdot (\mu_c \mathbf{H}_c) = 0, \tag{A.72}$$

where

$$\varepsilon_c = \epsilon_c + i\,\frac{\sigma_c}{\omega} = \epsilon^{'} - \frac{\sigma^{''}}{\omega} + i\left(\epsilon^{''} + \frac{\sigma^{'}}{\omega}\right) \tag{A.73}$$

and

$$\rho_c^{imp} = \frac{1}{i\omega}\nabla \cdot \mathbf{J}_c^{imp}. \tag{A.74}$$

In equations (A.69)-(A.72) the effect of the induced currents are included in ε_c. On the other hand, in the equivalent set of equations (A.63)-(A.66), the effect of the induced currents must be explicitly specify as shown in (A.67).

A.4 Fields at the interface between two media

When solving electromagnetic problems, it is usual to find materials with different electromagnetic properties in the domain under study. These properties are expected to change continuously (although abruptedly) in a narrow transition region between the two media. The assumption of a continuous change in the electromagnetic properties is made for consistency with the expected continuity of the physical fields and its derivatives.

In general, the specific details of what actually happens in the transition layer between two media are not of interest. Therefore, it is customary to replace the real situation by an idealized one, in which, the width of the transition layer tends to zero and becomes a surface. Using the Maxwell's equations, it can be demonstrated that, at the interface between two media, the fields must satisfy the following discontinuity conditions:

$$\hat{\mathbf{n}} \cdot (\mathbf{D}_1 - \mathbf{D}_2) = \rho_s, \tag{A.75}$$

$$\hat{\mathbf{n}} \times (\mathbf{H}_1 - \mathbf{H}_2) = \mathbf{J}_s, \tag{A.76}$$

$$\hat{\mathbf{n}} \times (\mathbf{E}_1 - \mathbf{E}_2) = 0, \tag{A.77}$$

$$\hat{\mathbf{n}} \cdot (\mathbf{B}_1 - \mathbf{B}_2) = 0, \tag{A.78}$$

where $\hat{\mathbf{n}}$ is the unit normal of the idealized transition layer surface *S* and it points from region 2 into region 1 (see Figure A.1). $\mathbf{J}_s$ and ρ_s represent, respectively, a surface current density and a surface electric charge density located at *S*. The fields $\mathbf{E}_1$, $\mathbf{D}_1$, $\mathbf{H}_1$ and $\mathbf{B}_1$ denotes the limiting values of the fields as *S* is approached from region 1. Analogously, $\mathbf{E}_2$, $\mathbf{D}_2$, $\mathbf{H}_2$ and $\mathbf{B}_2$ denotes the limiting values of the fields as *S* is approached from region 2. Expressions (A.75)-(A.78) are useful to connect solutions of the Maxwell's equations from different adjacent regions to obtain the fields in the entire domain of the problem.

Expressions (A.75)-(A.78) can be simplified with the help of (A.69)-(A.72) in the presence of linear isotropic materials and time-harmonic fields and sources:

$$\hat{\mathbf{n}} \cdot (\varepsilon_{c_1}\mathbf{E}_{c_1} - \varepsilon_{c_2}\mathbf{E}_{c_2}) = \rho_{cs}^{imp}, \tag{A.79}$$

$$\hat{\mathbf{n}} \times (\mathbf{H}_{c_1} - \mathbf{H}_{c_2}) = \mathbf{J}_{cs}^{imp}, \tag{A.80}$$

$$\hat{\mathbf{n}} \times (\mathbf{E}_{c_1} - \mathbf{E}_{c_2}) = 0, \tag{A.81}$$

$$\hat{\mathbf{n}} \cdot (\mu_{c_1}\mathbf{H}_{c_1} - \mu_{c_2}\mathbf{H}_{c_2}) = 0, \tag{A.82}$$

where ε_{c_1} and μ_{c_1} are the complex permittivity and permeability of the material 1 as it is defined in (A.73) and (A.55). Analogously, ε_{c_2} and μ_{c_2} are the complex permittivity and permeability of the material 2. The complex vector functions $\mathbf{E}_{c_1}$ and $\mathbf{H}_{c_1}$, which are defined in (A.38) and (A.41), denotes the limiting values of the fields as *S* is approached from region 1. Similarly, $\mathbf{E}_{c_2}$ and $\mathbf{H}_{c_2}$ denotes the limiting values of the fields as *S* is approached from region 2. $\mathbf{J}_{cs}^{imp}$ and ρ_{cs}^{imp}, which are defined in (A.67) and (A.74), represent, respectively, an imposed surface current density and an imposed surface electric charge density located at *S*.

In (A.79)-(A.82) it is not necessary to include explicitly the surface charge density and surface current density due to polarization, magnetization, and conductivity of the material. Their effects are implicit in the discontinuity of ε_c and μ_c. Only current densities or surface charge densities *imposed* on the discontinuity surface and not coming from polarization, magnetization, and conductivity, need to be included through $\mathbf{J}_{cs}^{imp}$ and ρ_{cs}^{imp}.

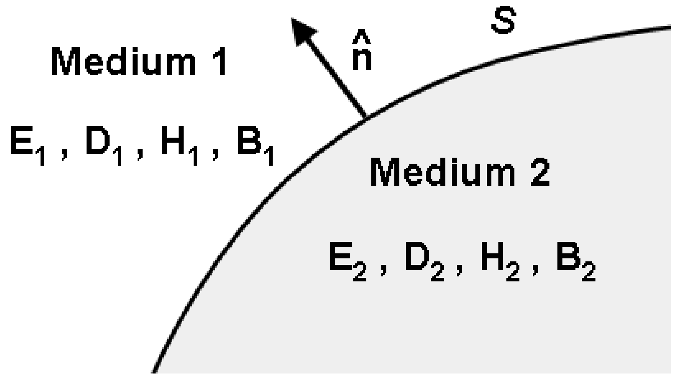

Figure A.1: Fields at the interface *S* between two media.

A.4.1 Perfect electric conductor

A *Perfect Electric Conductor* (PEC) is a material in which the electric conductivity is considered infinite. A remarkable feature of this idealized material is that, in its interior, any time-harmonic field is zero. This can be seen, heuristically, using equations (A.56) and (A.65) and assuming that, inside a PEC, the fields and the induced currents remain finite. From (A.56), if the conductivity is infinite, then $\mathbf{E}_c$ must vanish to have a finite $\mathbf{J}_c$. On the other hand, from (A.65), if $\mathbf{E}_c$ is zero, then $\mathbf{H}_c$ must also be zero.

For finite conductivity, linear isotropic materials, and time-harmonic regime, equations (A.79)-(A.82) can be deduced from (A.69)-(A.72). In the case of an infinite conductivity, equations (A.63)-(A.66) must be used instead of (A.79)-(A.82) to obtain the PEC interface conditions:

$$\hat{\mathbf{n}} \cdot \varepsilon_{c_1}\mathbf{E}_{c_1} = \rho_{cs}, \tag{A.83}$$

$$\hat{\mathbf{n}} \times \mathbf{H}_{c_1} = \mathbf{J}_{cs}, \tag{A.84}$$

$$\hat{\mathbf{n}} \times \mathbf{E}_{c_1} = 0, \tag{A.85}$$

$$\hat{\mathbf{n}} \cdot \mu_{c_1}\mathbf{H}_{c_1} = 0, \tag{A.86}$$

where the medium 2 of Figure A.1 is considered the PEC. $\mathbf{J}_{cs}$ and ρ_{cs} represent, respectively, a free surface current density and a free surface electric charge density imposed and confined in *S*.

An important characteristic behaviour of the fields on the surface of a PEC can be deduced from equations (A.85) and (A.86). Outside a PEC, the electric field is normal to the surface *S* and the magnetic field is tangential to surface *S*.

A.4.2 Imperfect electric conductor

If the material 2 in Figure A.1 is not a perfect conductor but it allows the fields to penetrate a small distance then, a time-harmonic electric field at *S* satisfies approximately the relation [70, 71]:

$$\frac{1}{\mu_{c_1}}\hat{\mathbf{n}} \times (\nabla \times \mathbf{E}_{c_1}) + \left(i\omega\sqrt{\frac{\varepsilon_{c_2}}{\mu_{c_2}}}\right)\hat{\mathbf{n}} \times (\hat{\mathbf{n}} \times \mathbf{E}_{c_1}) = 0. \tag{A.87}$$

Analogously, a time-harmonic magnetic field at *S* satisfies approximately the relation:

$$\frac{1}{\varepsilon_{c_1}}\hat{\mathbf{n}} \times (\nabla \times \mathbf{H}_{c_1}) + \left(i\omega\sqrt{\frac{\mu_{c_2}}{\varepsilon_{c_2}}}\right)\hat{\mathbf{n}} \times (\hat{\mathbf{n}} \times \mathbf{H}_{c_1}) = 0. \tag{A.88}$$

In the above expressions, the medium 2 of Figure A.1 is considered the imperfect conductor, $\mathbf{E}_{c_1}$ and $\mathbf{H}_{c_1}$ denotes the limiting values of the fields as *S* is approached from region 1, ε_{c_2} and μ_{c_2} are the complex permittivity and permeability of the imperfect conductor and, ε_{c_1} and μ_{c_1} are the complex permittivity and permeability of the material outside the imperfect conductor. The relations (A.87) and (A.88) are called *Impedance Boundary Conditions* (IBC).

A.5 Radiation condition

A boundary condition at infinite must be specified on problems with unbounded domain to ensure the uniqueness of the solution of the time-harmonic Maxwell's equations (A.69)-(A.72) [9]. Assuming all the sources and materials at a finite distance from the origin of coordinates and immersed in an infinite linear, isotropic, homogeneous, and lossless medium (characterize by the real parameters ϵ_r and μ_r), the time-harmonic electric and magnetic fields must satisfy:

$$\lim_{r\to\infty} r\left[\nabla \times \mathbf{E}_c - i\omega\sqrt{\epsilon_r\mu_r}\,(\hat{\mathbf{r}} \times \mathbf{E}_c)\right] = 0, \tag{A.89}$$

$$\lim_{r\to\infty} r\left[\nabla \times \mathbf{H}_c - i\omega\sqrt{\epsilon_r\mu_r}\,(\hat{\mathbf{r}} \times \mathbf{H}_c)\right] = 0, \tag{A.90}$$

where $r = |\mathbf{r}| = \sqrt{x^2 + y^2 + z^2}$ and $\hat{\mathbf{r}} = \mathbf{r}/|\mathbf{r}|$. Equations (A.89) and (A.90) are the well-known outgoing *Silver-Müller radiation boundary conditions* [39].

A.6 Wave equation

Applying the curl operator to (A.71) and using (A.70) to eliminate $\mathbf{H}_c$, the following equation is obtained:

$$\nabla \times \left(\frac{1}{\mu_c}\nabla \times \mathbf{E}_c\right) - \omega^2\varepsilon_c\mathbf{E}_c = i\omega\mathbf{J}_c^{imp}. \tag{A.91}$$

Applying now the curl operator to (A.70) and using (A.71) to eliminate $\mathbf{E}_c$, the following equation is obtained:

$$\nabla \times \left(\frac{1}{\varepsilon_c} \nabla \times \mathbf{H}_c \right) - \omega^2 \mu_c \mathbf{H}_c = \nabla \times \left(\frac{1}{\varepsilon_c} \mathbf{J}_c^{imp} \right). \tag{A.92}$$

These expressions are the *inhomogeneous electromagnetic vector wave equations* for time-harmonic fields in linear isotropic media. Equations (A.91) and (A.92) are very useful in numerical analysis because the fields are decoupled. The decoupling makes possible to calculate $\mathbf{E}_c$ solving only equation (A.91) and then find $\mathbf{H}_c$ with

$$\mathbf{H}_c = \frac{1}{i\omega\mu_c} \left(\nabla \times \mathbf{E}_c \right), \tag{A.93}$$

or, equivalently, solve only equation (A.92) and then find $\mathbf{E}_c$ with

$$\mathbf{E}_c = \frac{1}{i\omega\varepsilon_c} \left(\mathbf{J}_c^{imp} - \nabla \times \mathbf{H}_c \right). \tag{A.94}$$

A.7 Poynting theorem

Making the dot product of $\mathbf{E}$ with equation (A.2), the dot product of $\mathbf{H}$ with equation (A.3), subtracting both expressions, and using the identity

$$\nabla \cdot (\mathbf{F} \times \mathbf{G}) = \mathbf{G} \cdot \nabla \times \mathbf{F} - \mathbf{F} \cdot \nabla \times \mathbf{G}, \tag{A.95}$$

the following equation is obtained:

$$-\mathbf{E} \cdot \mathbf{J} = \left(\mathbf{E} \cdot \frac{\partial \mathbf{D}}{\partial t} + \mathbf{H} \cdot \frac{\partial \mathbf{B}}{\partial t} \right) + \nabla \cdot (\mathbf{E} \times \mathbf{H}). \tag{A.96}$$

Integrating (A.96) over a volume *V* delimited by a surface *S* and applying the divergence theorem to the last term in the right-hand side, it is obtained:

$$-\int_V \mathbf{E} \cdot \mathbf{J} \, dV = \int_V \left(\mathbf{E} \cdot \frac{\partial \mathbf{D}}{\partial t} + \mathbf{H} \cdot \frac{\partial \mathbf{B}}{\partial t} \right) dV + \oint_S (\mathbf{E} \times \mathbf{H}) \cdot d\mathbf{S}, \tag{A.97}$$

which is known as the *Poynting's theorem* [56]. This theorem states the conservation of energy for the electromagnetic fields. The left-hand side of equation (A.97) can be recognized as the power delivered against the Lorentz force (A.14) or, in other words, as the rate at which energy is supplied to the fields by the currents. The first term of the right-hand side accounts for the rate at which energy is stored in the electromagnetic fields and for the power dissipated within a material. This dissipation is produced by a time lag between the field applied to the medium and the resulting polarization or magnetization of its constitutive atoms or molecules. Finally, the last term of the right-hand side of (A.97) represents the power radiated out of the volume *V* through the surface *S*.

If the fields are time-harmonic, the time-average of expression (A.97) are usually of more interest than its point-wise time evolution. Using (A.51), (A.52), and (A.49), the time-average of (A.97) for time-harmonic fields can be expressed as:

$$
\begin{aligned}
-\frac{1}{2}\int_V Real\left[\mathbf{E}_c \cdot \bar{\mathbf{J}}_c\right]\, dV = &- \frac{\omega}{2}\int_V Imag\left[\mathbf{E}_c \cdot \bar{\mathbf{D}}_c + \mathbf{H}_c \cdot \bar{\mathbf{B}}_c\right]\, dV \\
&+ \frac{1}{2}\oint_S Real\,\left[\mathbf{E}_c \times \bar{\mathbf{H}}_c\right] \cdot d\mathbf{S},
\end{aligned}
\tag{A.98}
$$

where $Real[\,.\,]$ denotes the real part of the expression in brackets and $Imag[\,.\,]$ the imaginary part. If the materials present are linear and isotropic, definitions (A.53)-(A.56), (A.60)-(A.62), and (A.67), can be used to obtain

$$
\begin{aligned}
-\frac{1}{2}\int_V Real\left[\mathbf{E}_c \cdot \bar{\mathbf{J}}_c^{imp}\right]\, dV \;=\; &\frac{1}{2}\int_V \sigma^{'}|\,\mathbf{E}_c\,|^2\, dV \\
&+ \frac{\omega}{2}\int_V \left(\epsilon^{''}|\,\mathbf{E}_c\,|^2 + \mu^{''}|\,\mathbf{H}_c\,|^2\right) dV \\
&+ \frac{1}{2}\oint_S Real\,\left[\mathbf{E}_c \times \bar{\mathbf{H}}_c\right] \cdot d\mathbf{S},
\end{aligned}
\tag{A.99}
$$

where the left-hand side represents the time-average power delivered by the imposed sources, the first and second term of the right-hand side account for the time-average power dissipated as heat in the volume *V* due to conductivity, dielectric and magnetic losses, and the last term of the right-hand side represents the time-average power transmitted through the surface *S*.

A.8 Notation

From this point forward, the sub-index c indicating complex magnitude will be dropped. Unless otherwise specified, all magnitudes will be considered complex.

Appendix B

Finite element formulations

ERMES 20.0 solves numerically the time-harmonic Maxwell's equations with the Finite Element Method (FEM). This appendix presents the FEM approaches implemented in ERMES 20.0 along with its advantages and disadvantages. The FEM formulations implemented in ERMES 20.0 are:

- Double-curl Maxwell's equations with edge elements,
- Regularized Maxwell's equations with nodal elements,
- Local L^2 projection method with nodal and bubble elements.

All the above formulations include an $\mathbf{A} - \phi$ potentials version and switchable stabilization terms with Lagrange multipliers. All the possible combinations that can be used in ERMES 20.0 (formulation type, **E** field or $\mathbf{A} - \phi$ potentials, stabilization on/off) will be presented in the following. This appendix assumes a reader with knowledge on FEM. Therefore, the weak form of each formulation will be introduced without justification. For more details on FEM fundamentals, we recommend the references [24, 36, 69, 72, 26, 80].

B.1 Introduction

The typical approach when solving a general electromagnetic problem with FEM is to use edge elements with the double-curl Maxwell's equations. The edge elements proposed by Nédélec [42] seem to be the answer to most of the drawbacks exhibited by FEM when applied to electromagnetism [24, 69]. With edge elements, spurious-free solutions are obtained, boundary conditions are easier to implement and, the normal discontinuity and tangential continuity between different media are automatically satisfied. In addition, they present a better behaviour in non-convex domains than Lagrangian elements [77]. However, using the double-curl with edge elements formulation has also disadvantages [38, 37]. The most important one being the ill-conditioning of the generated matrices, which can even be singular in problems with a high number of unknowns [30]. The use of stabilization methods with Lagrange multipliers are reported to improve the conditioning of the matrix in [10, 22], or even be necessary to reach convergence [29, 31], but at the cost of increasing the number of unknowns. In our experience with ERMES, adding extra stabilization terms did not improve visibly the conditioning of the matrices nor the slow convergence of iterative solvers. The inclusion of Lagrange multipliers only showed to be useful at low frequencies, when they were practically mandatory for solving with direct methods. Therefore, although edge elements present several advantageous features, its FEM linear systems can be computationally expensive and difficult to solve. Because of this, it would be desirable to explore other FEM approaches that, maybe, will be able to provide better conditioned matrices.

An alternative approach to improve the FEM matrix conditioning problem is based on nodal elements and the regularized Maxwell's equations [20, 13]. The main advantages of this alternative formulation are that it provides spurious-free solutions with well-conditioned matrices and, moreover, only the three components of **E** are the unknowns; that is, there is no need of extra terms with Lagrange multipliers for stabilization purposes. Furthermore, its integral representation involves a less singular kernel (order 1 instead of 3), which makes the regularized FEM formulation best suited to hybridization with integral numerical techniques [20]. A study demonstrating the better computational performance of the nodal regularized formulation when compared with the edge elements double-curl approach was done in [50]. Unfortunately, new difficulties arise that were not present in the previous approach. The main drawback is that a non-physical solution can be obtained when the electromagnetic field has a singularity in the problem domain. Also, boundary conditions and field discontinuities are more laborious to implement. In this appendix is shown how ERMES 20.0 overcomes these difficulties.

The third FEM approach implemented in ERMES 20.0 is based on the Local L^2 Projection (LL2P) method and it uses nodal and bubble elements [17, 18]. The LL2P method main advantages are that it solves the singularity problem that affects the regularized formulation while still producing well-conditioned FEM matrices. Moreover, it does not need extra terms with Lagrange multipliers for stabilization purposes. Unfortunately, this approach is the most computationally demanding of the three presented here because of the bubble elements required for its stabilization. Anyway, the LL2P formulation can be useful on some scenarios where the other two methods can struggle to reach a solution.

B.2 Double-curl Maxwell's equations with edge elements

The generic problem to solve consists in finding the electric field **E** in a domain Ω with boundary $\partial\Omega$ produced by a current source **J** oscillating at an angular frequency ω. The equation that **E** satisfies in Ω is the so-called *double-curl Maxwell's equation* (see Section A.6):

$$\nabla \times \left(\frac{1}{\mu} \nabla \times \mathbf{E} \right) - \omega^2 \varepsilon \mathbf{E} = i\omega \mathbf{J}, \tag{B.1}$$

where μ and ε are, respectively, the complex magnetic permeability and the complex electric permittivity. If Ω is filled with Isotropic Homogeneous Linear (IHL) materials then, the definitions of μ and ε are given in Section A.3.1. If Ω contains a cold plasma then, ε is the cold plasma tensor derived in [75, 73].

Equation B.1 must be supplemented with information about the behaviour of the fields on the boundary $\partial\Omega$ to have a unique solution in Ω. On the surface of a Perfect Electric Conductor (PEC), the field $\mathbf{E}$ satisfies the following boundary condition (see Section A.4.1):

$$\hat{\mathbf{n}} \times \mathbf{E} = 0. \tag{B.2}$$

On the surface of a Perfect Magnetic Conductor (PMC), or symmetry surface, the field $\mathbf{E}$ satisfies:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} = 0. \tag{B.3}$$

In an open domain, the Silver-Müller radiation boundary condition must be added at infinity (see Section A.5):

$$\lim_{r\to\infty} \oint_{\partial\Omega_r} \| \, \hat{\mathbf{n}} \times \nabla \times \mathbf{E} \, + \, (i\omega\sqrt{\epsilon_0\mu_0}) \, \mathbf{E} \, \|^2 = 0. \tag{B.4}$$

In a waveguide port, the boundary conditions must account for the specific propagative modes of the waveguide. For instance, in a rectangular waveguide port, with only the fundamental mode TE_{10} propagating, the boundary condition can be expressed as:

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} = \gamma \, (\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) + \mathbf{U}, \tag{B.5}$$

where γ is the propagation constant of the fundamental mode, and

$$\begin{aligned} &\mathbf{U} = -2\,\gamma\,(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}_{10}), \\ &\mathbf{U} = 0 \end{aligned} \tag{B.6}$$

for the input and the output port, respectively [24]. The field $\mathbf{E}_{10}$ is the incident mode TE_{10} imposed at the input port. Boundary condition (B.5) can be adapted to represent the behaviour of $\mathbf{E}$ on the surface of an imperfect conductor (see Section A.4.2), or to approximate the Silver-Müller radiation boundary condition (B.4) with the so-called *first order Absorbing Boundary Condition* (1st ABC):

$$\hat{\mathbf{n}} \times \nabla \times \mathbf{E} = i\omega\sqrt{\epsilon_0\mu_0} \, (\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}). \tag{B.7}$$

The above set of equations is solved in ERMES 20.0 using an equivalent weak formulation, that is, if $L^2(\Omega)$ is the Hilbert space of all the square-integrable functions in Ω and $\mathbf{H}_0(\mathbf{curl};\Omega)$ is defined as

$$\mathbf{H}_0(\mathbf{curl};\Omega) := \{\, \mathbf{F} \in \mathbf{L}^2(\Omega) \;|\; \nabla \times \mathbf{F} \in \mathbf{L}^2(\Omega), \;\; \hat{\mathbf{n}} \times \mathbf{F} = 0 \;\text{ in PEC} \,\} \tag{B.8}$$

then, solving equation (B.1) is equivalent to find an $\mathbf{E} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ such that $\forall\, \mathbf{F} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ holds:

$$\int_\Omega \frac{1}{\mu}(\nabla \times \mathbf{E}) \cdot (\nabla \times \bar{\mathbf{F}}) \, - \, \omega^2 \int_\Omega \varepsilon \mathbf{E} \cdot \bar{\mathbf{F}} \, + \, \mathbf{B.C.}|_{\partial\Omega} \, = \, i\omega \int_\Omega \mathbf{J} \cdot \bar{\mathbf{F}}, \tag{B.9}$$

where the bar over the magnitudes denotes its complex conjugate and $\mathbf{B.C.}|_{\partial\Omega}$ is the term that accounts for the boundary conditions:

$$\mathbf{B.C.}|_{\partial\Omega} \;\; = \;\; \int_{\partial\Omega} \frac{1}{\mu} \, (\hat{\mathbf{n}} \times \nabla \times \mathbf{E}) \cdot \bar{\mathbf{F}}. \tag{B.10}$$

The weak formulation (B.9) discretized with edge elements is the classical way to apply FEM to electromagnetic field problems. In ERMES 20.0, the weak formulation B.9 is available with the first and second order edge elements given in references [24, 69]. The advantages and disadvantages of this approach were cited at the introduction of this appendix.

B.2.1 $\mathbf{A} - \phi$ potentials

ERMES 20.0 has available an $\mathbf{A} - \phi$ potentials version of the double-curl formulation. The $\mathbf{A} - \phi$ version can be useful at low frequencies where the $\mathbf{E}$ field version can be numerically unstable and difficult to solve. The $\mathbf{A} - \phi$ version uses the potentials defined in Section A.2 adapted to frequency domain as discussed in A.3:

$$\mathbf{B} = \nabla \times \mathbf{A}, \tag{B.11}$$

$$\mathbf{E} = i\omega \mathbf{A} - \nabla \phi. \tag{B.12}$$

The set of differentials equations that these time-harmonic potentials must satisfy is:

$$\begin{gathered} \nabla \times \left(\frac{1}{\mu} \nabla \times \mathbf{A} \right) - \omega^2 \varepsilon \left(\mathbf{A} + \nabla \phi' \right) = \mathbf{J}, \\ \nabla \left(\omega^2 \varepsilon \left(\mathbf{A} + \nabla \phi' \right) \right) = 0, \end{gathered} \tag{B.13}$$

where the scalar potential ϕ has been substituted by ϕ' to guarantee the symmetry of the FEM matrix, being ϕ' defined as:

$$\phi = -i\omega\phi'. \tag{B.14}$$

On the surface of a perfect electric conductor, $\mathbf{A}$ and ϕ' must satisfy an equivalent boundary condition to the PEC condition given in B.2:

$$\begin{gathered} \hat{\mathbf{n}} \times \mathbf{A} = 0, \\ \phi' = \phi'_0, \end{gathered} \tag{B.15}$$

where $\phi_0 = -i\omega\phi'_0$ is a voltage imposed on the PEC surface. Similarly, on a PMC symmetry surface, $\mathbf{A}$ and ϕ' must satisfy:

$$\begin{gathered} \hat{\mathbf{n}} \times \nabla \times \mathbf{A} = 0, \\ \hat{\mathbf{n}} \cdot \left(\varepsilon \mathbf{A} + \varepsilon \nabla \phi' \right) = 0. \end{gathered} \tag{B.16}$$

In a Robin boundary surface, where waveguide ports or radiation boundary conditions could be applied, the potentials must satisfy:

$$\begin{gathered} \hat{\mathbf{n}} \times \nabla \times \mathbf{A} - \gamma(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{A}) = \mathbf{U}', \\ \hat{\mathbf{n}} \cdot \left(\varepsilon \mathbf{A} + \varepsilon \nabla \phi' \right) = 0, \end{gathered} \tag{B.17}$$

where $\mathbf{U}' = \mathbf{U}/i\omega$, and γ and $\mathbf{U}$ are the same as those defined in B.5-B.6. The set of equations B.13 plus the boundary conditions B.15-B.17 are solved in ERMES 20.0 using the following weak formulation; if $H_0(grad; \Omega)$ is the functional space defined as:

$$H_0(grad; \Omega) := \{ G \in L^2(\Omega) \mid \nabla G \in L^2(\Omega), \ G = G_0 \ \text{ in PEC} \} \tag{B.18}$$

then, solving the system B.13-B.17 is equivalent to find an $\mathbf{A} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ and $\phi' \in H_0(grad;\Omega)$ such that $\forall\, \mathbf{F} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ and $\forall\, G \in H_0(grad;\Omega)$ holds:

$$\begin{gathered}\int_\Omega \frac{1}{\mu}(\nabla\times\mathbf{A})\cdot(\nabla\times\bar{\mathbf{F}}) - \omega^2\int_\Omega \varepsilon\left(\mathbf{A}+\nabla\phi'\right)\cdot\bar{\mathbf{F}} + \textbf{P.B.C.}|_{\partial\Omega} = \int_\Omega \mathbf{J}\cdot\bar{\mathbf{F}}, \\ -\omega^2\int_\Omega \varepsilon\left(\mathbf{A}+\nabla\phi'\right)\cdot\nabla\bar{G} + \omega^2\int_{\partial\Omega}\hat{\mathbf{n}}\cdot\left(\varepsilon\mathbf{A}+\varepsilon\nabla\phi'\right)\bar{G} = 0\end{gathered} \tag{B.19}$$

where $\textbf{P.B.C.}|_{\partial\Omega}$ accounts for the boundary conditions and it is equivalent to the term $\textbf{B.C.}|_{\partial\Omega}$ defined in B.10:

$$\textbf{P.B.C.}|_{\partial\Omega} = \int_{\partial\Omega}\frac{1}{\mu}\left(\hat{\mathbf{n}}\times\nabla\times\mathbf{A}\right)\cdot\bar{\mathbf{F}}. \tag{B.20}$$

In ERMES 20.0, the weak formulation B.19 has been implemented with first and second order edge elements for $\mathbf{A}$ and first order nodal elements for ϕ'.

B.2.2 Stabilization

Formulations B.9 and B.19 may require the presence of extra stabilization terms at low frequencies, when the solution is near the degenerate kernel of the double-curl operator. This extra stabilization can be achieved by controlling the divergence with Lagrange multipliers. ERMES 20.0 implements one of the several possible strategies proposed in [36]. Following this strategy, formulation B.9 can be re-expressed as; find $\mathbf{E} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ and $\varphi \in H_0(grad;\Omega)$ such as $\forall\, \mathbf{F} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ and $\forall\, \xi \in H_0(grad;\Omega)$ holds:

$$\begin{gathered}\int_\Omega \frac{1}{\mu}(\nabla\times\mathbf{E})\cdot(\nabla\times\bar{\mathbf{F}}) - \omega^2\int_\Omega \varepsilon\mathbf{E}\cdot\bar{\mathbf{F}} + \int_\Omega \varepsilon\mathbf{F}\cdot\nabla\varphi + \textbf{B.C.}|_{\partial\Omega} = i\omega\int_\Omega \mathbf{J}\cdot\bar{\mathbf{F}}, \\ \int_\Omega \varepsilon\mathbf{E}\cdot\nabla\xi - \int_\Omega \varepsilon\nabla\varphi\cdot\nabla\xi = 0.\end{gathered} \tag{B.21}$$

Similarly, the potential formulation B.19 is stabilized in ERMES 20.0 using the following weak form; find $\mathbf{A} \in \mathbf{H}_0(\mathbf{curl};\Omega)$, $\phi' \in H_0(grad;\Omega)$ and, $\varphi \in H_0(grad;\Omega)$ such that $\forall\, \mathbf{F} \in \mathbf{H}_0(\mathbf{curl};\Omega)$, $\forall\, G \in H_0(grad;\Omega)$ and, $\forall\, \xi \in H_0(grad;\Omega)$ holds:

$$\begin{gathered}\int_\Omega \frac{1}{\mu}(\nabla\times\mathbf{A})\cdot(\nabla\times\bar{\mathbf{F}}) - \omega^2\int_\Omega \varepsilon\left(\mathbf{A}+\nabla\phi'\right)\cdot\bar{\mathbf{F}} + \int_\Omega \mathbf{F}\cdot\nabla\varphi + \textbf{P.B.C.}|_{\partial\Omega} = \int_\Omega \mathbf{J}\cdot\bar{\mathbf{F}}, \\ -\omega^2\int_\Omega \varepsilon\left(\mathbf{A}+\nabla\phi'\right)\cdot\nabla\bar{G} + \omega^2\int_{\partial\Omega}\hat{\mathbf{n}}\cdot\left(\varepsilon\mathbf{A}+\varepsilon\nabla\phi'\right)\bar{G} = 0, \\ \int_\Omega \mathbf{A}\cdot\nabla\xi - \int_\Omega \nabla\varphi\cdot\nabla\xi = 0.\end{gathered}$$

In ERMES 20.0, the above stabilized formulations has been implemented with first and second order edge elements for $\mathbf{E}$ and $\mathbf{A}$, and first order nodal elements for ϕ' and φ.

B.3 Regularized Maxwell's equations with nodal elements

An equivalent approach for solving (B.1) is to use the so-called *regularized Maxwell's equation*:

$$\nabla \times \left(\frac{1}{\mu} \nabla \times \mathbf{E} \right) - \bar{\varepsilon}\, \nabla \left(\frac{1}{\tau \mu} \nabla \cdot (\varepsilon \mathbf{E}) \right) - \omega^2 \varepsilon \mathbf{E} = i\omega \mathbf{J}, \tag{B.22}$$

were ε represents a generic permittivity tensor and $\tau = tr(\bar{\varepsilon}\varepsilon)/3$. In the case of IHL materials, τ can be simplified to $\tau = \|\varepsilon\|^2$. Equation B.22 must also be supplemented with boundary conditions on $\partial\Omega$ to guarantee a unique solution in Ω. These boundary conditions need to be adapted to the new regularized approach. For instance, on the surface of a perfect electric conductor, the regularized version of the PEC boundary condition (B.2) is:

$$\begin{aligned} \nabla \cdot (\varepsilon \mathbf{E}) &= 0, \\ \hat{\mathbf{n}} \times \mathbf{E} &= 0. \end{aligned} \tag{B.23}$$

On the surface of a perfect magnetic conductor, the regularized version of the PMC boundary condition (B.3) is:

$$\begin{aligned} \hat{\mathbf{n}} \times \nabla \times \mathbf{E} &= 0, \\ \hat{\mathbf{n}} \cdot \mathbf{E} &= 0. \end{aligned} \tag{B.24}$$

On an infinite open domain, the regularized version of the Silver-Müller radiation boundary condition (B.4) is:

$$\begin{aligned} \lim_{r\to\infty} \oint_{\partial\Omega_r} \|\, \hat{\mathbf{n}} \times \nabla \times \mathbf{E} \,-\, i\omega\sqrt{\epsilon_0\mu_0}\, (\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E})\, \|^2 = 0, \\ \lim_{r\to\infty} \oint_{\partial\Omega_r} |\, \nabla \cdot \mathbf{E} \,-\, i\omega\sqrt{\epsilon_0\mu_0}\, (\hat{\mathbf{n}} \cdot \mathbf{E})\, |^2 = 0, \end{aligned} \tag{B.25}$$

and its regularized 1st ABC approximation:

$$\begin{aligned} \hat{\mathbf{n}} \times \nabla \times \mathbf{E} &= i\omega\sqrt{\epsilon_0\mu_0}\, (\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}), \\ \nabla \cdot \mathbf{E} &= i\omega\sqrt{\epsilon_0\mu_0}\, (\hat{\mathbf{n}} \cdot \mathbf{E}). \end{aligned} \tag{B.26}$$

On waveguide ports, boundary condition (B.5) must be adapted to the regularization as:

$$\begin{aligned} \hat{\mathbf{n}} \times \nabla \times \mathbf{E} &= \gamma\, (\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{E}) + \mathbf{U}, \\ \nabla \cdot \mathbf{E} &= \gamma\; (\hat{\mathbf{n}} \cdot \mathbf{E}). \end{aligned} \tag{B.27}$$

The above set of equations is solved in ERMES 20.0 using an equivalent regularized weak formulation; that is, if $\mathbf{H}_0(\mathbf{curl}, \mathrm{div}; \Omega)$ is the functional space defined as

$$\begin{aligned} \mathbf{H}_0(\mathbf{curl}, \mathrm{div}; \Omega) := \{\, \mathbf{F} \in \mathbf{L}^2(\Omega) \,|\, \nabla \times \mathbf{F} \in \mathbf{L}^2(\Omega),\ \nabla \cdot (\varepsilon \mathbf{F}) \in L^2(\Omega), \\ \hat{\mathbf{n}} \times \mathbf{F} = 0 \ \text{ in PEC},\ \hat{\mathbf{n}} \,\cdot\, \mathbf{F} = 0 \ \text{ in PMC} \,\}, \end{aligned} \tag{B.28}$$

solving (B.22) is equivalent to find an $\mathbf{E} \in \mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega)$ such that $\forall \mathbf{F} \in \mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega)$, the following holds:

$$\begin{aligned} \int_\Omega \frac{1}{\mu} (\nabla \times \mathbf{E}) \cdot \left(\nabla \times \bar{\mathbf{F}}\right) + \int_\Omega \frac{1}{\tau\mu} (\nabla \cdot (\varepsilon\mathbf{E})) \left(\nabla \cdot \left(\bar{\varepsilon}\bar{\mathbf{F}}\right)\right) \\ - \omega^2 \int_\Omega \varepsilon\mathbf{E} \cdot \bar{\mathbf{F}} + \mathbf{R.B.C.}|_{\partial\Omega} = i\omega \int_\Omega \mathbf{J} \cdot \bar{\mathbf{F}}, \end{aligned} \tag{B.29}$$

where $\mathbf{R.B.C.}|_{\partial\Omega}$ is the term, properly adapted to the regularization, that accounts for the boundary conditions:

$$\mathbf{R.B.C.}|_{\partial\Omega} = \int_{\partial\Omega} \frac{1}{\mu} (\hat{\mathbf{n}} \times \nabla \times \mathbf{E}) \cdot \bar{\mathbf{F}} - \int_{\partial\Omega} \frac{1}{\tau\mu} (\nabla \cdot (\varepsilon\mathbf{E})) \left(\hat{\mathbf{n}} \cdot \left(\bar{\varepsilon}\bar{\mathbf{F}}\right)\right). \tag{B.30}$$

The equivalence between the *classical* problem (B.1)-(B.10) and the *regularized* problem (B.22)-(B.30) is formally demonstrated in [20]. Although equation (B.29) looks like a penalized method, the regularized formulation has no undetermined constants and, with the help of the extra boundary condition terms, the problem is well-posed. It is worth emphasizing the importance of the extra boundary conditions: if they are omitted, the number of iterations needed for an iterative solver to converge can severely increase or even spurious solutions can appear. In ERMES 20.0 the regularized formulation is available for first and second order nodal (Lagrangian) elements.

As mentioned in the introduction of this appendix, the regularized approach has the main features sought in an efficient FEM formulation; that is, it calculates spurious-free solutions with $\mathbf{E}$ as the only unknown and, it produces well-conditioned matrices. Unfortunately, to avoid unphysical solutions, special care must be taken in the discontinuity surface between different media and in the presence of fields singularities. The following subsections explain how ERMES 20.0 solve these problems.

B.3.1 Field discontinuities

The regularized formulation requires nodal elements because of the properties imposed to its FEM bases in B.28-B.29. Nodal elements force normal and tangential continuity on its interfaces therefore, in the presence of a field discontinuity (see Section A.4), special techniques must be applied. ERMES 20.0 uses the method explained in [54] to deal with field discontinuities. This method consists of defining two different nodes (one on each side of the discontinuity) and, during the assembly procedure, relate the two nodes as follows:

$$\begin{pmatrix} E_x^+ \\ E_y^+ \\ E_z^+ \end{pmatrix} = \begin{pmatrix} n_x^2\xi + 1 & n_x^2 n_y^2 \xi & n_x^2 n_z^2 \xi \\ n_x^2 n_y^2 \xi & n_y^2 \xi + 1 & n_y^2 n_z^2 \xi \\ n_x^2 n_z^2 \xi & n_y^2 n_z^2 \xi & n_z^2 \xi + 1 \end{pmatrix} \begin{pmatrix} E_x^- \\ E_y^- \\ E_z^- \end{pmatrix} \tag{B.31}$$

being

$$\xi = \frac{\epsilon^- + i\left(\sigma^-/\omega\right)}{\epsilon^+ + i\left(\sigma^+/\omega\right)} - 1,$$

$\mathbf{n} = (n_x, n_y, n_z)$ is the unit normal at the surface, and superscripts "+" and "-" denote each side of the discontinuity surface. In this method, the node on side "+" is removed from the global linear system with the help of (B.31), and only the unknowns on the side "-" are solved. No extra unknowns are added, and the symmetry of the total FEM matrix is retained if the side "+" is removed using (B.31) and its transpose [7].

The double node technique explained above needs to be extended to consider the intersection of three or more discontinuity surfaces. For instance, in a situation like the one shown in Figure B.1, four different nodes are defined at the centre. One of the nodes plays the role of "-" in equation (B.31). In this case, the selected node is in material 1. Then, during the assembly procedure, equation (B.31) is applied to the pairs 2-1, 3-1 and 4-1, with unit normals $\mathbf{n}_{21}$, $\mathbf{n}_{31}$ and $\mathbf{n}_{41}$, respectively. Nodes 2, 3, and 4 are removed from the global linear system, and only the unknowns of node 1 are added to the system. In our experience, the best choice for the role of "-" is the node in the material with the smallest $|\epsilon + i\,(\sigma/\omega)\,|$. It was observed that this option always gives the lowest number of iterations when solving the global FEM matrix with iterative solvers. ERMES 20.0 applies automatically the procedure explained above after creating in GiD overlapped duplicate surfaces on the discontinuity and define a contact volume between them. To create a contact volume between two overleaped surfaces, click on the GiD upper menu: **Geometry** $\rightarrow$ **Create** $\rightarrow$ **Contact** $\rightarrow$ **Volume**, and select the surfaces.

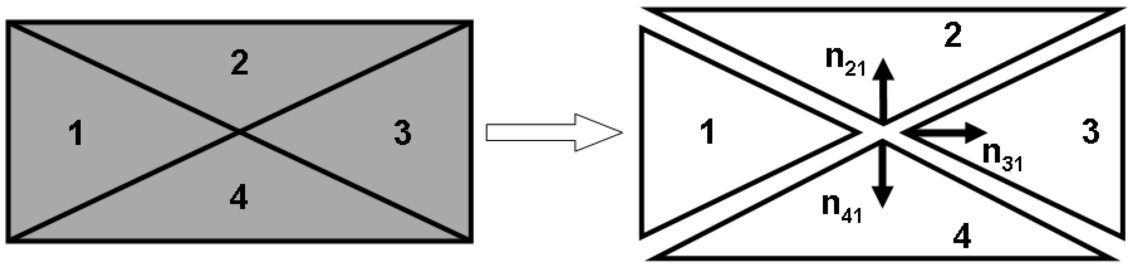

Figure B.1: Intersection of four different materials. The interface between different materials is considered as two separated surfaces related by equation (B.31). The vectors $\mathbf{n}_{21}$, $\mathbf{n}_{31}$ and $\mathbf{n}_{41}$ are the unit normal to the surfaces limiting the volume of materials 2, 3, and 4, respectively. In the intersection, equation (B.31) is applied to the pairs 2-1, 3-1, and 4-1.

B.3.2 Field singularities

In the reference [20] is shown that solving analytically (B.29) is completely equivalent to solve analytically (B.9). However, an unexpected problem appears when solving numerically (B.29) with nodal finite elements. As mentioned in B.1, a non-physical solution can be obtained when the electromagnetic field is singular on the problem domain. To understand the root causes of this singularity problem, assume that $\mathbf{V}_h$ is the vectorial space spanned by K nodal basis functions $N_i(\mathbf{r})$ as

$$\mathbf{V}_h := \left\{ \mathbf{u}_h \,|\, \mathbf{u}_h = \hat{\mathbf{x}} \sum_{i=1}^{K} c_{x_i} N_i(\mathbf{r}) + \hat{\mathbf{y}} \sum_{i=1}^{K} c_{y_i} N_i(\mathbf{r}) + \hat{\mathbf{z}} \sum_{i=1}^{K} c_{z_i} N_i(\mathbf{r})\,,\; c_i \in \mathbb{C} \right\}$$

with $\hat{\mathbf{n}} \times \mathbf{u}_h = 0$ on the PEC surfaces, and

$$\mathbf{H}_0^1(\Omega) := \left\{ \mathbf{F} \in \mathbf{L}^2(\Omega) \ \mid \ \frac{\partial \mathbf{F}}{\partial x}, \frac{\partial \mathbf{F}}{\partial y}, \frac{\partial \mathbf{F}}{\partial z} \in \mathbf{L}^2(\Omega)\,, \ \hat{\mathbf{n}} \times \mathbf{F} = 0 \ \text{ in PEC} \right\}.$$

It is clear that $\mathbf{V}_h$ is included in $\mathbf{H}_0^1(\Omega)$. But, for non-convex polyhedral domains, $\mathbf{H}_0^1(\Omega)$ is strictly included in $\mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega)$ and, moreover, $\mathbf{H}_0^1(\Omega)$ is closed in $\mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega)$ [11, 13]. As a consequence, if $\mathbf{E}$, the analytical solution of (B.29), belongs to $\mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega)$ but not to $\mathbf{H}_0^1(\Omega)$; then, it is impossible to approximate $\mathbf{E}$ using nodal elements. In fact, such an approximation is impossible using any $\mathbf{H}^1$-conforming finite element discretization [12]. This situation happens, for instance, when the electric field is singular at the corners or edges of a PEC. In other words, for non-convex polyhedral domains,

$$\mathbf{V}_h \subseteq \mathbf{H}_0^1(\Omega) \subset \mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega).$$

If the electric field is singular at some point of the domain, the analytical solution $\mathbf{E}$ belongs to $\mathbf{H}_0(\mathbf{curl}, \text{div}; \Omega)$ but not to $\mathbf{H}_0^1(\Omega)$. This means that $\mathbf{E}$ can not be approximated with nodal elements and (B.29), because any approximation of $\mathbf{E}$, regardless of the element size or the polynomial order used in the discretization, will belong to $\mathbf{H}_0^1(\Omega)$. In fact, what is approximated using (B.29) and nodal elements is the Garlekin projection of $\mathbf{E}$ onto $\mathbf{H}_0^1(\Omega)$, which is, in general, globally different to the physical solution $\mathbf{E}$. An example of this behaviour is shown in Figure B.2.

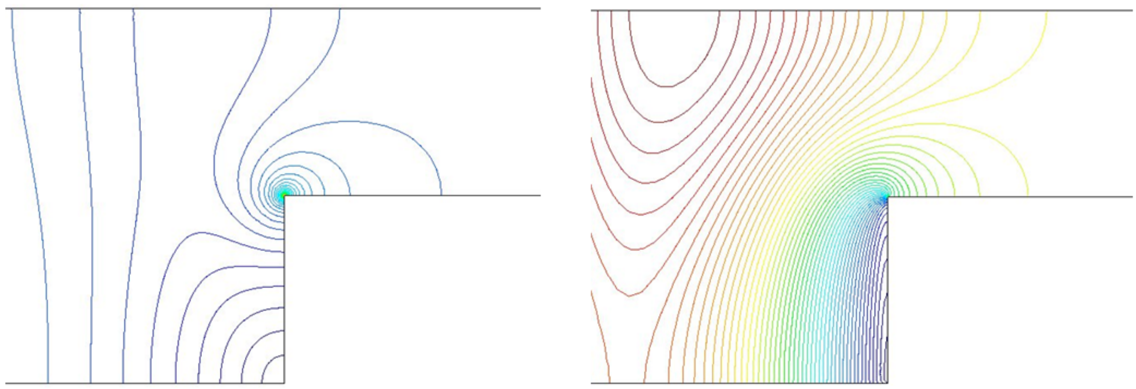

Figure B.2: Modulus of the electric field in a step discontinuity. Left: reference solution. Right: same problem solved with the regularized formulation.

A possible solution to the singularity problem is the *Weighted Regularized Maxwell's Equations* (WRME) method explained in [13]. There are other possibilities, such as [14, 2, 8], but the WRME method is more general and robust. In the WRME method, the divergence term of (B.29) is multiplied by a geometry-dependent weight. This weight tends to zero when approaching to a field singularity. To be more specific, the weight α is

$$\alpha := \left(\prod_{corners} r^{\gamma_c} \right) \cdot \left(\prod_{edges} \rho^{\gamma_e} \right), \tag{B.32}$$

where r and ρ are the distances from a point on the domain to the corner or edge where the field is singular. The coefficients γ_c and γ_e only depend on the geometry and they can be calculated theoretically beforehand [13]. The WRME method defines the space

$$\mathbf{X} := \left\{ \mathbf{u} \in \mathbf{H}_0(\mathbf{curl};\Omega) \mid \alpha(\nabla\cdot\mathbf{u}) \in L^2_{loc}(\Omega) \right\},$$

and solves the following weighted regularized weak form: find $\mathbf{E} \in \mathbf{X}$ such that $\forall\, \mathbf{F} \in \mathbf{X}$ holds

$$\begin{aligned}\int_\Omega \frac{1}{\mu}(\nabla\times\mathbf{E})\cdot\left(\nabla\times\bar{\mathbf{F}}\right) + \int_\Omega \frac{\alpha}{\tau\mu}(\nabla\cdot(\varepsilon\mathbf{E}))\left(\nabla\cdot\left(\bar{\varepsilon}\bar{\mathbf{F}}\right)\right) \\ -\ \omega^2\int_\Omega \varepsilon\mathbf{E}\cdot\bar{\mathbf{F}} \ +\ \textbf{W.R.B.C.}|_{\partial\Omega} \ =\ i\omega\int_\Omega \mathbf{J}\cdot\bar{\mathbf{F}},\end{aligned} \tag{B.33}$$

where $\textbf{W.R.B.C.}|_{\partial\Omega}$ is the term, properly adapted to the weighted regularization, that takes into account the boundary conditions:

$$\textbf{W.R.B.C.}|_{\partial\Omega} = \int_{\partial\Omega} \frac{1}{\mu}(\hat{\mathbf{n}}\times\nabla\times\mathbf{E})\cdot\bar{\mathbf{F}} - \int_{\partial\Omega}\frac{\alpha}{\tau\mu}(\nabla\cdot(\varepsilon\mathbf{E}))\left(\hat{\mathbf{n}}\cdot\left(\bar{\varepsilon}\bar{\mathbf{F}}\right)\right). \tag{B.34}$$

In [13], it is demonstrated that solving (B.33) is equivalent to solve the Maxwell's equations and also that $\mathbf{H}^1_0(\Omega)$ is dense in $\mathbf{X}$. These results imply that nodal elements converge to the right electromagnetic field solution even in the presence of singularities.

The method used in ERMES 20.0 to solve the singularity problem is a simplification of the WRME method. Instead of (B.32), ERMES 20.0 uses a weight which is equal to zero in the elements near a singularity and equal to one in the rest. The same idea was applied in [58] for eddy current problems with the $\mathbf{T}-\Omega$ formulation and, also, in [28] for magnetostatic problems with the potential **A** formulation. In our experience, to achieve the same level of accuracy, the simplified WRME method used in ERMES 20.0 always needed less mesh elements near the singularity and converged faster than the original WRME (B.32).

ERMES 20.0 control the radius of zero weight near the singularity with the so-called *Ungaged Layers* (ULs). Figure B.3 shows an example of a mesh with 3 ULs. When a problem is said to be solved with 1 UL, it means that only the elements with a node in contact with a singularity have a weight equal to zero. If a problem is said to be solved with 2 ULs, it means that the weight is set to zero in the elements with a node in contact with a singularity and also in the elements that have a node in contact with the elements of the first layer. An equivalent definition is applied for 3 ULs, 4 ULs, and so on. Only the elements inside the ULs have a weight $\alpha = 0$. For the rest, the weight $\alpha = 1$. The ULs must be applied in all the places of the domain where the field is singular. The field singularity locations can be known beforehand by just inspecting the geometry of the problem [6]. Therefore, ULs must be applied to:

1. Re-entrant corners and edges of PECs.

2. Corners and edges of dielectrics.

3. Intersection of several dielectrics.

Once these points are located, the number of ULs will depend on the size and order of the mesh elements. As an example, it usually requires 3 ULs to solve a singularity with second order elements. But, if very small elements are used around the singularity then, the number of ULs must be increased. On the other hand, if the element order is increased then, the number of ULs can be reduced. The number of ULs has an impact on the conditioning of the matrix, the more ULs are present, the worst is the conditioning. So the less ULs used, the better. More details about the ULs can be found in reference [45].

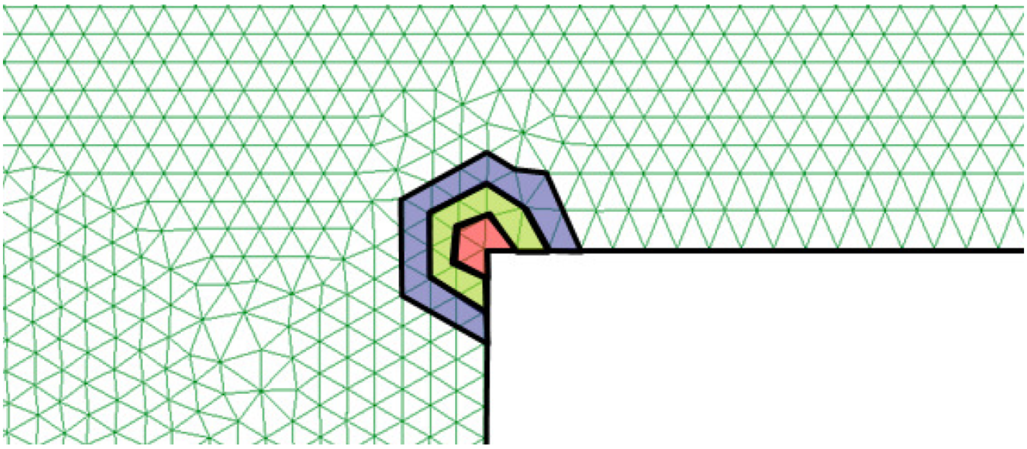

Figure B.3: Example of a mesh with 3 Ungaged Layers (ULs) around a waveguide edge. In (B.33), the coloured elements will have a weight $\alpha = 0$ and the rest of the elements will have a weight $\alpha = 1$.

B.3.3 $\mathbf{A} - \phi$ potentials

ERMES 20.0 has available an $\mathbf{A}-\phi$ potentials version of the regularized formulation. This version does not provide any additional stability to the nonetheless stable regularized formulation, as is the case with the double-curl formulation B.2.1. However, at low frequencies and in the absence of magnetic materials, it allows for solving problems without having to worry about singularities or discontinuities.

The regularized potential version is recommended for problems at low frequencies with non-magnetic materials. At higher frequencies, the singularity problem can re-appear when the skin depth approaches to zero. In the presence of magnetic materials, the normal component of $\mathbf{A}$ must be made discontinuous at the interfaces [4]. ERMES 20.0 solves the singularities and discontinuities of $\mathbf{A} - \phi$ with the same techniques as in B.3.1 and B.3.2 but adapted to the potentials. The set of differentials equations that the regularized potentials must satisfy is:

$$\begin{aligned} \nabla \times \left(\frac{1}{\mu}\nabla \times \mathbf{A}\right) - \nabla\left(\frac{1}{\mu}\nabla \cdot \mathbf{A}\right) - \omega^2\varepsilon\left(\mathbf{A} + \nabla\phi'\right) &= \mathbf{J}, \\ \nabla\left(\omega^2\varepsilon\left(\mathbf{A} + \nabla\phi'\right)\right) &= 0, \end{aligned} \tag{B.35}$$

where $\phi^{'}$ is defined in B.14. On the surface of a perfect electric conductor, the regularized potentials version of the PEC boundary condition is:

$$
\begin{aligned}
\hat{\mathbf{n}} \times \mathbf{A} &= 0, \\
\nabla \cdot \mathbf{A} &= 0, \\
\phi^{'} &= \phi_0^{'},
\end{aligned}
\tag{B.36}
$$

where $\phi_0^{'}$ is the applied voltage defined in equation B.15. Similarly, on a PMC symmetry surface, the regularized potentials $\mathbf{A}$ and $\phi^{'}$ must satisfy:

$$
\begin{aligned}
\hat{\mathbf{n}} \times \nabla \times \mathbf{A} &= 0, \\
\hat{\mathbf{n}} \cdot \mathbf{A} &= 0, \\
\hat{\mathbf{n}} \cdot \left(\varepsilon \mathbf{A} + \varepsilon \nabla \phi^{'}\right) &= 0.
\end{aligned}
\tag{B.37}
$$

In a Robin boundary surface, where waveguide ports or radiation conditions are applied, the regularized potentials satisfy:

$$
\begin{aligned}
\hat{\mathbf{n}} \times \nabla \times \mathbf{A} - \gamma(\hat{\mathbf{n}} \times \hat{\mathbf{n}} \times \mathbf{A}) &= \mathbf{U}^{'}, \\
\nabla \cdot \mathbf{A} - \gamma\left(\hat{\mathbf{n}} \cdot \mathbf{A}\right) &= 0, \\
\hat{\mathbf{n}} \cdot \left(\varepsilon \mathbf{A} + \varepsilon \nabla \phi^{'}\right) &= 0,
\end{aligned}
\tag{B.38}
$$

where $\mathbf{U}^{'}$ and γ are defined in B.17. The above set of equations and boundary conditions are solved in ERMES 20.0 with the following equivalent regularized weak formulation; find an $\mathbf{A} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ and $\phi^{'} \in H_0(grad;\Omega)$ such that $\forall\, \mathbf{F} \in \mathbf{H}_0(\mathbf{curl};\Omega)$ and $\forall\, G \in H_0(grad;\Omega)$ holds:

$$
\begin{aligned}
\int_\Omega \frac{1}{\mu}(\nabla \times \mathbf{A}) \cdot (\nabla \times \bar{\mathbf{F}}) + \int_\Omega \frac{1}{\mu}(\nabla \cdot \mathbf{A})(\nabla \cdot \bar{\mathbf{F}}) - \omega^2 \int_\Omega \varepsilon \left(\mathbf{A} + \nabla \phi^{'}\right) \cdot \bar{\mathbf{F}} & \\
+ \textbf{P.R.B.C.}|_{\partial\Omega} &= \int_\Omega \mathbf{J} \cdot \bar{\mathbf{F}}, \\
-\omega^2 \int_\Omega \varepsilon \left(\mathbf{A} + \nabla \phi^{'}\right) \cdot \nabla \bar{G} + \omega^2 \int_{\partial\Omega} \hat{\mathbf{n}} \cdot \left(\varepsilon \mathbf{A} + \varepsilon \nabla \phi^{'}\right) \bar{G} &= 0,
\end{aligned}
\tag{B.39}
$$

where **P.R.B.C.**$|_{\partial\Omega}$ accounts for the boundary conditions properly adapted to the regularization and the potentials:

$$
\textbf{P.R.B.C.}|_{\partial\Omega} = \int_{\partial\Omega} \frac{1}{\mu}\left(\hat{\mathbf{n}} \times \nabla \times \mathbf{A}\right) \cdot \bar{\mathbf{F}} - \int_{\partial\Omega} \frac{1}{\mu}(\nabla \cdot \mathbf{A})(\hat{\mathbf{n}} \cdot \bar{\mathbf{F}}). \tag{B.40}
$$

In ERMES 20.0, the weak formulation B.39 has been implemented with first and second order nodal elements for $\mathbf{A}$ and first order nodal elements for $\phi^{'}$.

B.4 Local L^2 projection method with nodal and bubble elements

The Local L^2 Projection (LL2P) method is a novel approach designed to address the issue with the singularities that affected the regularized formulation (see Section B.3.2). The LL2P method starts with the regularized weak form (B.29), using the same parameters and boundary conditions. It then applies local L^2 projections on the curl and div operators, expanding the unknown fields using first-order nodal basis functions, supplemented by one bubble function within each finite element and three bubble functions on each face of the element [17, 18]. Field discontinuities at material interfaces are handled using the procedure described in Section B.3.1; however, singularities require no special treatment. ERMES 20.0 solves the LL2P method with the following weak formulation; if $\mathbf{U}_h$ is the space of vectorial functions spanned by first-order nodal finite elements, augmented by one interior bubble within each element and three bubbles functions on each face of each element then, find an $\mathbf{E} \in \mathbf{U}_h$ such that $\forall\, \mathbf{F} \in \mathbf{U}_h$, the following holds:

$$\begin{aligned} \int_\Omega \frac{1}{\mu}\, \mathbf{L}\left(\nabla \times \mathbf{E}\right) \cdot \mathbf{L}\left(\nabla \times \bar{\mathbf{F}}\right) \;+\; \int_\Omega \frac{1}{\tau\mu}\, \hat{\mathbf{L}}\left(\nabla \cdot (\varepsilon \mathbf{E})\right) \hat{\mathbf{L}}\left(\nabla \cdot \left(\bar{\varepsilon} \bar{\mathbf{F}}\right)\right) & \\ -\; \omega^2 \int_\Omega \varepsilon \mathbf{E} \cdot \bar{\mathbf{F}} \;+\; \textbf{R.B.C.}|_{\partial\Omega} \;=\; i\omega \int_\Omega \mathbf{J} \cdot \bar{\mathbf{F}}, & \end{aligned} \tag{B.41}$$

where $\mathbf{L}$ and $\hat{\mathbf{L}}$ are local L^2 projector operators and the rest of the parameters and symbols are described in Section B.3. The projector operators $\mathbf{L}$ and $\hat{\mathbf{L}}$ are defined as; if $\mathbf{V}_h$ is the space of vectorial functions spanned by first order nodal elements and V_h is the space of scalar functions spanned by first order nodal elements then, for any $\mathbf{F} \in \mathbf{U}_h$ and $\forall\, \mathbf{Q} \in \mathbf{V}_h$ and $\forall\, Q \in V_h$ follows:

$$\begin{aligned} \int_\Omega \mathbf{L}\left(\nabla \times \mathbf{F}\right) \cdot \mathbf{Q} \;&=\; \int_\Omega \left(\nabla \times \mathbf{F}\right) \cdot \mathbf{Q} \;-\; \int_{PEC} \left(\hat{\mathbf{n}} \times \mathbf{F}\right) \cdot \mathbf{Q}, \\ \int_\Omega \hat{\mathbf{L}}\left(\nabla \cdot (\varepsilon \mathbf{F})\right) Q \;&=\; \int_\Omega \left(\nabla \cdot (\varepsilon \mathbf{F})\right) Q, \end{aligned} \tag{B.42}$$

Reference [18] explains in detail how to calculate the FEM matrices from the above expressions. The LL2P weak formulation (B.41) implemented in ERMES 20.0 is a refinement of several options proposed in [17, 18] and references therein. This version of the LL2P formulation was selected after extensive testing across a wide range of frequencies and problem types.

B.4.1 $\mathbf{A} - \phi$ potentials

ERMES 20.0 also features an $\mathbf{A} - \phi$ potentials version of the LL2P formulation. The advantage of this version is that it does not require any special treatment at material interfaces. Following a procedure similar to that of the LL2P $\mathbf{E}$-field version, the next weak form is obtained; find an

$\mathbf{A} \in \mathbf{U}_h$ and $\phi^{'} \in H_0(grad;\Omega)$ such that $\forall\, \mathbf{F} \in \mathbf{U}_h$ and $\forall\, G \in H_0(grad;\Omega)$ holds:

$$\begin{aligned}
\int_\Omega \frac{1}{\mu}\mathbf{L}\left(\nabla\times\mathbf{A}\right)\cdot\mathbf{L}\left(\nabla\times\bar{\mathbf{F}}\right) + \int_\Omega \frac{1}{\mu}\hat{\mathbf{L}}\left(\nabla\cdot\mathbf{A}\right)\hat{\mathbf{L}}\left(\nabla\cdot\bar{\mathbf{F}}\right) - \omega^2\int_\Omega \varepsilon\left(\mathbf{A}+\nabla\phi^{'}\right)\cdot\bar{\mathbf{F}}& \\
+\ \textbf{P.R.B.C.}|_{\partial\Omega} = \int_\Omega \mathbf{J}\cdot\bar{\mathbf{F}},& \quad \text{(B.43)}\\
-\omega^2\int_\Omega \varepsilon\left(\mathbf{A}+\nabla\phi^{'}\right)\cdot\nabla\bar{G} + \omega^2\int_{\partial\Omega}\hat{\mathbf{n}}\cdot\left(\varepsilon\mathbf{A}+\varepsilon\nabla\phi^{'}\right)\bar{G} = 0,&
\end{aligned}$$

where all the parameters and symbols have been defined in the previous sections. In ERMES 20.0, the weak formulation B.43 has been implemented with first-order nodal basis functions, plus one bubble function inside each finite element and three bubble functions on each face of each element for $\mathbf{A}$ and first order nodal elements for $\phi^{'}$.

B.4.2 Stabilization

Reference [17] adds a mesh dependant extra stabilization term to the weak forms B.41 and B.43:

$$\begin{aligned}
\mathbf{S_E} &= h_k^2\int_\Omega \left(\nabla\times\mathbf{E}\right)\cdot\left(\nabla\times\bar{\mathbf{F}}\right) + h_k^2\int_\Omega\left(\nabla\cdot\mathbf{E}\right)\left(\nabla\cdot\bar{\mathbf{F}}\right),\\
\mathbf{S_A} &= h_k^2\int_\Omega \left(\nabla\times\mathbf{A}\right)\cdot\left(\nabla\times\bar{\mathbf{F}}\right) + h_k^2\int_\Omega\left(\nabla\cdot\mathbf{A}\right)\left(\nabla\cdot\bar{\mathbf{F}}\right),
\end{aligned} \quad \text{(B.44)}$$

where h_k is the element radius. These terms are generally negligible compared to the other terms in B.41 and B.43. In our experience, they have proven useful only at low frequencies and with a fixed $h_k = 1.0$.

Bibliography

[1] Microsoft Visual Studio 2022. `https://visualstudio.microsoft.com`. Microsoft, 2024.

[2] F. Assous, P. Ciarlet Jr., S. Labrunie, and S. Lohrengel. The singular complement method. *The 13th International Conference on Domain Decomposition Methods, October 9-12, Lyon (France)*, 2000.

[3] C. A. Balanis. *Advanced Engineering Electromagnetics*. John Wiley & Sons, 1990.

[4] O. Bíró and K. Preis. On the use of the magnetic vector potential in the finite-element analysis of three-dimensional eddy currents. *IEEE Transactions on Magnetics*, 25:3145–3159, 1989.

[5] J. Van Bladel. Lorenz or Lorentz. *IEEE Antennas and Propagation Magazine*, 33:69, 1991.

[6] J. Van Bladel. *Singular Electromagnetic Fields and Sources*. IEEE Press, 1991.

[7] W. E. Boyse, D. R. Lynch, K. D. Paulsen, and G. N. Minerbo. Nodal-based finite-element modeling of Maxwell's equations. *IEEE Transactions on Antennas and Propagation*, 40:642–651, 1992.

[8] W. E. Boyse and A. A. Seidl. A hybrid finite element method for 3-D scattering using nodal and edge elements. *IEEE Transactions on Antennas and Propagation*, 42:1436–1442, 1994.

[9] M. Cessenat. *Mathematical Methods in Electromagnetism: Linear Theory and Applications*. World Scientific Publishing, 1996.

[10] P. Ciarlet. Augmented formulations for solving Maxwell equations. *Computer Methods in Applied Mechanics and Engineering*, 194:559–586, 2005.

[11] M. Costabel. A coercive bilinear form for Maxwell's equations. *Journal of Mathematical Analysis and Applications*, 157(2):527–541, 1991.

[12] M. Costabel and M. Dauge. Maxwell and Lamé eingevalues on polyhedra. *Mathematical Methods in Applied Science*, 22:243–258, 1999.

[13] M. Costabel and M. Dauge. Weighted regularization of Maxwell equations in polyhedral domains. *Numerische Mathematik*, 93(2):239–277, 2002.

[14] A. S. Bonnet-Ben Dhia, C. Hazard, and S. Lohrengel. A singular field method for the solution of Maxwell's equations in polyhedral domains. *SIAM Journal on Applied Mathematics*, 59(6):2028–2044, 1999.

[15] GiD: The personal pre- and post- processor. `https://www.gidsimulation.com`. International Center for Numerical Methods in Engineering (CIMNE), 2024.

[16] D. J. Griffiths. *Introduction to Electrodynamics*. Prentice-Hall, 3rd edition, 1999.

[17] H. Duan, P. Lin, and R.C.E Tan. Solving a Maxwell interface problem by a local L^2 projected C^0 finite element method. *Numerical Mathematics and Advanced Applications - ENUMATH 2013*, 103:795–802, 2015.

[18] H. Duan, R.C.E Tan, S.-Y. Yang, and C.-S. You. Computation of maxwell singular solution by nodal-continuous elements. *Journal of Computational Physics*, 268:63–83, 2014.

[19] R. F. Harrington. *Time-Harmonic Electromagnetic Fields*. McGraw-Hill, 1961.

[20] C. Hazard and M. Lenoir. On the solution of the time-harmonic scattering problems for Maxwell's equations. *SIAM Journal on Mathematical Analysis*, 27:1597–1630, 1996.

[21] O. Heaviside. On the forces, stresses and fluxes of energy in the electromagnetic field. *Philosophical Transactions of the Royal Society*, 183A:423–480, 1892.

[22] R. Hiptmair. Finite elements in computational electromagnetism. *Acta Numerica*, 1:237–339, 2002.

[23] J. D. Jackson. *Classical Electrodynamics*. John Wiley & Sons, Inc., 3rd edition, 1999.

[24] J. Jin. *The Finite Element Method in Electromagnetics*. John Wiley & Sons, 2nd edition, 2002.

[25] J. Jin and D. J. Riley. *Finite Element Analysis of Antennas and Arrays*. John Wiley & Sons, 2009.

[26] C. Johnson. *Numerical Solution of Partial Differential Equations by the Finite Element Method*. Cambridge University Press, 1987.

[27] K. Fujiwara and T. Nakata. Results for benchmark problem 7 (asymmetrical conductor with a hole). *COMPEL-The International Journal for Computation and Mathematics in Electrical and Electronic Engineering*, 9(3):137–154, 1990.

[28] M. Kaltenbacher and S. Reitzinger. Appropiate finite-element formulation for 3-D electromagnetic-field problems. *IEEE Transactions on Magnetics*, 38:513–516, 2002.

[29] F. Kikuchi. Mixed and penalty formulations for finite element analysis of an eigenvalue problem in electromagnetisim. *Computer Methods in Applied Mechanics and Engineering*, 64:509–521, 1987.

[30] I. E. Lager. *Finite Element Modelling of Static and Stationary Electric and Magnetic Fields*. PhD thesis, Delft Univesity, 1996.

[31] P. D. Ledger and K. Morgan. The application of the hp-finite element method to electromagnetic problems. *Archives of Computational Methods in Engineering*, 12(3):235–302, 2005.

[32] P. Lorrain, D. R. Corson, and F. Lorrain. *Electromagnetic fields and waves*. W.H. Freeman and Co., 3rd edition, 1988.

[33] M. Lennholm et al. Plasma control for the STEP prototype power plant. *Nuclear Fusion*, 64(9):096036, 2024.

[34] J. C. Maxwell. A dynamical theory of the electromagnetic field. *Royal Society Transactions*, 155:459–512, 1865.

[35] J. C. Maxwell. *A Treatise on Electricity and Magnetism*. Oxford Clarendon Press, 1873.

[36] P. Monk. *Finite Element Methods for Maxwell's Equations*. Oxford University Press, 2003.

[37] G. Mur. Edge elements, their advantages and their disadvantages. *IEEE Transactions on Magnetics*, 30:3552–3557, 1994.

[38] G. Mur. The fallacy of edge elements. *IEEE Transactions on Magnetics*, 34:3244–3247, 1998.

[39] C. Müller. *Foundations of the Mathematical Theory of Electromagnetic Waves*. Springer-Verlag, 1969.

[40] N. Petroulakis et al. NextGEM: Next-Generation Integrated Sensing and Analytical System for Monitoring and Assessing Radiofrequency Electromagnetic Field Exposure and Health. *International Journal of Environmental Research and Public Health*, 20(12):6085, 2023.

[41] P. J. Nahin. *Oliver Heaviside: The Life, Work, and Times of an Electrical Genius of the Victorian Age*. Johns Hopkins University Press, 2002.

[42] J. C. Nedelec. Mixed finite elements in R^3. *Numerische Mathematik*, 35:315–341, 1980.

[43] R. Nevels and C.-S. Shin. Lorenz, Lorentz, and the gauge. *IEEE Antennas and Propagation Magazine*, 43:70–71, 2001.

[44] NumPy. `https://numpy.org`. NumPy developers, 2024.

[45] R. Otin. Regularized Maxwell equations and nodal finite elements for electromagnetic field computations. *Electromagnetics*, 30(1-2):190–204, 2010.

[46] R. Otin. Numerical study of the thermal effects induced by a RFID antenna in vials of blood plasma. *Progress In Electromagnetics Research Letters*, 22:129–138, 2011.

[47] R. Otin. ERMES: A nodal-based finite element code for electromagnetic simulations in frequency domain. *Computer Physics Communications*, 184(11):2588–2595, 2013.

[48] R. Otin. A numerical model for the search of the optimum frequency in electromagnetic metal forming. *International Journal of Solids and Structures*, 50(10):1605–1612, 2013.

[49] R. Otin. `https://ruben-otin.blogspot.com`. Personal website, 2024.

[50] R. Otin, L. E. Garcia-Castillo, I. Martinez-Fernandez, and D. Garcia-Doñoro. Computational performance of a weighted regularized Maxwell equation finite element formulation. *Progress In Electromagnetics Research*, 136:61–77, 2013.

[51] R. Otin and H. Gromat. Specific absorption rate computations with a nodal-based finite element formulation. *Progress In Electromagnetics Research*, 128:399–418, 2012.

[52] R. Otin and R. Salmon. Electromagnetic field exposure assessment of TIG-AC and CD stud welding in accordance with the EU Directive 2013/35/EU. *The 70th International Institute of Welding Annual Assembly and International Conference, Shanghai (P. R. China), June 25-30*, 2017.

[53] R. Otin, J. Verpoorte, and H. Schippers. A finite element model for the computation of the transfer impedance of cable shields. *IEEE Transactions On Electromagnetic Compatibility*, 53(4):950–958, 2011.

[54] K. D. Paulsen, D. R. Lynch, and J. W. Strohbehn. Three-dimensional finite, boundary, and hybrid element solutions of the Maxwell equations for lossy dielectric media. *IEEE Transactions on Microwave Theory and Techniques*, 36:682–693, 1988.

[55] PETSc. `https://petsc.org`. Argonne National Laboratory, 2024.

[56] J. H. Poynting. On the transfer of energy in the electromagnetic field. *Philosophical Transactions of the Royal Society of London*, 175:343–361, 1884.

[57] D. M. Pozar. *Microwave Engineering*. John Wiley & Sons, 1998.

[58] K. Preis, O. Bíró, and I. Ticar. Gauged current vector potential and reetrant corners in the FEM analysis of 3D eddy currents. *IEEE Transactions on Magnetics*, 36:840–843, 2000.

[59] R. Otin, D. Milanesio, P. Jacquet, and I. Monakhov. Validation of ERMES 20.0 finite element code for JET A2 antennas coupling studies. *EPJ Web of Conferences*, 346:01027, 2026.

[60] R. Otin, G. Urbanczyk, and W. Tierens. Plasma-wave interaction modelling in magnetic confined devices with the finite element method. *The 64th Annual Meeting of the APS Division of Plasma Physics, Spokane (Washington, USA)*, October 17-21, 2022.

[61] R. Otin, J. Verpoorte, H. Schippers, and R. Isanta. A finite element tool for the electromagnetic analysis of braided cable shields. *Computer Physics Communications*, 191(1):209–220, 2015.

[62] R. Otin, R. Mendez, and O. Fruitos. A frequency domain approach for computing the Lorentz force in electromagnetic metal forming. *International Journal of Applied Electromagnetics and Mechanics*, 46(1):125–142, 2014.

[63] R. Otin, S. Aria, V. Thompson, R. Lobel, J. Willians, Z. Vizvary, D. Iglesias, and M. Porton. Computational electromagnetics for nuclear fusion engineering and design. *NAFEMS Benchmark Magazine*, pages 60–68, January 2020.

[64] R. Otin, T. C. Hender, G. Xia, O. Bardsley, and J. Paterson. Plasma control system design and disruption forces modelling with the open-source platform ERMES. *The 65th Annual Meeting of the APS Division of Plasma Physics, Denver (Colorado, USA)*, October 30-November 3, 2023.

[65] R. Otin, W. Tierens, F. Parra, S. Aria, E. Lerche, P. Jacquet, I. Monakhov, P. Dumortier, B. Van Compernolle, and JET Contributors. Full wave simulation of RF waves in cold plasma with the stabilized open-source finite element tool ERMES. *AIP Conference Proceedings*, 2254:050009, 2020.

[66] R. Otin, Y. H. M. Liang, T. Wilson, S. Freethy, and V. Hall-Chen. Validation of ERMES 20.0 finite element code for MAST Upgrade O-X mode conversion. *EPJ Web of Conferences*, 346:01009, 2026.

[67] J. R. Reitz, F. J. Milford, and R. W. Christy. *Foundations of Electromagnetic Theory*. Addison-Wesley, 1996.

[68] E. J. Rothwell and M. J. Cloud. *Electromagnetics*. CRC Press LLC, 2001.

[69] M. Salazar-Palma, T. K. Sarkar, L.-E. García-Castillo, T. Roy, and A. Djordjevic. *Iterative and Self-Adaptive Finite-Elements in Electromagnetic Modeling*. Artech House Publishers, 1998.

[70] T. B. A. Senior. Impedance boundary conditions for imperfectly conducting surfaces. *Applied Science Research, sec. B*, 8:418–436, 1960.

[71] T. B. A. Senior and J. L. Volakis. *Approximate Boundary Conditions in Electromagnetics*. IEE Press, 1995.

[72] P. P. Silvester and R. L. Ferrari. *Finite Elements for Electrical Engineers*. Cambridge University Press, 1983.

[73] T. H. Stix. *Waves in plasma*. Springer-Verlag, 1992.

[74] J. A. Stratton. *Electromagnetic Theory*. McGraw-Hill, 1941.

[75] D. G. Swanson. *Plasma waves*. IoP Publishing, 2003.

[76] R. K. Wangsness. *Electromagnetic fields*. John Wiley & Sons, Inc., 1986.

[77] J. P. Webb. Edge elements and what they can do for you. *IEEE Transactions on Magnetics*, 29:1460–1465, 1993.

[78] L. P. Wheeler. *Josiah Willard Gibbs: the history of a great mind*. Ox Bow Press, 1998.

[79] E. B. Wilson and J. W. Gibbs. *Vector analysis: A text-book for the use of students of mathematics and physics,founded upon the lectures of J. Willard Gibbs*. Charles Scribner's Sons, 1901.

[80] O. C. Zienkiewicz and R. L. Taylor. *El Método de los Elementos Finitos*, volume I y II. CIMNE-McGraw Hill, 1994.